

AGENTIC QUANT TRADING

Agentic Quant Trading

with Python

From HMM regime detection to SAC reinforcement learning

Broker-ready quant systems built with AI agents



forward-only HMM · Kalman spread · SAC agent · IBKR/KIS

Ted Park

Request | Build | Verify | Repeat

Agentic Quant Trading with Python

Request · Build · Verify —Shipping a quant trading system with AI agents

Copyright © 2026 Ted Park.

All rights reserved.

First edition, 2026.

Set in Apple SD Gothic Neo | Menlo.

Contents

Preface	1
How This Chapter Is Organized	1
Learning Goals	1
The Chart That Fooled Me	2
What Technical Analysis Looks Like Today	2
What Renaissance Technologies Figured Out	3
A Different Way of Seeing the Market	3
Why I Wrote This Book	4
What AI Coding Assistants Made Possible	4
How to Read This as a Machine-Learning Trading Book	5
A Map for When You Get Lost	6
A Walk-Through Example	8
Breaking Down the Model More Simply	8
What This Book Covers, and What It Doesn't	9
What You'll Be Able to Build by the End	10
Principles Drawn from the Renaissance Story	10
1. The Causality Principle	11
2. The Ensemble Principle	11
3. The Walk-Forward Validation Principle	11
The Road Ahead	11
A Note on Reading Backtest Results	12
Full Chapter Roadmap	13
Reading Differently on Each Pass	14
Minimum Goal by Part	15

Using AI as a Pair Programmer	16
Key Takeaways from This Chapter	16
Prerequisites	16
Chapter 0. Vibe Coding Quickstart: Verify One Small Example First	18
Establish the expected result first	18
Run the check directly from the text	19
How to ask AI for help	20
Check the order of events	20
Narrow down a failure	20
Before moving on	21
Chapter 1. System Architecture —The Design Philosophy of an AI Quant Trading System	22
What This Chapter Covers	23
Learning Goals	24
1.1 System Structure	25
Why Split Into Layers	25
The Same Architecture in Machine-Learning Terms	26
What Is a Model Here?	27
Reading Every Chapter the Same Way	28
1.2 Directory Structure	29
1.3 Core Components	30
SignalGenerator —The Shared Contract	30
SIGNAL_REGISTRY —The List of Registered Strategies	30
BacktestResult —The Standard Result Format	31
SAC —A Reinforcement Learning Agent Built From Scratch	32
1.4 The Causality Principle	32
1.5 Data Flow —The Order a Backtest Runs In	33
1.6 Setting Up the Development Environment	34
Python 3.12 Environment	34
Core Packages	34
Configuration File	35

1.7 Initializing the Data Cache	35
1.8 Running Your First Backtest	36
Exercises	37
Recap	38
Further Reading	38
Check your understanding	38
Chapter 2. The Data Pipeline —DuckDB and Live Feeds	39
How this chapter flows	40
Learning goals	40
2.1 The Big Picture —Data Flow	41
How raw data becomes features	42
2.2 <code>download_bars()</code> —One Download Entry Point	43
Intervals the function supports	43
Core flow —two-stage fallback	44
Confirming the fallback actually triggered	45
2.3 <code>DuckDBCache</code> —Parquet + In-Memory DuckDB	45
Why DuckDB?	45
Storage layout —monthly Parquet files	46
Handling concurrent access	46
Four steps inside the save function	47
2.4 <code>tz_localize(None)</code> —An Unwritten Rule of This Project	47
The correct pattern	48
2.5 Feature Engineering —Four Daily Features	48
What each of the four features looks at	48
The core calculation —dimension 0 (log return)	49
Missing Values —Empty Cells the Model Must Not Read	50
2.6 Kalman Smoothing —A One-Liner That Reduces Noise	51
2.7 Hourly Features —A 6-Dimensional Extension	51
2.8 Z-Score —A Mean-Reversion Signal	52
2.9 Feeding the Model —A Working Pattern	53
Exercises	54

Wrap-Up	54
Further reading	55
Chapter 3. The Backtest Engine —A Simulation You Can Trust	56
How this chapter flows	58
Learning goals	58
3.1 Signals, fills, and equity in time order	59
3.2 A runnable accounting exercise	59
3.3 Count each cost and final liquidation once	62
3.4 Performance — <code>_build_result()</code>	62
Volatility-adjusted performance	63
MDD and Win Rate	63
More refined metrics (catalog)	63
3.5 PnL Gini —a concentration metric	64
3.6 The Deflated Sharpe Ratio (DSR)	64
3.7 <code>PortfolioConfig</code> —recommended settings per strategy	65
3.8 How to read performance metrics	65
Exercises	66
Summary	67
Further reading	67
Chapter 4. HMM Basics —Inferring Market Regimes with Probability	69
HMM as a Machine-Learning Model	71
The Same Signal Means Different Things in Different Regimes	72
How this chapter is organized	73
Learning goals	73
What to focus on during a first read	74
4.0 Big Picture —the <code>hmm_daily_mr.py</code> pipeline	75
4.1 The three HMM ingredients —A, B, π	76
Transition matrix A	76
Emission distribution B	76
Initial state distribution π	77
4.2 Baum-Welch —intuition for training	77

4.3 <code>_DailyHMM.fit()</code> —the actual implementation	78
Why we use <code>covariance_type="full"</code>	80
The role of <code>min_covar=1e-3</code>	80
4.4 Sorting regimes by variance	81
4.5 <code>_DailyHMM.predict_proba()</code> —computing regime probabilities	82
4.6 <code>_update_hmm()</code> —refitting every 10 bars	83
4.7 <code>_check_entry()</code> —the Z-score plus HMM gate	84
4.8 <code>_check_exit()</code> —six exit conditions	86
4.9 <code>PureZscoreDailySignal</code> —a baseline	87
4.10 The full flow —call order in <code>on_bar()</code>	87
Exercises	88
What to remember from this chapter	88
Further reading	89
Check your understanding	90
Chapter 5. HMM Forward-Only —Real-Time Inference Without Future Information	91
Seeing Forward-Only in One Scene	94
How This Chapter Is Organized	96
Learning Goals	96
5.0 Big Picture — <code>hmm_core.py</code> Pipeline	97
5.1 Why <code>predict_proba</code> Causes Look-Ahead Bias	98
5.2 Forward Algorithm Recursion	99
5.3 <code>_BWHMM.forward_proba_causal()</code> —Initializing α After Training	100
5.4 <code>_BWHMM.step_forward()</code> —One-Step Online Inference	101
How the training and OOS periods connect	103
5.5 <code>_precompute_cov_fast()</code> —Speed Optimization	103
5.6 7-State Regime Structure — <code>_REGIME_A_SCALE</code>	104
5.7 <code>GlobalRegimeLayer.step()</code> —Reducing to a Single Multiplier	104
5.8 <code>dominant_state</code> —3-Bar Confirmation for Stability	105
5.9 A Walk-Forward Example	106
5.10 Transition risk	107
Exercises	107

What to Remember from This Chapter	108
Further Reading	109
Chapter 6. The Kalman Filter —Denoising and the Innovation Z-Score	110
What this chapter covers	113
Learning goals	113
6.0 The big picture —two roles for the Kalman filter	113
6.1 The local level model —the math, made intuitive	114
6.2 KalmanFilter1D —smoothing HMM features	116
6.3 Per-feature Kalman parameters for the 4-dimensional input	117
6.4 _kalman_innovations() —pulling innovations out of price	118
6.5 _build_z_series() —a causal Z-score	119
6.6 _calc_half-life() —estimating an AR(1) half-life	120
6.7 KalmanSelfMRSignal —entry and exit logic	121
A four-filter entry	121
A six-check exit	122
6.8 kalman_self_mr vs. hmm_daily_mr	123
6.9 A backtest run	124
Exercises	124
Things to remember from this chapter	125
Further reading	126
Chapter 7. Cross-Asset Joint HMM —Regime Detection Across Multiple Markets	127
What this chapter covers	129
Learning goals	130
7.0 The full Pipeline —six stages	130
7.1 Why Cross-Asset?	132
7.2 Bar-level noise reduction —Kalman filter	132
7.3 The 20-dimensional session observation vector	133
7.4 Layer 1 —Macro HMM (daily)	134
7.5 Layer 2 —Micro HMM (session)	135
7.6 The walk-forward setup	136

7.7 macro_bias -> Kelly modulation	136
7.8 Computing P(up) —how the up-probability is built	137
7.9 The structural choices behind the results	137
Try it in code —the Macro HMM training flow on synthetic data	138
Exercises	139
Summary	140
Further reading	140
Chapter 8. Two-Layer HMM Ensemble —Layer A × Layer B	141
Chapter Roadmap	143
Learning Objectives	143
8.1 Why Two Layers?	144
8.2 Effect First —Backtest Performance	145
8.3 Layer A: GlobalRegimeLayer	145
8.3.1 Regime Stabilization —Three-Bar Confirmation	147
8.4 Layer B: Overview of the Eight Sub-Models	147
8.4.1 HMM Sub-Models (Two Representatives)	147
8.4.2 Non-HMM Sub-Models (Two Representatives)	148
8.5 SubModelSpec —Declaring Sub-Models as Data	148
8.6 HMMForwardPredictor: Layer B HMM Predictor	149
8.6.1 fit() —What the Training Step Does	150
8.6.2 step() —Updating Expected Return Each Bar	150
8.7 W2 Wasserstein Voting —Suppressing Outliers	150
8.8 RenaissanceEnsemble5m.step() —Combining the Two Layers	151
8.9 Design Principles in Brief	152
A Look at the Code —Minimal Two-Layer HMM	153
Exercises	154
8.10 Key Takeaways	154
Further Reading	155
Chapter 9. Half-Kelly Position Sizing	156
Flow of this chapter	158

Learning goals	159
9.1 Kelly Criterion Basics	159
9.2 The structure of <code>calculate_kelly_size()</code>	160
9.3 <code>vol_mult</code> —shrinking the position when volatility rises	162
9.4 <code>macro_gate</code> —shrinking when the market state is unclear	163
9.5 <code>novelty_gate</code> —shrinking when the market looks unfamiliar	163
9.6 ER Amplifier —growing the position when signals agree	164
9.7 Adjusting macro bias by asset class	165
9.8 <code>fractional_kelly()</code> —the simple version for 5-minute bars	166
9.9 Comparing the two Kelly implementations	166
9.10 Why <code>KELLY_MIN = 0.10</code> ?	167
Walking through scenarios	168
Exercises	168
Summary	169
Further reading	169
Check your understanding	170
Chapter 10. HMM Forward Ensemble —V1 -> V3 Evolution	171
Why an Ensemble	172
How This Chapter Is Organized	173
Learning Goals	173
What to Focus On During Your First Read	174
10.1 Architecture Comparison —V1, V2, V3	174
10.2 The V1/V2 Shared Core —Computing $E[r]$ with Forward α	175
10.2.1 Avoiding the Causality Trap	176
10.3 What Each Version Added	177
Added in V2 —Sector & Order Flow	177
Added in V3 —Multi-Timeframe & Non-HMM	177
V3 Dynamic Weights —More Weight on Submodels That Did Well	178
10.4 V2 Key Idea —Optuna Threshold Re-Optimization	179
10.5 V2/V3 Performance Comparison Summary	180
10.6 Adaptive Threshold — $\sigma \times \text{scale}$	180

Try It Yourself —Threshold Optimization with Optuna (Optional)	181
Exercises	182
Wrap-Up	183
Further Reading	183
Chapter 11. Statistical Arbitrage —Pair Selection	185
What this chapter covers	187
Learning goals	187
11.1 Why pick pairs within the same sector?	188
11.2 The pair-selection pipeline	189
Cointegration test —judging a single pair	190
Full pipeline —testing every pair, sector by sector	190
11.3 Calculating the half-life	192
11.4 A dynamic hedge ratio with the Kalman filter	192
11.5 Pair-quality checks	193
11.6 Pair-selection parameters at a glance	194
11.7 Pair-selection criteria, summarized	195
Try it yourself —running the pipeline on a synthetic pair	195
Exercises	197
Summary	197
Further reading	198
Chapter 12. Statistical Arbitrage —Generating Signals	199
Exits come before entries	200
What’s in this chapter	201
Learning goals	202
12.1 Entry prerequisites	202
12.2 Entry score (<code>entry_score</code>)	203
Why multiplication?	204
A one-line numerical example	204
Convergence momentum (<code>conv_boost</code>)	205
12.3 Exit conditions (in priority order)	205
Which condition fires first —two scenarios	206

12.4 BW-HMM spread regime adjustment	207
12.5 Flow summary for entry and exit	208
12.6 Looking at the code —entry score and prerequisite checks	208
12.7 How to combine with RL (preview of the next chapter)	210
Wrap-up	211
Exercises	212
Further reading	212

Preface

How This Chapter Is Organized

This preface sets the direction for the book. We'll look at why chart-based judgment hits a wall, and then we'll see how this book builds a quant trading system.

- A time when a chart fooled me
- Why technical analysis struggles in today's market
- The key perspective drawn from Renaissance Technologies
- What this book covers and what it does not
- The system we'll build and what you need to prepare

Learning Goals

After reading this preface, you should be able to explain the following.

- Why this book starts with data pipelines and validation rather than chart patterns
- Why “secretly peeking at future data (Look-ahead Bias)” is fatal in a quant system

- What roles HMM, reinforcement learning, and statistical arbitrage play in this book
- What basic ML terms like training, inference, features, and missing data mean inside a trading system
- How much Python and statistics knowledge you need before reading this book

The Chart That Fooled Me

I once stared at a candlestick chart for a long time and decided it was the right moment to buy. The price had bounced off the same support level three times, RSI was in the oversold zone, and the MACD golden cross looked exactly like the textbook example. Several signals seemed to point the same way, and I trusted them and bought.

The next day the price sliced right through that “perfect” support level. It wasn’t the first time, and it wasn’t the last. But after that day my question shifted a little. Was I simply bad at reading charts, or were the tools I’d been leaning on simply not built for today’s market?

What Technical Analysis Looks Like Today

For the next few months I read papers, browsed trading forums, and looked through performance data, asking the same question over and over. The short answer: the tools I’d been relying on were not enough to explain today’s market.

Traditional technical analysis means tools like candlestick patterns, moving-average crossovers, Bollinger Bands, and RSI. These tools were created in an era when markets moved slowly and human psychology drove most of the price action. Back then they actually worked. Today is different.

In today’s market, algorithms place orders far faster than people can, and they account for a large share of trading volume. Patterns that once reflected crowd psychology are now signals that machines detect first and act on first.

When a support level shows up on a chart, humans aren’t the only ones who see it. Algorithms see it too. The spot where stop-loss orders pile up is also where liquidity pools, and that zone

often gets deliberately tested. A signal that worked well in the old market may not behave the same way in today's market.

I'm not trying to be pessimistic. I'm just pointing out that many of the tools individual investors commonly use were built around a different kind of market than the one we have now.

That's why this book doesn't start with "how to read charts better." Instead, it treats the market as a process that produces data. From there, we move toward estimating the hidden states inside that process and checking whether those estimates accidentally peeked at future data.

What Renaissance Technologies Figured Out

The place that pursued this approach first, and most rigorously, is Renaissance Technologies. In 1988, the mathematician Jim Simons founded a hedge fund called Renaissance Technologies. Instead of hiring traders who read stock charts, he hired mathematicians, physicists, and computer scientists. The goal wasn't to analyze candlestick shapes. It was to find statistical patterns that repeat inside price data. The fund that came out of this work, the Medallion Fund, has posted one of the strongest long-term track records in investing history over several decades.

A Different Way of Seeing the Market

Simons didn't have some special inborn intuition. The difference was in how he looked at the market.

Renaissance didn't view the market as something governed by one fixed law. Instead, they saw the market as moving between several "hidden states." A hidden state is a regime that doesn't show itself directly but still controls how the market behaves. In some periods a trend keeps going strongly; in others, prices that went up snap back to the average; in others, volatility suddenly explodes.

You can't see these states directly. But you can estimate them from the data. If you can figure out which state the market is in right now, you can pick a strategy that fits that state.

The HMM (Hidden Markov Model) is the tool that handles exactly this “estimate the invisible state from data” problem. We’ll cover it in detail in the middle of the book.

I’m not an insider at Renaissance Technologies, and I don’t know their exact methodology. But I was able to implement part of their philosophy in code by drawing on public interviews and academic research. This book is a record of that process.

Why I Wrote This Book

Quant trading is hard to learn. Not because each individual concept is impossible, but because too many pieces arrive at once. You need statistics, signal processing, Python implementation, a backtest engine, broker integration, and risk management to work together.

Among all of that, one thing is the trickiest. **Building a system that doesn’t fool itself.**

There’s a trap called “Look-ahead Bias.” It’s the mistake of secretly using future data while building a strategy. It hides in many places in your code, and it’s not easy to spot. Without careful validation, a strategy that peeked at the future can look like it produced amazing results. Returns that beat the market average are usually called “alpha,” but what looks like alpha may just be the result of peeking at future data.

Most people who want to get into quant trading give up before they’ve built this foundation. Then they fall back to candlestick charts and RSI. Not because those tools are enough, but because they are easy to reach for. I wanted to close that gap. Without hiding the math or the implementation, I wanted to lay the material out in an order a reader can actually follow.

What AI Coding Assistants Made Possible

Since 2023, AI coding assistants have reached a practical level. That’s what made this book possible. Tools like GitHub Copilot, Claude, and ChatGPT help explain complex algorithms, find subtle implementation bugs, and unpack what a formula means in code. They don’t do the understanding for you. But they sharply cut the time and trial-and-error it takes to get there.

I built the system in this book with AI as a working partner. When I got stuck on difficult math, when I was hunting subtle bugs, when I was picking hyperparameters (the settings that control

how well a model performs), AI helped at every step. This book walks through that process from the beginning. The goal is not to stop at theory. It is to show how to build a real system with the tools you can use right now.

How to Read This as a Machine-Learning Trading Book

This book covers HMM and reinforcement learning, but more broadly it is a learning guide for building a stock-trading system with machine learning and deep learning.

The core flow is simple.

```
stock market
-> data
-> features
-> machine-learning model
-> deep-learning / RL agent
-> validation
-> order execution
```

The word **model** can feel vague at first. Intuitively, think of it this way: a model is not a magic box that predicts tomorrow's price. It is a function whose internal numbers were learned from past data. A normal function has rules written directly by a person. A machine-learning model has the same input-output shape, but the internal weights are fitted from data.

```
feature row today -> trained model -> regime probability / action
proposal
```

If the word **model** still feels vague, use this simpler image.

model = a decision calculator that has learned its internal numbers from data

A normal calculator returns 5 when you enter $2 + 3$. A trading model receives numbers such as return, volatility, volume, and z-score, then returns a judgment such as “mean-reversion regime 70%” or “use 35% of the allowed position size.” It is not the data itself. It is not just the

algorithm name. It is the learned decision function the algorithm produces after training on data.

```
def model(today_features):  
    return prediction
```

That is why this book separates three words carefully.

Word	Easy meaning	Example in this book
Algorithm	The method used to build a model	HMM Baum-Welch, SAC training
Parameter	A number learned inside the model	HMM transition matrix, neural-network weights
Model	The learned decision function	today's features -> regime probabilities

As a cooking analogy, data is the ingredient, the algorithm is the recipe, parameters are the heat level and seasoning ratios, and the model is the finished way of cooking after those ratios have been fixed. The same HMM algorithm can produce different models depending on which market period it trained on.

Training means fitting the model on historical data. Inference means freezing that trained model and feeding it new data one row at a time. OOS validation hides later data from the model during training, then checks whether the trained model still works there.

At first, do not try to understand every algorithm name at once. Hold on to this backbone: **data -> features -> training -> inference -> validation -> orders**. HMM, SAC, risk management, and MLflow all attach to that one backbone.

A Map for When You Get Lost

Here is the same idea in trading language.

Machine-learning word	Trading meaning in this book
Data	Historical prices, volume, macro indicators, and broker records
Feature	A number the model can read, such as return, volatility, momentum, z-score
Training	Letting the model fit its internal numbers from past data
Inference	Asking the trained model to judge today's new row
OOS	The exam period hidden from the model during training
Serving	Running the trained model as a service that the trading engine can call
Risk layer	The final guard that can block or shrink the model's proposal

If you get lost later, return to this table. Most of the harder chapters are really just one row of this table explored in more depth. Chapter 2 builds data and features. Chapters 4-5 train and infer HMM regimes. Chapters 13-14 train an RL agent. Chapter 16 limits risk. Chapter 27 serves the trained model.

For the first pass, use this checklist.

- ☐ Can I explain what data goes into the model?
- ☐ Can I explain what feature is created from that data?
- ☐ Can I separate training data from inference data?
- ☐ Can I point to the OOS region?
- ☐ Can I say what the model outputs?
- ☐ Can I say what risk check happens before an order?

If you can answer those questions, you are following the real structure of the book even if HMM, SAC, Kelly, or CVaR still feel unfamiliar. The details become much easier once the pipeline is clear.

A Walk-Through Example

A concrete example helps. Suppose you have five years of daily data. A beginner-friendly split might train the model on the first two years and run inference on the next three years. The second part is still historical data on your disk, but during validation it must behave like future data. The model should not fit on it. It should only receive one feature row at a time and produce an output.

```
5 years of features

first 2 years: training data
  -> fit scaler
  -> fit model
  -> save model parameters

next 3 years: OOS inference data
  -> load trained model
  -> feed one row at a time
  -> record model output
  -> pass through risk checks
```

Breaking Down the Model More Simply

This is also the easiest way to understand a model file. A model file is not the training data itself. It is the result of training: the learned parameters, the scaler, the mapping rules, and sometimes the last state needed to continue inference. Serving means loading that model file and making it answer requests.

```
training creates the model file
serving loads the model file
inference sends new features into the loaded model
risk decides whether the model output can become an order
```

The same pattern appears throughout the book. HMM training creates regime parameters. HMM inference reads today's regime probability. SAC training creates an actor. SAC inference proposes an action. MLflow records which model file came from which training run. Risk management keeps the model output from becoming an oversized order.

In live trading, training and inference run on two separate cadences.

```
Slow clock: training / validation / model registration
historical data -> training -> OOS validation -> save model file ->
approve

Fast clock: real-time inference / order flow
today's data -> features -> load model file -> inference -> risk check
-> order -> fill check -> monitoring
```

During the trading day, the model is not a student studying again every minute. It is closer to a calculator in the exam room. The learning already happened elsewhere. The live system loads the approved model file, sends today's feature row into it, receives the output, then lets the risk layer decide whether that output can become an order.

A saved model file therefore means more than "one file on disk." It fixes the input format, the learned numbers, and the meaning of the output together. If training used [return, volatility, momentum, volume] but inference sends [return, volume, volatility, momentum], the model still receives four numbers, but their meanings are wrong. This is why later chapters keep returning to feature order, scalers, regime label mapping, serving, and production validation.

What This Book Covers, and What It Doesn't

Let's draw the boundary of this book first.

What this book is not

- It's not a Python beginner book. It assumes you can write loops and import libraries.
- It's not a chart-trading manual. It doesn't cover chart-based technical analysis like support lines or MACD divergence.
- It doesn't promise quick riches. Quant trading is hard, and the risk is real.

What this book is

It’s a hands-on guide to building a quant trading system from scratch. We’ll construct a system in the style used by quant hedge funds like Renaissance Technologies, step by step, with help from AI. The code in this book actually runs, and the results are reproducible numbers.

What You’ll Be Able to Build by the End

System	One-line description
HMM market-regime detector	Estimates whether the market is in an up, sideways, or down regime using only the data available so far —no peeking at the future.
Multi-asset HMM	Reads signals from several assets such as stocks and bonds together, for a more accurate regime read.
Two-layer macro/micro HMM	Looks at the macro economy and individual assets in two layers and classifies the market as Risk-On, Neutral, or Risk-Off.
Reinforcement-learning trading agent	Takes the regime detected by the HMM as input and decides positions on its own.
Statistical arbitrage system	Finds pairs of assets that move together and trades the gap between their prices.
Production deployment system	Connects to brokers like IBKR (Interactive Brokers) and KIS (Korea Investment & Securities) to place real orders.

The real goal of this book is to understand why each part is needed, and how it breaks when built wrong.

Principles Drawn from the Renaissance Story

Three principles run through this entire book, all drawn from how Renaissance Technologies approaches trading.

1. The Causality Principle

Future data must never be used in a current decision.

It sounds obvious, but it is surprisingly easy to violate when you write code. Even widely used library functions sometimes mix future information into the result. In this book, instead of relying on those functions, we directly implement an approach (the Forward Algorithm) that keeps the promise of “only look at data available up to now.” Chapter 5 explains the reasoning in detail. Once you understand it, you will start spotting the same kind of problem in other code.

2. The Ensemble Principle

Don't depend on a single model.

This system uses a daily-bar HMM, an hourly-bar HMM, a reinforcement-learning agent, and a statistical-arbitrage signal together. When the models disagree, we wait. When they all agree, we act.

3. The Walk-Forward Validation Principle

The only way to confirm a strategy works is to test it repeatedly, in time order, on data the model has never seen.

Splitting the data once into a training set and a test set isn't enough. Walk-forward validation is the baseline standard applied to every strategy in this book.

The Road Ahead

We'll stack the whole system in this order: data -> noise reduction -> regime detection -> position sizing -> validation and live operation. Each step ties to a cluster of chapters.

Step	What we do
1	Collect and store market data
2	Reduce the noise mixed into prices
3	Use a multi-asset HMM to estimate the market regime
4	Sort the regime into Risk-On / Neutral / Risk-Off using two layers, macro and micro
5	Decide how large a position to take based on the regime
6	Look for extra opportunities through statistical arbitrage
7	Have a reinforcement-learning agent help with position decisions
8	Add risk management, broker connection, and monitoring for live operation

On the first read, think of Chapters 1-18 as **one pass through the basic system from end to end**. Chapters 19-27 go deeper into the same material. They cover pair theory, the complete HMM pipeline, production validation, distributional RL, and MLflow serving —the topics you'll come back to when you start actually running the system.

A Note on Reading Backtest Results

This book shows several backtest results, but no single number should be read as a promise of performance. Even the same strategy can produce very different results depending on the data period, trading costs, the universe of symbols, the retraining frequency, and the slippage assumptions.

On your first read, look at these three things rather than the numbers themselves.

- Did it avoid using future data?
 - Was maximum drawdown shown alongside trading costs?
 - Does the pattern still hold on out-of-sample data?
-

Full Chapter Roadmap

Part	Chapters	Topic
I	Preface ·1-3	The full system picture, Look-ahead Bias, data pipeline, backtest engine
II	4-5	HMM basics and learning algorithm, inferring without using future information
III	6-8	Kalman filter (noise reduction), multi-asset HMM, two-layer macro/micro HMM (the heart of the system)
IV	9-10	Position sizing, making directional signals with an HMM ensemble
V	11-12	Finding pairs that move together and building statistical-arbitrage signals
VI	13-15	Reinforcement-learning environment, SAC agent, hyperparameter tuning
VII	16-18	Risk management, broker integration, real-time monitoring
VIII	19-27	Deep dives: pair theory, regime-based Z-score, full pipeline, production validation, distributional RL, MLflow serving

Most of the terms in this table will look unfamiliar at first. Don't try to memorize them now. On the first read, take one loop through Parts I-VII to go from "data to orders and monitoring." Treat Part VIII as the deeper section you turn to on a second read, or when you start polishing a real system.

Reading Differently on Each Pass

On the first pass, read for the flow. Do not stop because one equation is unclear. Ask only where the data enters, what the model outputs, and where risk blocks an order. On the second pass, return to the code and formulas. On the third pass, start replacing the examples with your own symbols, costs, and validation periods.

This book is not meant to be fully absorbed in one pass. If you try to understand every parameter and every implementation detail from the beginning, you will lose the main flow. Use a different goal for each pass.

Pass	Goal	How to read
First pass	Grasp the big flow	Watch how data becomes model judgment and then orders
Second pass	Connect code and concepts	Check each chapter's input, output, and failure point in code
Third pass	Experiment and modify	Change parameters and see how OOS results move

On the first pass, it is fine if the formulas are not fully clear. Instead, ask which of the following questions the chapter is answering.

Is this chapter preparing data?
Is it reading the market?
Is it finding an opportunity?
Is it deciding size?
Is it blocking risk?
Is it checking operational health?

On the second pass, drill into the code. Check where `fit()` is called, where `predict()` or `step_forward()` is allowed to run, and whether the OOS region is kept out of training. At that point, equations stop being something to memorize and become a map explaining why the code is shaped the way it is.

On the third pass, experiment. Change window length, entry threshold, number of regimes, trading costs, and risk limits. But do not keep tuning against the same OOS result. If you change a value, write down the hypothesis behind the change and keep validation and final test periods separate.

Minimum Goal by Part

On the first read, each Part has one sentence you should be able to explain.

Part	Minimum understanding goal
I	Build the foundation so data and backtests do not lie
II	HMM reads market state as probabilities, not future price
III	Noise reduction and multi-market views make regime judgment more stable
IV	A signal gives direction, and Kelly turns that direction into position size
V	Pairs trading trades the relationship between two prices, not the direction of one stock
VI	RL is not a magic AI that creates every signal; it adjusts position size
VII	Even if the model is right, bad risk, orders, or monitoring can break the live system
VIII	Re-check the same material more strictly and shape it into an operational system

Use this table as the first-read standard. If you can explain the minimum goal of each Part, the first pass has done its job even if some formulas still feel difficult.

Using AI as a Pair Programmer

Throughout this book, I'll show directly how I used an AI coding assistant whenever I got stuck. The prompts I used to understand an algorithm, the subtle bugs the AI found, and the moments where AI nudged me toward a better direction are all woven into the text.

The goal isn't to hand over your thinking to the AI. **Treat it like a strong senior colleague.** Ask about what you don't know, confirm what you think you understand, move quickly through parts you already know, and work through the hard parts together.

Quant trading isn't a domain reserved for PhDs alone. With the right tools and direction, you can build a practical trading system yourself. This book is a record of doing exactly that.

Key Takeaways from This Chapter

- This book isn't about memorizing chart patterns better. It's about building a system that sees the market through data.
- The single most important promise is **“don't use future data in a current decision.”** If that promise breaks, every impressive-looking result is fake.
- Don't stake everything on a single model. Act only when several models point the same way.
- You don't need to master all the math up front. Each chapter walks you through “why this concept is needed” and “what goes wrong if you build it incorrectly,” in order.

Prerequisites

Before we dive in, you should have at least the following ready.

- **Python 3.10 or higher**
- **Basic NumPy and Pandas experience** —enough to handle arrays and manipulate dataframes.

- **Basic statistics concepts**—knowing what mean, variance, and the normal distribution are is enough. Harder concepts are explained inside the book when they're needed.
- **Curiosity about why the market moves the way it does**

If you're ready, the next chapter rolls out the blueprint for the entire system.

Chapter 0. Vibe Coding

Quickstart: Verify One Small Example First

The goal of this chapter is to **check whether the profit and loss explained by AI is correct**, using a small numerical example. You do not need the author's production repository, a broker account, or market data. If you have Python, run the code below. Otherwise, calculate the table by hand.

Establish the expected result first

Start with 1,000 in cash and buy 10 units of an asset priced at 100. The price then falls to 80 and returns to 100. Ignore costs for this first calculation.

Point in time	Cash	Quantity	Price	Equity
Before the trade	1,000	0	100	1,000
After buying	0	10	100	1,000
After the price falls	0	10	80	800
After the price recovers	0	10	100	1,000
After closing the position	1,000	0	100	1,000

Equity is $\text{cash} + \text{quantity} \times \text{price}$. The final profit is zero, but the maximum drawdown is $(1000-800)/1000=0.2$, or 20%. Looking only at the final balance hides the risk taken along the way.

Run the check directly from the text

Paste this into a new file named `first_check.py`. It uses only Python's standard library; no downloads are required.

```
prices = [100.0, 80.0, 100.0]
initial = 1000.0
quantity = initial / prices[0]
cash = initial - quantity * prices[0]
equity = [initial] + [cash + quantity * p for p in prices]
peak = initial
mdd = 0.0
for value in equity:
    peak = max(peak, value)
    mdd = max(mdd, (peak - value) / peak)
final_cash = cash + quantity * prices[-1]
assert final_cash == 1000.0
assert mdd == 0.2
print(final_cash, mdd)
```

Run `python3 first_check.py` in a terminal. The output is `1000.0 0.2`. On Windows, use `py first_check.py` if your Python command is `py`. This checks profit and loss for a fixed quantity. It does not generate signals, place real orders, or train an HMM.

How to ask AI for help

Review the table and Python code together.

1. Calculate equity and maximum drawdown by hand.
2. Explain what risk is missed if only cash is recorded.
3. Calculate the final cash if buying and selling each incur a 10 bp fee.
4. Do not add a broker, external data, or new packages.

After answering, identify the table values you used to verify your answer.

Ten basis points (10 bp) is 0.1% of the trade value. Buying and selling 10 units at 100 costs 1 each time, or 2 in total, leaving 998 in cash. Because this example uses the entire initial balance to determine quantity, cash temporarily becomes negative by the purchase fee of 1. An actual account that forbids borrowing must buy fewer units to leave room for the fee. Distinguish a fixed-quantity exercise from real account constraints.

Check the order of events

A signal based on today's closing price cannot be filled at today's earlier opening price. The book's basic exercise uses **finalize the signal after bar t closes -> fill at bar t+1's open -> value the position at bar t+1's close**. It is valid to include that day's profit or loss for a position opened using a previously finalized signal.

Narrow down a failure

Symptom	First check
Python command not found	Check whether the installed command is python3 or py
Indentation error	Check that you copied only the code and preserved its indentation
Drawdown is zero	Check that equity of 800 at a price of 80 was recorded
Final cash and total profit do not reconcile	Check for profit added twice or duplicated costs

Compare the failing lines with the example before installing more packages. Prioritize agreement between the expected result and actual output over how convincing the generated answer sounds.

Before moving on

- Explain why zero final profit and a 20% maximum drawdown can both be correct.
- Charge each fee once per actual fill.
- Distinguish signal generation time from execution time.

Once these checks make sense, continue to the architecture in Chapter 1. The `accounting-lab` ZIP is optional material with additional tests and CSV files; its availability must be checked on the seller's delivery page. Names such as `app.trading` describe examples from the author's system architecture. Understanding the concepts does not require access to that private package.

Chapter 1. System Architecture —The Design Philosophy of an AI Quant Trading System

Part I —The Foundation (Chapters 1-3) Before building a strategy that makes money, this part builds the foundation that keeps the strategy from fooling you. Chapter 1 lays out the full blueprint, Chapter 2 covers the data pipeline, and Chapter 3 covers the backtest engine. On a first read, all you need is the order: data goes in, and a verifiable backtest result comes out.

When I first learned to read candlestick charts, I thought I had found a way to read the market. I drew support and resistance lines, memorized candle patterns, and studied formations like doji, hammer, and engulfing patterns. The more I studied, the more shapes I saw. I still remember the moment the chart seemed to finally make sense.

Real trading was different. Some days the price bounced off a line I had drawn as support. Other days it sliced straight through. The same candle pattern worked one day and reversed the next. It was hard to tell whether the patterns were really in the chart, or whether I was reading them in after the fact.

I figured out the reason later. Algorithms account for more than 70% of market volume. Those algorithms know where individual investors draw their support lines. They know that stop-loss orders (orders that automatically sell once losses pass a certain level) cluster around those lines. So the algorithms drive the price to that exact point and then reverse, or tap it just hard enough to trigger forced selling before pushing the price right back up. Chart patterns work and fail for the same reason —automated systems already know about them.

Once you accept this, you have two choices: keep watching charts and hope the algorithms do not take advantage of you, or change the way you look at the market entirely.

I chose the second, and this book is the result. The central idea of the book is simple —**we do not try to predict the price itself. Instead, we infer the market regime hidden behind the price.** The tool for that job is the HMM (Hidden Markov Model). The details come in later chapters.

Before AI, building a system like this on your own was nearly impossible. You needed statistics, a data pipeline, a backtest engine, an execution system, and monitoring, and you had to build them alone. Today, with AI alongside you, one person can build this architecture end to end. This book walks through that process from start to finish.

Chapter 1 covers the design philosophy of the system as a whole. Once you see why each component sits where it does and how the pieces connect, the rest of the book is much easier to follow.

What This Chapter Covers

We start with the big picture, then drop down into environment setup and a first run.

- System structure —how the five layers stack
- Directory structure —what each folder is for
- Core components —the shared contract every strategy follows
- The causality principle —the rule that keeps future data out
- Data flow —the order in which a backtest runs
- Setting up the dev environment and running your first backtest

Learning Goals

By the end of this chapter, you should be able to:

- Draw the path data takes through the five-layer architecture
- Say in one sentence what the HMM signal, the SAC RL agent, and the StatArb signal each do
- Explain where data, features, training, inference, and order execution fit in the five-layer architecture from a machine-learning perspective
- Explain what look-ahead bias (the bug of accidentally using future data) is and why it is dangerous
- List the registered strategies and run your first backtest

What this chapter builds toward: You will not write a single line of strategy code, but by the end of this chapter the command below actually runs.

```
python -m app.trading.backtest.runner --signal hmm_daily_mr --days 365
```

Running it prints numbers such as score (return relative to risk/volatility), Sortino (return relative to downside risk), and MDD (maximum drawdown). The rest of the book explains how all of this works. For now, focus on the overall structure.

1.1 System Structure

Why Split Into Layers

If you write a strategy as one big script, data collection, signal generation, and order execution all blend together in a single file. A simple request like “let’s swap the data source and rerun” then forces you to touch the whole codebase. Splitting responsibilities into layers means you can change one layer without disturbing the others.

This system is split into five layers. Each layer can be tested on its own, and adding a new strategy does not require touching the layers below.

Layer	Role
Layer 5 —Monitoring	Show results on a dashboard
Layer 4 —Execution & Risk	Send orders and manage risk
Layer 3 —Ensemble	Combine multiple signals into a final position
Layer 2 —Signal generation	Produce buy and sell signals from HMM, reinforcement learning, and statistical arbitrage
Layer 1 —Data	Pull in price data and turn it into features

Data flows from the bottom up. Layer 1 takes raw price data and turns it into features. The signal generators in Layer 2 read those features and produce buy and sell signals. Layer 3’s ensemble combines the signals into a final position. Layer 4 sends the orders, and Layer 5 shows the results.

HMM, SAC, and StatArb appear in turn in later chapters. They are not competing techniques—they are **components, each playing a different role in a different layer**.

Another reason to split the system into layers is that it helps you find failures faster. When returns get worse, do not immediately say “the model is bad.”The data may be broken, the feature order may have changed, or only part of an order may have filled.

Layer	Responsibility	Failure it should catch
Layer 1 —Data	Turn market records into features	Missing values, timezone errors, stale data
Layer 2 — Signals	Generate HMM / pairs / RL judgments	Future data leakage, wrong regimes, excessive signals
Layer 3 — Ensemble	Combine several judgments	Over-trusting a single model's misread
Layer 4 — Execution & Risk	Final safety check before orders	Oversized positions, failed stops, fill mismatches
Layer 5 — Monitoring	Watch operational health	Disconnections, performance decay, missing alerts

If you're new to this, you can read the architecture as the order in which the market gets broken down into pieces.

1. Record the market	Data
2. Turn it into numbers	Features
3. Read the state	HMM
4. Find an opportunity	Pairs / Z-score
5. Choose the size	SAC / RL
6. Block bad risk	Risk
7. Send the order	Broker
8. Keep watching	Monitoring

Machine learning is mainly used at step 3 to read the market state. Deep learning / RL is mainly used at step 5 to choose the action size. If you try to understand both at once, the system feels larger than it is. First separate them like this: **HMM reads the market, SAC chooses the action size.**

The Same Architecture in Machine-Learning Terms

You can also read the five layers as one machine-learning pipeline.

Here, a model is not just one file or one formula. Intuitively, think of it as a **decision calculator**: feed it market numbers and it returns a market state, a signal, or a suggested position size.

Layer 1 turns raw market data into **features**. Layer 2 runs **model inference** on those features. Layer 3 combines model outputs into a decision. Layer 4 and beyond sit outside machine learning—they are risk, execution, and operations, where real money is on the line.

Training fits the model's internal values on past data. Inference takes that trained model and feeds in today's features. Because live trading never knows the future, inference must use only data up to the current moment.

In this system, HMM reads the market state, pairs trading finds the opportunity, SAC adjusts action size, and risk management decides whether the account is allowed to take that action. If you remember only one sentence from this chapter, make it this: **models suggest; risk and execution decide what actually happens.**

What Is a Model Here?

The word “model” can sound vague. In this book, read it very concretely.

input numbers → model → output numbers

An algorithm is the general method, such as HMM or SAC. A trained model is what you get after that algorithm has seen a specific training window and fitted its internal parameters. Two HMMs trained on different years can be the same algorithm but different models.

Training means fitting those internal values on past data. For an HMM, that means learning transition probabilities and the observation distribution for each hidden state. For SAC, it means adjusting neural-network weights. Inference means taking an already trained model, feeding in today's features, and asking for its output without letting future data shape the answer.

So when you pick up any chapter in this book, ask three questions first.

1. What goes into the model?
2. What comes out of the model?
3. Which data was allowed to shape the model?

This is often more useful than asking whether the model is “smart.” A model with clean inputs, a clear output, and honest OOS validation is useful. A model with leaked future data is not useful, even if its backtest looks impressive.

On a first pass through the architecture, use this single sentence to describe every component:

This layer receives ____, changes it into ____, and passes it to ____.

That one sentence keeps the system from collapsing into a jumble of names. Data becomes features. Features become model outputs. Model outputs become position proposals. Position proposals pass through risk and execution. This is the whole book in miniature.

Reading Every Chapter the Same Way

The chapters in this book have different names, but they share the same underlying structure. Before reading a chapter in detail, fill in these four boxes.

What to check	Question
Input	What data does this chapter’s code receive?
Transformation	What number does it turn that data into so a decision can be made?
Output	What signal, probability, size, or state comes out?
Risk	If this part is wrong, how does the backtest or live system break?

For example, Chapter 6 on the Kalman filter looks like this.

Item	Answer
Input	Noisy price or feature values

Item	Answer
Transformation	Decide how much to trust the new observation and smooth it
Output	Less noisy features, innovation Z-score
Risk	Mistaking noise for a regime change

The IBKR connection in Chapter 17 can be read the same way.

Item	Answer
Input	Order intent that has passed the risk layer
Transformation	Submit it to the broker and confirm the response
Output	Filled order, position, cash update
Risk	Thinking an order happened when it did not, or missing a partial fill

Read this way, the machine-learning chapters and the operations chapters are not separate subjects. They all follow the same pattern: **input -> transformation -> output -> validation**.

1.2 Directory Structure

You do not need to memorize the file layout. As a beginner, just remember the role of the four groups below.

```
stock_trading_ai_py/
+-- app/trading/
|   +-- backtest/          # The engine that validates strategies
|   +-- backtest/signals/  # HMM, StatArb, and RL signal definitions
|   +-- rl/                # SAC reinforcement learning implementation
|   +-- cache/             # Price data cache
+-- refresh_cache.py       # Refresh the data cache
```

```
+-- pyproject.toml          # Python dependencies
```

`backtest/` owns execution and accounting, `signals/` owns causal decisions, `rl/` holds the reinforcement learning model, and `cache/` owns data access. These directory names illustrate responsibilities. You can follow the input across those boundaries without reconstructing the original file tree.

1.3 Core Components

SignalGenerator —The Shared Contract

If every strategy had a different input and output format, the backtest engine could not treat them the same way. So every strategy follows the same contract (interface). No matter what the strategy does, it runs in two steps.

1. `initialize()` —Called once when the strategy starts, to prepare data.
2. `on_bar()` —Called every time a new bar (one slot of price data) arrives, returning a buy or sell signal.

The minimum strategy contract has two stages: prepare from past data, then return a target signal using observations available so far. Inputs include bar time, finalized observations, and current holdings. Outputs include direction, target size, effective time, and rejection reason. A separate engine handles fills and accounting so the signal function cannot modify the balance directly.

SIGNAL_REGISTRY —The List of Registered Strategies

`SIGNAL_REGISTRY` maps strategy names to actual implementation classes. What you should remember is not the individual names but the pattern: **you pick which implementation to run by passing one string**.

The main signals fall into three groups.

Group	Examples	Role
HMM family	<code>hmm_daily_mr</code> , etc.	Estimate the market regime
StatArb family	<code>stat_pair_rl</code> , etc.	Build pair and mean-reversion signals
Combined family	<code>regime_orchestrator</code> , etc.	Combine multiple signals

To see the full list of registered strategies, run:

```
python -m app.trading.backtest.runner --list
```

BacktestResult —The Standard Result Format

If every strategy reported results in its own format, comparison would be painful. `BacktestResult` is the object that holds every strategy's performance in the same format. At first, focus on these four numbers.

Metric	Meaning	Why look here first
<code>score</code>	Return relative to risk/volatility	The first-pass measure of strategy quality
<code>mdd</code>	Maximum drawdown	How far the account fell from its peak
<code>win_rate</code>	Share of winning trades	A sanity check that the signal isn't just noise
<code>annual_return</code>	Annualized return	The actual scale of expected returns

Metrics like `sortino`, `cvar_95`, and `calmar` come up again later. In Chapter 1, all you need to take away is that **every strategy's result lands in the same standard format**.

Minimum capital: `capital = $100,000` is the default, but buying 500 shares of SPY needs more than \$300k. If the account does not have enough, position sizing breaks.

Either change `capital` to match your real account size, or set a per-symbol cap with the `max_position_usd` parameter.

SAC —A Reinforcement Learning Agent Built From Scratch

The reinforcement learning part of this book does not use a general-purpose library like `stable-baselines3`. Instead, we **implement SAC (Soft Actor-Critic) directly in PyTorch**, so we can shape the details for trading.

Chapters 13 and 14 dig into the architecture. For now, just keep the shape of the input and output in mind.

- **Input:** 28 market features
- **Output:** a single position size

The full training configuration lives in `app/trading/rl/config.py`.

1.4 The Causality Principle

WARNING

Using future data in a present-time decision inflates backtest performance and produces negative returns in real trading. Only fit on training data, and only use data up to time t for inference.

Look-ahead bias is the most common bug that inflates backtest returns artificially. It happens when information from time $t + 1$ or later leaks into a decision being made at time t . In a backtest you already have access to future data, so it leaks in by accident very easily.

The two traps beginners hit most often are:

Trap 1 —Letting the HMM see future observations

`hmm.predict_proba(X_all)` looks convenient, but it leaks future data. The function processes the entire time series at once and recomputes the regime probability at every step, so the value at time t ends up influenced by data from after t . The result at time t must be

computed using only $x_1, \dots, x_t$. This book uses only `_do_forward_pass` to enforce that rule (the actual code is in Chapter 5).

Trap 2 —Fitting the scaler on the whole period

```
# Incorrect/unsafe example.
scaler = StandardScaler()
scaler.fit(X_all)

# Correct pattern.
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

If `fit` sees future data when computing the mean and standard deviation, the backtest looks great—but in live trading you would never have access to that mean. **The rule is simple — call `fit` only on the training segment, and call `transform` on the test segment.**

Walk-forward validation: instead of a single split, you repeat the split in time order. In this project `OOS_RATIO=0.40` uses the last 40% of the period as the OOS (Out-of-Sample, the validation segment never used for training) window. Chapter 3 covers this in detail.

1.5 Data Flow —The Order a Backtest Runs In

A single backtest runs in four steps.

1. Collect the time intervals (1d, 1h, 5m) that the chosen signals need.
2. Read the required price data from the cache, or download it fresh.
3. Run each signal on the same data under the same conditions.
4. Return the results as a list of `BacktestResult` objects.

The comparison runner supplies the same data snapshot, evaluation interval, starting equity, costs, and position limits to each strategy, then collects results by strategy ID. Even in parallel execution, each strategy needs its own account state. Fix inputs so one strategy cannot change another strategy's cache or random state.

1.6 Setting Up the Development Environment

Python 3.12 Environment

You only need to create the virtual environment once. Run these four lines in order.

```
cd stock_trading_ai_py
uv venv .venv --python 3.12
source .venv/bin/activate
uv sync
```

`uv sync` automatically picks the CUDA build (Linux) or MPS build (macOS) to match your OS. If it fails, check the Python version first —`pyproject.toml` pins `requires-python = ">=3.12,<3.13"`.

Core Packages

Skim the names and roles once. The details show up again in the chapter that uses each one.

Package	Role
<code>hmmlearn</code>	HMM (regime estimation)
<code>torch</code>	SAC neural network
<code>optuna</code>	Hyperparameter optimization
<code>duckdb</code>	Local data cache
<code>yfinance</code>	Price data collection
<code>pykalman</code>	Kalman Filter
<code>statsmodels</code>	Time series statistical tests
<code>ib-async</code>	IBKR order API

The exact versions are pinned in `pyproject.toml`.

Pin versions exactly: if you only specify a range like `torch>=2.0`, an update a few months later can break your code. Before deploying to production, lock down exact versions with `uv lock` or `pip freeze > requirements.txt`.

Configuration File

API keys, file paths, and constants are kept in a single `.env` file. Keeping them out of the source code makes it much less likely you will push a key to a git repository by accident.

```
cp .env.example .env
```

```
# .env
IBKR_HOST=127.0.0.1
IBKR_PORT=7497
IBKR_CLIENT_ID=1
DASHBOARD_TOKEN=<CHANGE_ME>
DASHBOARD_SERVER_URL=http://127.0.0.1:8765
```

Change `DASHBOARD_TOKEN` to your own value. Leaving the default in place effectively turns off dashboard authentication.

1.7 Initializing the Data Cache

Pull daily and hourly data for the S&P 500 in one shot. The first download takes around 30 minutes.

```
python refresh_cache.py
python _check_duckdb_cache.py
```

The output of `_check_duckdb_cache.py` looks something like this:

Interval	Coverage	Range
daily_730d	498/500 symbols	2023-04 ~ 2025-04
hourly_730d	493/500 symbols	2023-04 ~ 2025-04
5m	312/500 symbols	2025-02 ~ 2025-04 (60 days)

The cache is about 3.2 GB. A few symbols out of 500 will be missing because of delistings or gaps in yfinance responses, which is expected. If something fails, check disk space and network first.

1.8 Running Your First Backtest

This section makes good on the promise from the start of the chapter. One command actually runs a backtest.

```
# 단일 전략
python -m app.trading.backtest.runner --signal hmm_daily_mr --days 365

# 두 전략 비교
python -m app.trading.backtest.runner \
    --signal hmm_daily_mr hmm_daily_pure_z \
    --days 730

# 등록된 전략 목록
python -m app.trading.backtest.runner --list
```

Running the comparison command prints a result table that looks something like this.

Strategy	Key change	What to check
HMM Daily MR	Uses the regime filter	Have loss periods shrunk?
Pure Z-score (no HMM)	Uses only a fixed Z-score	Is it fragile when the market regime shifts?

What to focus on here is not the code but how to read the result table. Even with the same Z-score signal, adding an HMM regime filter changes the loss periods and the trade frequency. This is the core pattern you will see repeated throughout the book.

How to read the results: when you run for the first time, which numbers matter?

Metric	First question to ask
Equity curve	Are the profits concentrated in only one segment?
MDD	Is the drawdown something the account can absorb?
Win rate	Does the signal look better than random?

On the first run, worry less about the absolute numbers and more about whether the result is reproducible and whether the direction holds up once trading costs are added.

If something goes wrong: if the output shows “error” rather than “ok”, check in this order —(1) whether the cache is empty (python `_check_duckdb_cache.py`), (2) whether `capital` is too small, (3) whether `--days` extends beyond the cached range.

Exercises

1. In your own words, describe how data flows from Layer 1 to Layer 4 in the five-layer architecture. One line per layer is enough.
2. Which command prints the list of registered strategies? From the output, pick out the HMM-family strategies.
3. Explain in one or two sentences why the following code causes look-ahead bias.

```
scaler = StandardScaler().fit(X_all)
X_scaled = scaler.transform(X_all)
```

Recap

The five things you must take away from this chapter:

- **Five layers** —data flows in the order data -> features -> signal -> ensemble -> execution.
- **Signals follow a shared contract** —prepare in `initialize()` and emit signals in `on_bar()`.
- **All strategies are compared in the same format** —they are gathered into `BacktestResult`.
- **No future data** —fit only on the training segment, inference only up to time t .
- **You can run your first backtest yourself** —one command prints the result table.

In the next chapter, we walk through the code that shows how `downloadBars()` caches data into DuckDB and how feature engineering is structured.

Further Reading

- **Book:** Marcos Lopez de Prado, *Advances in Financial Machine Learning* (Wiley, 2018) —a careful treatment of causality and look-ahead bias.
- **Paper:** “Why Most Backtests Fail”—analyzes why backtests fall apart in live trading.
- **Code:** the `mlfinlab` library —provides time-series-aware features and validation tools.

Check your understanding

“Tabulate the data, signal, execution, and accounting stages in this chapter. Identify each input/output and one place future information could enter.”

Chapter 2. The Data Pipeline —DuckDB and Live Feeds

A quant system cannot be better than the data it runs on. That sounds obvious, but if you skip the pipeline and dive straight into strategy code, the lesson comes the hard way. No matter how clever the model is, bad inputs can turn a strong-looking backtest into something much less useful: a model that learned the quirks and errors of the data, not a repeatable trading edge.

When I first started backtesting, all I had was CSV files downloaded from Yahoo Finance. Running a backtest on that OHLCV data gave good numbers, but live trading told a different story. The strategy logic wasn't broken. From the very beginning, the backtest had been looking at data that did not match real trading conditions.

There were three causes.

The first is **survivorship bias**. Companies that were delisted during your backtest window may no longer appear in today’s data. If you quietly drop them, you’re only watching the survivors, and the results look better than reality.

The second is **using future data**. If you compute an indicator from today’s close and then trade on the same day, you’re using information you couldn’t possibly have until after the market closed. That trade is impossible in real life.

The third is **missing dividend and split adjustments**. If you don’t adjust for a stock split, the price looks like it suddenly fell by half on that day. The strategy treats this artificial jump as a real signal.

Any one of these is enough to make backtest results untrustworthy.

A quant data pipeline solves these problems at the code level. The central tool is DuckDB—a lightweight database that can run SQL queries directly on Parquet files. It handles larger datasets more reliably than Pandas alone, and it doesn’t need to load everything into memory at once. When the pipeline takes care of live feeds, data quality checks, and survivorship bias, your strategy code stays much simpler.

How this chapter flows

We’ll start with the big picture, then drop down into implementation and verification.

1. The overall data flow—from API to model input
2. `download_bars()`—a single download entry point with a fallback path
3. DuckDBCache—Parquet files combined with in-memory DuckDB
4. `tz_localize(None)`—an unwritten rule of this project
5. Four-dimensional daily features and handling missing values
6. Kalman smoothing, Z-score, and the shape of the model input

Learning goals

By the end of this chapter you should be able to:

- Explain how `download_bars()` first requests data from FMP and, if that fails, switches to yfinance —a **fallback** structure.
- Explain why DuckDBCache uses Parquet files together with in-memory DuckDB, and why multiple processes can access it concurrently without breaking it.
- Compute four features from daily data —returns, realized volatility, momentum, and volume ratio —and understand why a Kalman filter is used to reduce noise.
- Explain why missing values (NaN) caused by rolling windows must be dropped or handled separately before the model reads the data.
- Explain what goes wrong when a DataFrame mixes timezone-aware and timezone-naive indexes, and why `tz_localize(None)` prevents it.

What you build in this chapter: a pipeline that downloads S&P 500 daily bars with `download_bars()`, stores them in a DuckDB Parquet cache, and turns them into a 4-feature array.

2.1 The Big Picture —Data Flow

Let's look at how the data moves. It's a single line from an API request to the feature matrix that goes into the model.

```

yfinance / FMP API
  |
  v (first download)
download_bars()
  |
  ├── FMP first
  |   | failed or coverage < 30%
  |   v
  └── yfinance / DuckDB fallback
      |
      v
DuckDBCache.save_ohlcv_data()
  |

```

```

      v
cache/.../ohlcv/{symbol}/{interval}/YYYY-MM.parquet
|   (monthly partitions, append-only)
      v
compute_daily_features()
|
      v
np.ndarray shape=(T, 4)  ->  HMM.fit()

```

If the cache already has the data, we read it directly from Parquet without hitting the network. Speed is only part of the reason this matters. To compare results from one backtest to the next, you need the same inputs each time. If the API returns slightly different numbers on each call, you can't tell the effect of a strategy change apart from the effect of a data change.

How raw data becomes features

The first confusing point for many beginners is this: how does price data become model input? A model does not look at a chart image. It reads a table of numbers. So instead of feeding raw OHLCV data directly, we turn it into summary numbers the model can use.

Raw data					
date	open	high	low	close	volume
2020-01-02	...	...	...	...	...
2020-01-03	...	...	...	...	...

Feature matrix X				
date	return	volatility	momentum	volume_ratio
2020-01-02	...	...	...	...
2020-01-03	...	...	...	...

OHLCV is familiar to humans, but for a model it is still raw material. The feature matrix X is a table of clearer questions: “How much did price move today?”, “Has volatility increased?”, “Is volume above normal?”

One row is one day. One column is one viewpoint the model reads.

```

row    = one point in time
column = one question for the model
X[t]   = the bundle of numbers given to the model at time t

```

From this data point of view, training and inference are simply a time split of the same feature matrix.

Feature matrix X: 5 years

```

|-----|-----|
      training data      validation / inference data
      first 2 years      next 3 years

```

```

X_train -> fit(model)
X_oos   -> step forward one row at a time

```

X_oos is still historical data in the file, but we hide it from the model during training and treat it like the future. During validation, the code must move forward one day at a time and feed only x_today into the model. This is OOS validation from the data perspective.

2.2 download_bars() —One Download Entry Point

download_bars() is the single entry point for getting data anywhere in this book. Whether the source is Yahoo Finance or FMP, everything goes through this one function.

When data fetching is scattered across several files, fallback rules, timezone handling, and cache storage drift apart. Pulling it into one function lets us keep those rules in one place.

Intervals the function supports

Interval	Max range	Primary source	Fallback
1d	unlimited	FMP	yfinance
1h	730 days	FMP	DuckDB
5m-30m	60 days	FMP	yfinance

4h and 1wk aggregate base intervals (1h and 1d). Within the same exchange timezone and session boundaries, take the first open, maximum high, minimum low, last close, and sum of volume. Distinguish an unfinished aggregate from a closed bar. Weekly bars record the final actual trading timestamp of the trading week, accounting for holidays. Grouping every four rows from UTC midnight can misalign sessions.

Core flow —two-stage fallback

The loader contract is `(symbol, interval, start, end) -> chronologically sorted OHLCV`. Resolve duplicate timestamps using source and revision timestamps, and validate positive prices, high greater than or equal to low, and nonnegative volume. Never cache a failed download as a valid empty dataset. The sequence is:

1. **Stage 1:** Request data from FMP.
2. **Stage 2:** If the number of returned symbols is below 30% of what we asked for, treat that as a failure. Switch to yfinance for daily bars and to the DuckDB cache for hourly bars.

The result is a dictionary shaped like `{symbol: DataFrame}`. If no symbol comes back, you get an empty `{}`.

Why the 30% threshold? If we ask FMP for 500 symbols and fewer than 150 come back, we treat it as failure. Some symbols can be missing because of market holidays, API rate limits, or network issues.

No retry —be careful in production: the current implementation switches to the fallback as soon as a request fails. A one-second network blip behaves the same as a full outage. For live systems, it's safer to add retries with a library like `tenacity`.

Confirming the fallback actually triggered

Fallbacks happen silently, so to see where the switch occurred you need to turn on debug logging.

```
import logging
logging.getLogger("quant.data").setLevel(logging.DEBUG)
```

The output looks like this:

```
DEBUG: FMP coverage 18% < 30% for AAPL ... — falling back to yfinance
WARNING: All sources failed for TSLA 2008-01-01 — returning empty DataFrame
```

If something goes wrong, check this first: if you see the final returning empty DataFrame warning, the signals for that window will be treated as zero. When a backtest looks strange, this log is the first place to look.

2.3 DuckDBCache —Parquet + In-Memory DuckDB

The cache layer handles two things: **where** to store data (Parquet files) and **how** to read it (in-memory DuckDB).

What is partitioning? It's the practice of splitting data across many files. When DuckDB runs a query like `WHERE symbol='AAPL' AND ym='2024-01'`, it only reads the files that match and skips the rest.

Why DuckDB?

Here are the trade-offs compared with CSV and SQLite.

	SQLite	CSV	Parquet + DuckDB
Query speed	Moderate	Slow	Fast
Multi-process	Single writer	Yes	Yes (append-only)

	SQLite	CSV	Parquet + DuckDB
Compression	Optional	None	Automatic
File size	Moderate	Large	Small

Backtests repeatedly read “this symbol, this date range,” so a layout that lets you pick just the relevant files is a great fit.

Storage layout —monthly Parquet files

The directory tree splits along three levels: symbol, interval, and year-month.

```
cache/historical_data/ohlc/
├─ AAPL/
│   └─ 1d/
│       └─ 2023-01.parquet
│       └─ 2023-02.parquet
│       └─ ...
│   └─ 1h/
│       └─ 2024-11.parquet
└─ MSFT/...
```

The reason for splitting by month is simple. If a backtest needs a year of data, it opens 12 files. If we stored everything for a symbol in a single blob, we’d read all ten years just to look at one.

Handling concurrent access

When multiple processes update the same cache, distinguish file locking from atomic replacement. A lock prevents simultaneous writers; replacement prevents readers from seeing a partially written file. An in-process thread lock cannot exclude other processes. Apply both patterns below.

- **Singleton:** only one DuckDBCache object exists per process. This prevents cache-path configuration from drifting between modules.

- **ThreadLocal connections:** each thread gets its own DuckDB connection. Sharing a connection across threads can corrupt query state.

Parquet files can be **read** by many processes concurrently. Writes only happen in append-only fashion to monthly partition files. Because we never overwrite the same file at the same time, there are no locking conflicts.

Four steps inside the save function

The storage contract writes a validated table to a temporary path and replaces the final path only after success. On an exception, clean up the temporary file and preserve the last valid cache. If data and metadata are replaced separately, include a generation ID and verify that both belong to the same version.

1. Move the `DatetimeIndex` into a date column.
2. Strip timezones (`tz_localize(None)`).
3. Lowercase the column names (`Open -> open`).
4. Group by month and write each group to `YYYY-MM.parquet`.

If the file for that month already exists, the new data is merged into the existing data, duplicates are dropped by date, and the file is rewritten. Re-running the same download many times will not pile up duplicate rows.

2.4 `tz_localize(None)` —An Unwritten Rule of This Project

yfinance and some APIs return a `DatetimeIndex` carrying a UTC timezone. If you leave it as is, two problems show up.

Problem 1 —pandas concat error: trying to combine a timezone-aware `DataFrame` with a timezone-naive one raises `TypeError: Cannot join tz-naive and tz-aware DatetimeIndex`.

Problem 2 —DuckDB save error: DuckDB’s `TIMESTAMP` type expects timezone-naive values by default. Sending tz-aware values fails the conversion.

The correct pattern

Add one line immediately after the download, before any other processing.

```
def download_ohlcv(symbol, interval="1d"):
    df = yf.download(symbol, period="5y", interval=interval,
auto_adjust=True)
    idx = pd.to_datetime(df.index)
    if idx.tz is not None:
        idx = idx.tz_convert("UTC").tz_localize(None)
    df.index = idx
    df.columns = [c.lower() for c in df.columns]
    return df
```

Check this: `df.index.tz` must be `None`. If it’s UTC, every step after this breaks.

2.5 Feature Engineering —Four Daily Features

An HMM does not take raw prices as input. It takes values computed from prices —what we call **features**.

What each of the four features looks at

Dim	Meaning	What it tells you
0	1-day log return	How much today moved versus yesterday
1	5-day realized volatility	How choppy prices have been recently
2	20-day momentum	Which direction the trend has gone over a month

Dim	Meaning	What it tells you
3	Volume ratio	How active trading is compared to normal

Compare designs with a relatively larger Q/R for responsive features such as dimensions 0 and 3, and a smaller ratio for noise reduction in dimensions 1 and 2. Q is state-change variance and R is observation-noise variance. Both have the squared units of their feature: do not reuse raw-price values for standardized returns. Choose settings on the training interval using the prediction/update equations and innovation residuals in Chapter 6, then freeze them for OOS evaluation.

The core calculation —dimension 0 (log return)

```
import numpy as np

def compute_daily_features(close, volume):
    close = np.asarray(close, dtype=float)
    volume = np.asarray(volume, dtype=float)
    if close.ndim != 1 or volume.shape != close.shape:
        raise ValueError("equal-length 1D arrays required")
    if not np.all(np.isfinite(close)) or np.any(close <= 0):
        raise ValueError("close must be finite and positive")
    if not np.all(np.isfinite(volume)) or np.any(volume < 0):
        raise ValueError("volume must be finite and nonnegative")
    n = len(close)
    feat = np.full((n, 4), np.nan)
    ret = np.full(n, np.nan)
    ret[1:] = np.diff(np.log(close))
    feat[:, 0] = ret
    for t in range(n):
        if t >= 5:
            feat[t, 1] = np.std(ret[t-4:t+1], ddof=1)
        if t >= 20:
            feat[t, 2] = close[t] / close[t-20] - 1
            base = np.mean(volume[t-20:t])
            if base > 0:
                feat[t, 3] = volume[t] / base
    return feat
```

```
features = compute_daily_features([100.0] * 25, [10.0] * 25)
assert features.shape == (25, 4)
assert np.allclose(features[20], [0.0, 0.0, 0.0, 1.0])
assert np.isfinite(features).all(axis=1).sum() == 5
```

This standalone NumPy function uses a five-return sample standard deviation without annualization. Momentum needs the close 20 bars ago; volume uses the previous 20 bars excluding today. The first 20 rows are not fully warmed up. Use each row only after its close is finalized. Do not turn missing features into valid zero signals:

```
# Incorrect/unsafe example.
X = np.nan_to_num(features, nan=0.0)
```

The line that matters: `np.maximum(close, 1e-10)` is a guard that prevents `log(0)` when a close value is zero or negative.

Missing Values —Empty Cells the Model Must Not Read

Rolling-window features cannot be computed until enough past rows exist. A 20-day momentum value, for example, cannot exist on day 3 because the model has not seen 20 days yet. Those early rows become NaN, and most ML models cannot learn from or infer on rows that contain NaN.

What to check in the output: `feat.shape == (T, 4)`, and the first 19 rows are NaN (the 20-day momentum fills last). You must drop the NaN rows before passing the array to the model.

2.6 Kalman Smoothing —A One-Liner That Reduces Noise

The features above are noisy. The volume ratio in particular jumps around day by day. The tool that smooths this so the model can see the trend is the **Kalman filter**.

The intuition is enough.

- Each new observation slightly updates the running estimate.
- Two parameters control the behavior: how much to trust the observation (R), and how slowly the underlying signal is assumed to change (Q).
- A larger $R \rightarrow$ trust the observation less $\rightarrow$ stronger smoothing $\rightarrow$ smoother curve.
- A smaller $R \rightarrow$ trust the observation more $\rightarrow$ closer to the raw input.

The one-dimensional Kalman update is: predicted variance $P_{\text{pred}}=P+Q$, gain $K=P_{\text{pred}}/(P_{\text{pred}}+R)$, estimate $x=x+K*(z-x)$, and variance $P=(1-K)*P_{\text{pred}}$. Initialize x from the first valid observation and keep P , Q , and R finite and positive. If an observation is NaN, skip the measurement update and record the missing state separately. After a prolonged gap, do not use a smoothed value as a trading input merely because one exists. This completes the basic flow of collection, storage, features, and smoothing.

2.7 Hourly Features —A 6-Dimensional Extension

Hourly strategies need more than four features, so we use six. Only the two extra dimensions are listed here.

New dim	Meaning
Dim 4	Bar completion ratio —which direction price closed within the bar
Dim 5	VWAP deviation —how far the close is above or below the volume-weighted average price

Both dimensions use only observations finalized by the decision time. For example, compute five-minute returns from consecutive closes and realized volatility from a trailing return window. Until enough history accumulates, mark the input as warming up instead of filling it with zero and treating it as valid. The same number of intraday bars and daily bars represents different elapsed time.

2.8 Z-Score —A Mean-Reversion Signal

The entry signal for a mean-reversion strategy is the **Z-score**. It asks: “How many standard deviations is the current price from its recent average?”

```
def compute_zscore(close, window=20):
    T = len(close)
    z = np.full(T, np.nan, dtype=np.float64)
    for i in range(window, T):
        win = close[i - window: i]
        mu = float(np.mean(win))
        sigma = float(np.std(win))
        if sigma > 1e-10:
            z[i] = (close[i] - mu) / sigma
    return z
```

The line that matters: the window indices are `i - window : i`, which does **not** include the current point `i`. This is how we guarantee **causality** —the calculation never peeks at future data.

How to read it: `- z > +1.5` -> overbought -> consider going short `- z < -1.5` -> oversold -> consider going long

Daily bars use `window=20` (about a month); hourly bars use `window=140`.

2.9 Feeding the Model —A Working Pattern

Now let's put the pieces together and see the full flow from download to HMM input.

```
from app.data.download import download_bars
from app.trading.backtest.features import compute_daily_features

data = download_bars(["AAPL"], days=730, interval="1d")
df = data["AAPL"]

feat = compute_daily_features(df["close"].values, df["volume"].values)

valid = ~np.isnan(feat).any(axis=1)
X = feat[valid]                                # shape: (T_valid, 4)

train_len = int(len(X) * 0.7)
X_train = X[:train_len]
X_oos    = X[train_len:]
```

What to check in the output: - $X.shape[1] == 4$ (feature dimension). - $X.shape[0] < \text{len}(df)$ —shorter than the original because NaN rows were dropped. - $\text{len}(X_{\text{train}}) + \text{len}(X_{\text{oos}}) == \text{len}(X)$. - Never shuffle a time series before splitting. Keep front-to-back time order.

If something fails, check here: if `HMM fit()` raises `ValueError: Input contains NaN`, you skipped step 3.

Drawn on a time axis, the split looks like this.

```
X = feature matrix read by the model

past                                recent
|-----|-----|
          X_train          X_oos
      training data    validation / inference data
```

```

scaler.fit(X_train)
hmm.fit(X_train)

for x in X_oos:
    proba = hmm.step_forward(x)    # judge only; do not train again

```

The key is that `X_oos` never goes into `fit()`. It is already inside your historical file, but during validation it must be treated like data that has not arrived yet. The model learns its rules from `X_train`, then receives `X_oos` one row at a time and performs inference only.

Exercises

1. Describe how `download_bars('AAPL', days=365)` behaves differently on the first call versus the second. Find on disk where the Parquet files are created.
 2. The current partitioning is `symbol / interval / year-month`. If you change it to `symbol / interval / quarter`, which queries get faster and which get slower? Give a concrete example for each.
 3. Deliberately remove the `tz_localize(None)` line and try downloading with `yfinance` and saving to DuckDB. Reproduce the actual error message.
-

Wrap-Up

The pipeline we built in this chapter does two things. It saves OHLCV (open, high, low, close, volume) data from the API into Parquet files, and it turns that data into a feature matrix the model can read. The output flows straight into the Backtest Engine in the next chapter.

Things to remember from this chapter

- `download_bars()` is the one and only entry point for data. If FMP fails, it switches automatically to `yfinance` or the DuckDB cache.

- The DuckDB cache splits Parquet files by symbol, interval, and year-month. Reading is fast because only the necessary files are touched.
- Data from `yfinance` must have its timezone stripped with `tz_localize(None)`.
- The daily model input is a 4-dimensional feature matrix (return, realized volatility, momentum, volume ratio). The Kalman filter is a noise-reduction layer on top of those features.
- Z-score uses only past windows that do not include the current point. If you look at future data, the backtest lies to you.

In the next chapter, we'll build the Backtest Engine. We'll walk through `PortfolioSimulator`'s `on_bar()` loop, **walk-forward validation** (splitting the data in time order to prevent overfitting), and how to verify performance metrics like Sortino and MDD in code.

Further reading

- **Official documentation:** [DuckDB Python API](#)
- **Library:** `yfinance` on GitHub —see the docs for configuring automatic retries on download failure.

Chapter 3. The Backtest Engine —A Simulation You Can Trust

“It worked great in the backtest, but it doesn’t work in real trading.”

It sounds like a joke in the quant community, but it’s the most common and costly mistake. A backtest is a way of checking a strategy against historical data. All it proves is that the strategy worked in the past—it does not guarantee it will work in the future.

Most backtests assume an unrealistically optimistic version of reality. There is no slippage (the gap between the price you wanted and the price you actually got), commissions are lower than they would be in practice, and there is no market impact (large orders pushing the price against you). Numbers produced under those conditions are not real results—they are results from an idealized simulation.

A trustworthy backtest engine needs three things.

First, **look-ahead bias prevention**. When you generate a signal, you must use only data that was actually available at that point in time. If future data leaks in, the strategy looks better than it really is.

Second, **realistic trading costs**. Spread, slippage, and commissions all have to be included.

Third, **liquidity checks**. The order size must not exceed what the market can actually absorb. If any of these are missing, the backtest will overstate the strategy's performance.

In this chapter we build an engine that has all three, step by step.

For a beginner, a backtest is a **simulated live market built from historical data**. After a model finishes training, we return to some point in the past and walk forward one bar at a time using only the information that would have been known on that day. So a backtest is not just code that calculates return. It is a validation device that checks whether a machine-learning model behaves as if it were live.

The key split is OOS. If you have five years of data, you might train on the first two years and validate on the next three. The second part is historical in the file, but it must be treated as unseen future data by the model. That is what makes the backtest meaningful.

From the model's point of view, the backtest is also a dress rehearsal for live trading. The model has already finished training. During the OOS loop, the engine should only build the day's inputs, get a prediction, run risk checks, and record the result. If the OOS loop quietly retrain the model or refits preprocessing, it is no longer simulating live trading.

The most important phrase is **only today's data is revealed**. On the OOS loop's March 5, 2024 step, the engine must behave as if March 6 and later do not exist. The computer has the full file, but the backtest deliberately hides future rows from the model.

```

trained state
HMM model file, SAC Actor, scaler, and feature order are already fixed
    |
    v
OOS backtest loop
reveal one day -> build features -> run model inference -> create
signal -> assume fill -> record PnL
    |
    v
performance evaluation
return, MDD, trade count, win rate, cost-adjusted result

```

This is why a backtest is closer to a rehearsal than a report. It asks whether the trained system can walk forward under the same information limits it would face in live trading.

How this chapter flows

We start with the big picture of what the backtest does on every bar. Then we read the core code, and finally we look at how performance is turned into numbers.

- 3.1 Big Picture —the five-step loop that runs on every bar
- 3.2 The core code —`simulate()`
- 3.3 Computing PnL when a position closes
- 3.4 Performance —`_build_result()`
- 3.5 PnL Gini —a concentration metric
- 3.6 The Deflated Sharpe Ratio (DSR)
- 3.7 `PortfolioConfig` —recommended settings per strategy
- 3.8 How to read performance metrics

Learning goals

By the end of this chapter you should be able to:

- Explain the order in which `simulate()` processes each bar.
- Point out exactly where the OOS (Out-of-Sample) 60/40 split lives in the code.
- Say in one sentence what performance, MDD, and Win Rate each measure. Know Sortino, Calmar, Omega, and VaR at the catalog level.
- Explain why the Deflated Sharpe Ratio (DSR) is needed, and how to read it through its p-value.

What we build in this chapter: a simulator that processes signal -> fill -> evaluation in a fixed order on every bar. “Why does the backtest look good but live trading doesn’t?”—this chapter gives a structural answer to that question.

3.1 Signals, fills, and equity in time order

Separate training from evaluation chronologically. A 60/40 split is an introductory design, not a guarantee. Repeatedly tuning after seeing OOS turns that interval into training data; use separate validation and a final holdout or walk-forward evaluation.

The sequence is: fit on past data; record starting cash/quantity/equity; fill the previous-close target at this bar's open; update cash, quantity, and fill costs; liquidate on the final bar; mark equity at the close; compute its change; decide the next target using only observations now available. Do not compute a signal from a completed close and assume an earlier fill at that same close.

3.2 A runnable accounting exercise

Copy this function and the check below into `backtest.py`. They use only the Python standard library, with no broker, CSV, or HMM dependency. The optional accounting-lab archive contains the same function plus additional tests and a command-line interface.

Cash is currency; quantity is signed units; equity is $\text{cash} + \text{quantity} * \text{close}$. Bar P&L is the change in equity. Omitting unrealized P&L or adding cumulative P&L again at closing distorts the result. `targets[t]` is known after `close[t]` and filled at `open[t+1]`. Signal generation is responsible for using only past information. Final liquidation and its cost occur before the last equity observation.

```
import math

def simulate(
    bars, targets, initial=1000.0, fee_bps=0.0, slippage_bps=0.0
):
    """Fill targets[t], known after close[t], at open[t+1].

    Use fixed unit size.

    cash is currency; quantity is signed units; equity = cash +
    quantity*close.
    Omit financing, margin, taxes, dividends and market impact.
```

```

"""
if len(bars) != len(targets) or not bars:
    raise ValueError("Provide one target per nonempty bar sequence")
if not math.isfinite(initial) or initial <= 0:
    raise ValueError("initial must be positive and finite")
if any(
    not math.isfinite(x) or x < 0 or x >= 10000
    for x in (fee_bps, slippage_bps)
):
    raise ValueError(
        "costs must be finite and between 0 and 10000 bps"
    )
if any(t not in (-1, 0, 1) for t in targets):
    raise ValueError("target must be -1, 0 or 1")
if any(
    not math.isfinite(b[k]) or b[k] <= 0
    for b in bars
    for k in ("open", "close")
):
    raise ValueError("prices must be positive and finite")
cash, quantity = initial, 0.0
unit = initial / bars[0]["open"]
equity, pnl, fills = [initial], [], []

def trade(delta, price, index, reason):
    nonlocal cash, quantity
    execution = price * (
        1 + math.copysign(slippage_bps / 10000, delta)
    )
    fee = abs(delta) * execution * fee_bps / 10000
    cash -= delta * execution + fee
    quantity += delta
    fills.append(
        {
            "bar": index,
            "quantity": delta,
            "price": execution,
            "fee": fee,
            "reason": reason,
        }
    )

```



```

for i, bar in enumerate(bars):
    desired = targets[i - 1] * unit if i else 0.0
    delta = desired - quantity
    if delta:
        trade(delta, bar["open"], i, "previous_close_target")
        # Liquidate before recording the last equity sample.
        # Include the final fill cost.
    if i == len(bars) - 1 and quantity:
        trade(-quantity, bar["close"], i, "final_liquidation")
        current = cash + quantity * bar["close"]
        pnl.append(current - equity[-1])
        equity.append(current)
peak, mdd = initial, 0.0
for value in equity:
    peak = max(peak, value)
    mdd = max(mdd, (peak - value) / peak)
assert math.isclose(initial + sum(pnl), equity[-1], abs_tol=1e-8)
return {
    "equity": equity,
    "bar_pnl": pnl,
    "final_equity": equity[-1],
    "return": equity[-1] / initial - 1,
    "mdd": mdd,
    "fills": fills,
    "fees": sum(f["fee"] for f in fills),
}

```

Append this check and run python3 backtest.py:

```

bars = [
    {"open": 100.0, "close": 100.0},
    {"open": 100.0, "close": 80.0},
    {"open": 80.0, "close": 100.0},
]
result = simulate(bars, [1, 1, 0])
assert result["equity"] == [1000.0, 1000.0, 800.0, 1000.0]
assert math.isclose(result["mdd"], 0.2)
print(result["final_equity"], result["mdd"]) # 1000.0 0.2

```

3.3 Count each cost and final liquidation once

With prices 100->80->100 and initial equity 1000, the final gross P&L is zero but drawdown is 20%. Equity is [1000,1000,800,1000], including the starting baseline. Recording only final cash hides the loss path.

Buying 10 units at 100 and selling at 100 with 10bp on each fill costs 2, leaving 998. Do not subtract round-trip fees again if entry cost was already charged. In this fixed-quantity toy, fees can make cash slightly negative; it does not enforce a cash-only affordability constraint.

To vary costs directly, call `simulate(bars, [1,1,0], fee_bps=10, slippage_bps=5)`. The optional archive adds unittest cases and CLI switches. Buys slip upward and sells downward. Cost candidates such as 0/5/10/25bp are sensitivity scenarios, not universally adequate assumptions.

This engine omits financing, margin, taxes, corporate actions, and market impact. It is a small accounting exercise, not a production portfolio engine or broker connector.

3.4 Performance —`_build_result()`

Once the backtest finishes, the result-building step computes the performance metrics and returns them as a result object. There are three numbers you absolutely need to look at on your first pass.

- **Performance:** how much we earned per unit of risk (volatility). Higher is better.
- **MDD (Maximum Drawdown):** how far equity fell from its peak at the worst point. A deep MDD means that even a strong-looking strategy can be unbearable in real trading.
- **Win Rate:** the share of trades that came out positive. The first question is whether we're above coin flips (50%).

NOTE

Judging a strategy by a single metric is risky. Performance alone doesn't tell you how deep the losses are; MDD alone doesn't tell you whether profit is steady. Read them together, like a checklist.

Volatility-adjusted performance

Looking only at average return is a quick way to misread a strategy. A return earned through big daily swings is not the same as the same return earned through small ones. So the result-building step looks at returns and volatility together.

On a first read you do not need to memorize formulas. The point is this: **even if a strategy earns a lot, we do not call it a good strategy if volatility and drawdowns grow with it.** The exact formulas live in the result-building function in the repository; the book focuses on interpretation.

MDD and Win Rate

MDD is the largest peak-to-trough drawdown ratio. Win Rate is the share of trades with positive PnL. Both are simple calculations, so for the code you can refer to the result-building function in the repository.

$$\text{MDD} = \max_t \left(\frac{\text{Peak}_t - \text{Value}_t}{\text{Peak}_t} \right)$$

More refined metrics (catalog)

Sortino divides mean excess return by downside deviation; Calmar divides annualized return by positive MDD. Omega divides gains above a chosen threshold by shortfalls below it. Zero denominators are undefined. Use consistent returns, periods, and signs.

Metric	One-line meaning
Sortino	Uses only downside volatility in the denominator, removing the penalty for upside swings
Calmar	Annual return ÷ MDD —return relative to drawdown
Omega	Sum of positive returns ÷ absolute value of sum of negative returns
VaR(95%)	The bottom-5% point of daily returns —the size of rare large losses

Metric	One-line meaning
CVaR(95%)	The average loss in the 5% region below VaR
Profit Factor	Total profit ÷ total loss
PnL Gini	Concentration of profit across symbols (see 3.5)

3.5 PnL Gini —a concentration metric

PnL Gini measures how much of the profit is concentrated in a few symbols. Zero means every symbol contributed equally; close to one means a single symbol produced almost all of the profit. The practical threshold is < 0.4 . Above that, the strategy collapses the moment one or two symbols disappear. See the result-building function in the repository for the formula.

3.6 The Deflated Sharpe Ratio (DSR)

The more backtests you run, the more likely it is that something looks good by pure chance. The **Deflated Sharpe Ratio (DSR)** corrects for this bias.

If you tested a single strategy and got a good result, that result is worth examining. But if you tested 500 strategies and picked only the best one, the chances are much higher that you simply got lucky. DSR adjusts for how many attempts you made and discounts the result accordingly.

`n_trials` should not be set to just the number of final candidates. To make DSR reflect the real risk of overfitting, you have to include ideas you discarded along the way, parameter combinations you tried, and experiments you reran with a tweaked setup.

```
from app.trading.backtest.shared import compute_deflated_score_pvalue

dsr_pval = compute_deflated_score_pvalue(
    score_obs = score,          # observed performance
    n_obs     = len(bar_ret),   # number of bars
```

```

    n_trials          = n_strategies_tested,    # number of strategies
    tried
    skew              = sk,
    kurtosis_excess   = ku,
)

```

p-value	Reading
< 0.005	Strong statistical significance (accept)
0.005 ~ 0.05	Significant (accept after review)
> 0.05	May be coincidence (reject)

For derivations and the statistical background, see Bailey & Lopez de Prado (2014).

3.7 PortfolioConfig —recommended settings per strategy

Different strategies suit different position counts and sizes. A trend-following strategy that bets big on a few names is set up very differently from a pair-trading strategy that spreads across many pairs. Each signal returns a recommended configuration that fits its character, and the simulator either uses it directly or overrides it with the defaults.

`max_positions` caps simultaneous holdings; `per_trade_alloc` is allocation per trade; `commission` is the fill cost rate; `is_pair_strategy` links both legs; `rebalance_freq` sets the interval. Five slots at 10% allocate at most 50% to new entries. Specify whether a pair's 10% means both legs combined or each leg. Account-wide risk limits take precedence.

3.8 How to read performance metrics

Absolute values on their own don't mean much. You need a reference point to tell whether a number is good or bad.

Metric	Question
Volatility-adjusted performance	Does the return look inflated relative to its volatility?
Sortino	Is the downside volatility something you can live with?
MDD	Could a real account survive the worst drawdown?
Calmar	Is the return high enough given the drawdown?
Omega	Is the return distribution badly skewed in one direction?
Win Rate	Do the signals look better than random?
Profit Factor	Does total profit comfortably exceed total loss?
PnL Gini	Is the profit concentrated in just one or two symbols?
DSR p-value	Did we just cherry-pick a lucky result out of many experiments?

Read the strategy results in this book through the same questions. Don't aim at a single target number —check drawdown, trading costs, out-of-sample performance, and profit concentration together.

Exercises

1. Backtest the same strategy twice: once with `commission=0` and once with `commission=0.001`. Compare both results in terms of performance and annual trade count. See in numbers how much performance drops once real trading costs are applied.
2. The `on_bar()` loop usually runs in the order: compute signal -> decide positions -> fill. Swap that order and you create a look-ahead bias. Look-ahead bias is the error where

code behaves as if it had already seen data that wasn't available yet. Write a code example with the wrong ordering and explain exactly where the bias enters.

3. Extend the accounting exercise with dated equity observations and compute annual returns. Check which year performed best, and keep annual return separate from risk-adjusted metrics.

Summary

In this chapter we finished a backtest engine that respects causality, along with the main performance calculations. From now on, every strategy is validated through this engine.

What you must remember from this chapter:

- **OOS separation** is the starting point of evaluation. Train on the first 60%, measure returns only on the last 40%.
- **Order matters within a bar**: exit → mark-to-market → enter. Change it and the result is distorted.
- **Commission is deducted on both entry and exit** ($2 * \text{commission}$).
- **Don't read one metric in isolation**. Look at performance (return relative to volatility), MDD (worst drawdown), and Win Rate (share of wins) together.
- **DSR** filters out “things that only look good because you tried many times.” It needs to pass $p\text{-value} < 0.05$.

The next chapter covers the basic theory of HMM (Hidden Markov Model). The buy/sell signals produced by an HMM still have to pass through the engine from this chapter before they are worth anything —whatever the model, the evaluation standard is the same.

Further reading

- **Book**: Ernest Chan, *Algorithmic Trading* (Wiley, 2013) —backtest engine design and trading-cost modeling

- **Paper:** Bailey & Lopez de Prado, “The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting and Non-Normality” (2014) —statistical significance of performance under multiple testing
- **Library:** `vectorbt` —a high-performance vectorized backtest engine built on Numba

Count completed round trips for win rate, not fills. Use returns from the same marked-to-market equity curve for Sharpe, with an annualization factor matching the bar interval. Undefined variance or insufficient history is not a zero score.

Chapter 4. HMM Basics —Inferring Market Regimes with Probability

Part II —HMM Basics (Chapters 4-5) In this part, we stop treating the market as “one fixed state” and start treating it as a process where several regimes take turns. Chapter 4 covers the basic structure of HMM, and Chapter 5 covers forward-only inference that does not peek at future data. On a first read, focus on the feeling of “estimating the current regime as a probability” before chasing the equations.

You can read prices, volumes, moving averages, RSI, and MACD straight off a chart. But the thing behind those numbers —what state the market is actually in right now —is invisible. Distinctions like trending market, sideways market, and high-volatility market are exactly this “hidden state.”

Experienced traders judge that state by intuition. They say things like “we’re in a trending market, so ignore RSI” or “this is a range market, so momentum strategies won’t work.” The problem is that this intuition differs from person to person and is not consistent. It is hard to turn into a system.

HMM (Hidden Markov Model) turns that intuition into math. HMM answers a single question: “Given the observable data (returns, volatility, and so on), what is the probability that the market is currently in each regime?” Instead of trying to predict the price itself, it first infers the regime, which is the structure that produces the price.

You do not need to memorize all the equations on a first read. There are three core intuitions to take away from this chapter.

1. HMM does not directly ask “will the price go up?” It first asks “what state is the market in right now?”
2. The state cannot be observed directly. So we estimate its probability from observations like returns, volatility, and volume.
3. The result is not a single right answer but a probability vector. For example, it comes out as [MR 70%, Trend 20%, Volatile 10%].

HMM has three parameters. Just remember the names for now.

- **A** —Transition probabilities between states (“how long does each state last?”)
- **B** —Data distribution for each state (“what kind of data shows up in each state?”)
- **π** —Initial state probabilities (“which state do we assume we start in?”)

The equations in this chapter are not “math for deriving the model yourself.” They are “a map for reading why the code looks the way it does.” When you call `GaussianHMM(n_components=3)`, the library does most of the calculation for you. But which features to feed in, how many states to use, and how to relabel the regimes are decisions you have to make. To make those decisions, you need to know what A, B, and π mean.

HMM as a Machine-Learning Model

HMM sounds intimidating because of the name, but if you look at it as a regular machine-learning workflow, the structure is simple. The flow has four steps.

1. **Build input features from price data.** Instead of feeding raw price into the model, convert it into returns, volatility, momentum, and volume ratios.
2. **Train the HMM on a past window.** `fit()` looks at past features and learns A , B , and π .
3. **Run inference on each new bar.** The trained model receives today's features and returns regime probabilities.
4. **Connect those probabilities to trading signals.** For example, allow mean-reversion entries only when the MR probability is high.

`GaussianHMM` is the blueprint that defines the algorithm; the object you get back after calling `fit()` is the trained model. Before `fit()`, it has not seen the market yet. After `fit()`, A , B , and π have been fitted to the data, so the model can receive new features and return regime probabilities.

```
Before training:
GaussianHMM(n_components=3)
an empty shell prepared to split the market into 3 states

Training:
empty shell + X_train --fit()--> trained HMM model

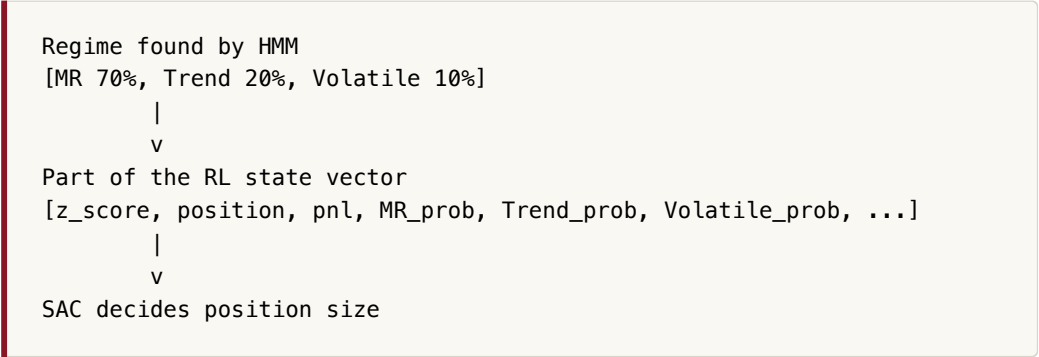
Inference:
trained HMM model + today's features
-> [MR 70%, Trend 20%, Volatile 10%]
```

As an analogy, `GaussianHMM` is a market classifier whose scale has not been calibrated yet. `fit()` calibrates the scale with historical data. Only after that calibration can you feed in today's data and ask, "what kind of market is this?"

So HMM is the first machine-learning model in this book that is actually applied to stock data. It is not supervised learning with a label like "this day is MR, that day is Trend." It is closer to **unsupervised learning**: the HMM groups days with similar observed patterns into hidden states, and after training we attach names such as MR / Trend / Volatile to those states.

This distinction also makes the later RL chapters easier to follow. HMM estimates the market state; SAC receives that state and other features to decide position size. Both sit on the same **data -> features -> training -> inference** flow, but they have different jobs.

The regime probabilities from HMM do not stop in this chapter. In the reinforcement-learning chapters, they become part of the SAC agent’s input.



That is why HMM is not just a filter. It also acts as a sensor that tells the later deep-learning agent about the market context. HMM reads the market weather; RL decides how fast to move in that weather.

The Same Signal Means Different Things in Different Regimes

The easiest way to see why HMM matters is to take the same z-score and place it in two different scenarios. Both scenarios show $z\text{-score} = +2.5$. In a quiet mean-reversion regime, that number can mean the spread is stretched and likely to come back. In a trend or crisis regime, the same number can mean the relationship is breaking. HMM gives the signal context before the strategy acts.

Signal	HMM regime	Interpretation
Large z-score (e.g. +2.5)	Mean-reversion regime	Entry may be reasonable
Large z-score (e.g. +2.5)	Trend regime	Wait; the spread may widen further

Signal	HMM regime	Interpretation
Large z-score (e.g. +2.5)	High-volatility regime	Reduce or block exposure

This is the core idea of the book: do not trade on a single feature by itself. First read the market state, then interpret the signal inside that state. The RL chapters inherit the same logic.

How this chapter is organized

We first pin down what question HMM answers, then drill down into its three ingredients, the code flow, and operational pitfalls.

- 4.0 Big Picture —the `hmm_daily_mr.py` pipeline
- 4.1 The three HMM ingredients (A , B , π)
- 4.2 Baum-Welch —intuition for training
- 4.3 `_DailyHMM.fit()` —the actual implementation and look-ahead pitfalls
- 4.4 Sorting regimes by variance
- 4.5 `_DailyHMM.predict_proba()` —computing regime probabilities
- 4.6 `_update_hmm()` —refitting every 10 bars
- 4.7 `_check_entry()` —the Z-score plus HMM gate
- 4.8 `_check_exit()` —six exit conditions
- 4.9 `PureZscoreDailySignal` —a baseline for comparison
- 4.10 The full flow —call order in `on_bar()`

Learning goals

By the end of this chapter you will be able to:

- Explain what each of HMM's three parameters (A , B , π) represents
- Intuitively describe how Baum-Welch finds parameters through iterative training

- Understand why we choose `covariance_type="full"` and what `min_covar=1e-3` does
- Read code that automatically assigns states to MR / Trend / Volatile regimes based on variance
- Explain how the Z-score signal and the HMM gate combine to produce entry signals

What to focus on during a first read

This chapter introduces HMM, a model that is probably new to you. You do not need to memorize every option and constant on the first pass. Pinning down just these three things is enough.

- HMM is not a model that predicts price directly. It is **a model that estimates the current market state as a probability distribution**.
- In this book we do not use HMM as a buy/sell signal directly. We use it as **a gate that allows or blocks a signal**.
- To avoid look-ahead bias, regime probabilities must be computed without future data mixed in. We cover this more carefully in Chapter 5.

Details like `min_covar`, the refit interval, and the table of exit conditions can wait for a second read. For now, focus on “why we need HMM” and “where the danger lies.”

What we build in this chapter

In Part 2 we apply **HMM** to market regime detection. A regime describes the character of how the market is moving right now. For example, “currently a quiet sideways market” or “currently a directional trending market” are regimes.

HMM assumes that this regime cannot be observed directly. While only looking at numbers like returns and volatility, it probabilistically infers which regime we are in. The core principle is simple. **We judge the current regime using only data observed up to now, without using any future data.**

Observed data	Hidden state (regime)	Meaning
Low return / low volatility	REGIME_MR (0)	Good for mean reversion
Medium return / medium volatility	REGIME_TREND (1)	Trending market
Sharp return swings / high volatility	REGIME_VOLATILE (2)	Risky zone

Entry signals are only allowed in the REGIME_MR state. When the market switches to REGIME_VOLATILE, new entries are blocked and any open positions are closed early.

4.0 Big Picture —the `hmm_daily_mr.py` pipeline

Before looking at the code, let's get the big picture first. The process by which OHLCV data passes through a regime probability vector and turns into a trading signal has four steps. These steps run in order on every bar, and the HMM is retrained every 10 days so it can adapt to changes in market structure.

Step	Call	Input -> Output
1	<code>compute_daily_features()</code>	Raw OHLCV data -> 4-dimensional feature matrix
2	<code>_DailyHMM.fit()</code>	Feature matrix -> trained GaussianHMM
3	<code>_DailyHMM.predict_proba()</code>	HMM -> regime probability vector [MR, Trend, Volatile]
4	<code>_check_entry()</code> / <code>_check_exit()</code>	Probability vector -> buy / short / exit signal

Let's lock in the most important point. **HMM does not produce buy or sell signals directly.**

HMM only tells you the probability of “what state the market is in right now.” The actual buy/sell decision is made by a separate signal like the Z-score. HMM acts as a **gate** that decides whether to let that signal through or block it.

For example, even if a mean-reversion signal fires, if the probability of a high-volatility regime is large, HMM blocks the entry. Separating signal generation from filtering this way makes it easy to find where the problem is when the model is wrong.

4.1 The three HMM ingredients —A, B, π

Transition matrix A

The transition matrix is a table of probabilities for which regime we move to in the next bar. With three regimes it is a 3×3 matrix, and the probabilities in each row sum to 1.

For example, if $a[0][0] = 0.92$, then a market in the mean-reversion regime stays in the mean-reversion regime on the next bar with 92% probability. If $a[0][2] = 0.03$, the probability of jumping straight from mean reversion to high volatility is 3%. Market regimes usually do not change dramatically in a single day, so the diagonal entries are often 0.9 or higher.

Thanks to this matrix, HMM does not overreact to a single day's movement. It reflects the property of markets that regimes tend to persist for some time.

Emission distribution B

This is a probability distribution that describes which data values are likely to appear in each regime. Put simply, it computes “given this regime, how likely is this pattern of returns and volatility?”

In a Gaussian HMM each regime has its own Gaussian (that is, normal) distribution with mean (μ) and covariance (Σ). In a high-volatility regime, large negative returns and high volatility naturally appear; in a mean-reversion regime, low volatility and small return swings are natural. When new data arrives, HMM compares it to each regime's distribution and updates “which regime is most likely now.”

Using `covariance_type="full"` makes the covariance a full 4×4 matrix that captures the correlations between features. We come back to the detailed comparison in Section 4.3.

Initial state distribution π

This is the probability of being in each regime at the very first time step. With enough data it does not have a large effect on the result, and `hmmlearn` estimates it automatically during training. For a first read, “just know that it exists” is enough.

4.2 Baum-Welch —intuition for training

How do we get HMM’s parameters (A, μ, Σ, π)? The problem is that the regime is hidden, so we do not know “which day belongs to which regime.” That means we cannot compute the parameters directly.

The method that solves this problem is **EM (Expectation-Maximization)**. The idea is simple.

1. Initialize the parameters at random.
2. Assuming those parameters, compute “the probability that each day belongs to each regime” (E-step).
3. Use those probabilities as weights to recompute the means and variances (M-step).
4. Go back to step 2 with the new parameters.
5. Repeat until there is no more improvement.

Baum-Welch is the algorithm that implements this EM idea specifically for HMMs. Groupings that started out almost random get organized over iterations into regimes that reflect the structure of the data. The convergence criterion is that the **log-likelihood** (a score for how well the model explains the data) stops increasing.

If you want to dig deeper into the math, see the appendix and the `hmmlearn` official documentation. For a first read, the intuition “we iteratively find better parameters” is enough.

One catch: EM can converge to different answers depending on its starting point —a **local optimum** problem. That means the same data with different initial values can lead to different results. That’s why in Section 4.3 we fix the random number generator with `random_state=SEED` to make runs reproducible.

One more thing: the algorithm does not always find good regimes. If the input features do not actually carry information about the market state, HMM will just split noise into three plausible groups. **That’s why feature design matters more than the name of the model.**

You need inputs that can actually explain market state, such as returns, volatility, momentum, and volume ratios.

4.3 `DailyHMM.fit()` —the actual implementation

WARNING

Watch out for look-ahead bias —this is the biggest pitfall running through the whole chapter.

If you train on the **entire period** of data using `fit(X_all)`, or grab probabilities for all time points at once with `predict_proba(X_all)` and then use them at past time points, the model ends up in a situation where it “saw the future to label the past.” If you have a train/test split, training must use only `fit(X_train)`, and in a live environment you must only use the probability for the current time point.

Chapter 5’s `forward_proba_causal()` is the tool we use to solve this problem properly. In this chapter, just stick to the principle “only look at data before the current bar.”

`fit()` does two things.

1. Standardizes the input features (with `StandardScaler`) and trains a `GaussianHMM`.
2. After training, renumbers the states to MR -> Trend -> Volatile in order of increasing variance.

The key configuration values are pulled out as constants.

Constant	Value	Meaning
<code>HMM_N_STATES</code>	3	Number of states (regimes)
<code>HMM_N_ITER</code>	200	Maximum Baum-Welch iterations
<code>HMM_MIN_DAYS</code>	120	Minimum bars required for the first training
<code>HMM_REFIT_DAYS</code>	10	Refit interval (in bars)
<code>SEED</code>	42	Random seed (for EM reproducibility)

The core of the training function looks like this. You can see the full implementation in `app/trading/backtest/signals/hmm_daily_mr.py`.

```
def fit(self, features: np.ndarray) -> bool:
    X = features[~np.any(np.isnan(features), axis=1)]
    if len(X) < 60:
        return False

    scaler = StandardScaler()
    Xs = scaler.fit_transform(X)

    model = GaussianHMM(
        n_components=HMM_N_STATES,
        covariance_type="full",
        n_iter=HMM_N_ITER,
        random_state=SEED,
        tol=1e-4,
        min_covar=1e-3,
    )
    model.fit(Xs)

    total_var = [float(np.trace(model.covars_[i])) for i in
range(HMM_N_STATES)]
    order = np.argsort(total_var)
    self._state_map = {
        int(order[0]): self.REGIME_MR,
        int(order[1]): self.REGIME_TREND,
        int(order[2]): self.REGIME_VOLATILE,
    }
    self.model, self._scaler = model, scaler
    return True
```

When reading this, two lines matter most.

- **(3) Call `fit_transform` on the training data only.** If validation data gets folded in, look-ahead bias creeps in.
- **(6) Run Baum-Welch** —this is where the EM iteration happens. The library handles it for you.

If something fails, look here: if (2) returns `False`, there is not enough data; if (6) raises an exception, the variance is too small or the data is heavily skewed. The caller keeps using the previous model when a fit fails.

Caution about a fixed refit interval: The default is `HMM_REFIT_DAYS=10`. When market structure changes sharply in a short window —like the 2022 rate spike —a 10-day interval can be too long. You can consider automatically refitting when the VIX rises above 30 or when a regime change is detected.

Why we use `covariance_type="full"`

Type	Characteristic	When to use
"diag"	Assumes feature independence, fast	When features are high-dimensional or data is scarce
"full"	Captures correlations between features	When there are few features and correlations are meaningful

Here we only have 4 features, so even with `"full"` the parameter count stays at about 30. There are also meaningful relationships between features —for example, the negative correlation where volatility tends to fall when returns are high —so we picked `"full"`.

The role of `min_covar=1e-3`

If a particular regime ends up with almost no samples assigned during training, its covariance values get close to zero and the math becomes unstable. `min_covar=1e-3` is a **safety net** that prevents the covariance from dropping below this value.

A tiny covariance floor can permit numerical instability; a large floor can blur regimes. Treat `1e-3` as a candidate for standardized features, not a universal optimum. Compare candidates and seeds on training data, checking singular covariance, failed convergence, and very small state occupancy before selecting on separate validation data.

4.4 Sorting regimes by variance

When Baum-Welch finishes training, we end up with three states. But these states only have numeric labels attached —the model does not tell us “which number corresponds to which kind of market.” **We have to attach meaning to each state ourselves after training.**

The criterion we use is **volatility**. A mean-reversion (MR) market is a stretch where the price oscillates in a narrow range around its average, so its volatility is low. A sharply moving market, by contrast, has the highest volatility.

Mathematically, the sum of the diagonal entries of each state’s covariance matrix (the trace) is that state’s total variance. We just sort the states by this value.

Variance rank	Assigned regime
Lowest	MR (mean reversion)
Middle	Trend
Highest	Volatile

The key code looks like this.

```
total_var = [np.trace(model.covars_[i]) for i in range(3)]

order = np.argsort(total_var)
state_map = {
    int(order[0]): _DailyHMM.REGIME_MR,
    int(order[1]): _DailyHMM.REGIME_TREND,
    int(order[2]): _DailyHMM.REGIME_VOLATILE,
}
```

Why sort by variance instead of by mean return?

Intuitively, you might want to classify “the state with the highest mean return is Trend, the lowest is risky.” But that rule has a big problem. Mean returns flip sign —they are positive in uptrends and negative in downtrends —so **sorting by mean return splits the two ends of**

a single ‘trending’ character into opposite regimes. Variance ignores the sign and captures the “size of the swing,” so it classifies both trending and crashing markets consistently. On top of that, what matters for trading decisions is not “direction” but “is this market quiet enough for a mean-reversion strategy to work?”—and variance answers that directly.

Regime meaning summary:

Regime	Constant value	Market meaning	Signal handling
REGIME_MR	0	Narrow range, reversal opportunity	Entry allowed
REGIME_TREN	1	Trending market	Conditional allow
REGIME_VOLATILE		Sharp swings, risky	Block entry + early exit

4.5 `_DailyHMM.predict_proba()` —computing regime probabilities

When you call `predict_proba()`, you get the probability of each of the three regimes at each time point, in vector form.

```
# Example: 3 bars x 3 regimes
# [MR probability, Trend probability, Volatile probability]
[[0.82, 0.11, 0.07], # Strong mean reversion -> entry allowed
 [0.31, 0.55, 0.14], # Trend dominant -> wait
 [0.08, 0.13, 0.79]] # High volatility -> block new entries
```

If the MR probability exceeds `MR_THRESHOLD=0.60`, we move on to the next step: the Z-score check.

Inside, the work goes in three steps.

1. Standardize the input features using the scaler built during training.
2. Get the raw per-state probability matrix from `hmmlearn`.
3. Use `_state_map` to reorder the columns into [MR, Trend, Volatile] order.

```

def predict_proba(self, features: np.ndarray) -> Optional[np.ndarray]:
    X = features[~np.any(np.isnan(features), axis=1)]
    if self.model is None or len(X) < 3:
        return None

    Xs = self._scaler.transform(X)
    raw = self.model.predict_proba(Xs)

    mapped = np.zeros_like(raw)
    for raw_s, logical in self._state_map.items():
        mapped[:, logical] = raw[:, raw_s]
    return mapped                                     # 0=MR, 1=Trend, 2=Volatile

```

What to check in the output: each row of the returned matrix sums to 1. `mapped[-1]` is the regime probability for the most recent bar.

If something fails, look here: if `self.model` is `None`, training has not happened yet or the fit failed. The caller refrains from producing an entry signal when it gets `None` back.

Caution —the look-ahead warning from Section 4.3 applies here too: `predict_proba(recent[-30:])` itself only takes the last 30 bars as input, so if you use only `mapped[-1]`, you do not look at any data beyond the current point. But if you also use the past rows inside the window for performance calculations, those earlier rows were computed using information from later bars inside the same window—that is look-ahead bias. In Chapter 5 we will eliminate this confusion by computing forward probabilities directly with `step_forward()`.

We have walked through HMM’s three ingredients, the Baum-Welch algorithm, and the fit/predict code. From the next section on, we look at how the model is refreshed in practice and how trading signals are produced.

4.6 _update_hmm() —refitting every 10 bars

Market structure changes over time, so a fixed set of HMM parameters cannot keep up with the latest patterns. So we retrain the model every 10 bars (about 2 weeks of trading days), and after training, we refresh the current regime probability using the last 30 bars.

Two lines are the key.

- When training -> pass in only **data before the current bar** with features[:bar_idx].
- When predicting -> look only at the recent window with features[bar_idx-30:bar_idx].

The crucial property in both cases is **we do not look at future data**.

```
def _update_hmm(self, bar_idx: int) -> None:
    for sym in self._symbols:
        hmm = self._hmms[sym]

        need_fit = (hmm.model is None
                    or bar_idx - hmm._last_fit_idx >= HMM_REFIT_DAYS)

        if need_fit and bar_idx >= HMM_MIN_DAYS:
            feat_train = self._features[sym][:bar_idx]
            if hmm.fit(feat_train):
                hmm._last_fit_idx = bar_idx

        if hmm.model is not None:
            recent = self._features[sym][max(0, bar_idx - 30): bar_idx]
            probs = hmm.predict_proba(recent)
            if probs is not None and len(probs) > 0:
                self._regime_probs[sym] = probs[-1]
```

The important line: `feat_train = self._features[sym][:bar_idx]` —this follows Chapter 3’s causality principle (no look-ahead) directly. `HMM_MIN_DAYS=120` is a safety net that delays the first training until at least 120 bars (about 6 months) have accumulated.

4.7 _check_entry() —the Z-score plus HMM gate

`_check_entry()` is the gate that grants final approval to an entry signal. For a signal to fire, two conditions must hold at the same time.

- **Z-score** —how far the price has drifted from its average (the standardized indicator from Chapter 3)
- **HMM gate** —whether the market is in a state suited to mean reversion

Because the two conditions are combined with AND, if either one fails, no signal is produced.

Asymmetric Z-score thresholds

The buy and short thresholds are not the same.

Direction	Threshold	Reason
Buy	$z < -1.5$	Standard oversold entry
Short	$z > +2.0$	More conservative to cover borrowing costs

Shorting involves borrowing stock, which has a cost. So we only enter shorts when the expected return is large enough to clear that cost.

The HMM gate's double condition

If either of these two conditions is met, the entry is blocked immediately.

1. $\text{mr_prob} < 0.3$ —mean-reversion regime probability is below 30%
2. $\text{volatile_prob} > 0.4$ —volatile regime probability is above 40%

In particular, if the volatile regime probability is high, the entry is blocked immediately regardless of the Z-score condition.

The combined gate in code is captured well by this short excerpt.

```
if mr_prob < 0.3 or rp[_DailyHMM.REGIME_VOLATILE] > 0.4:
    return None

if z < -self._entry_z:          action = SignalAction.BUY          # -1.5
elif z > self._short_entry_z:   action = SignalAction.SHORT_SELL   # +2.0
else:                           return None

return TradingSignal(
    symbol=symbol,
    action=action,
    confidence=mr_prob * abs(z),
)
```

confidence = mr_prob × |z|

This is a score that decides priority when multiple symbols fire signals at the same time. The higher the probability of a mean-reversion market and the larger the price deviation, the higher the score. `PortfolioSimulator` opens positions starting from the highest-scoring signal.

You can see the full implementation in `app/trading/backtest/signals/hmm_daily_mr.py`.

4.8 `_check_exit()` —six exit conditions

An exit closes a position you are holding. Exits have more cases to handle than entries, so their logic is more involved.

`_check_exit()` evaluates six conditions from top to bottom on every bar, and closes the position immediately when any one of them is met.

#	Condition	Default	Meaning
1	<code>mr_revert</code>	$ z \leq 0.3$	Mean reversion complete —the most common profit take
2	<code>z_stop</code>	$ z > 4.0$	Z-score keeps widening —MR has failed
3	<code>price_stop</code>	-4%	Absolute loss limit
4	<code>trailing_stop</code>	-2% from peak	Lock in gains; exit if price falls 2% off the peak
5	<code>max_hold</code>	15 bars	Prevent time-value erosion
6	<code>volatile_exit</code>	<code>volatile_prob > 0.5</code>	Market environment deteriorating —strategy invalidated

The string the function returns becomes the `exit_reason` recorded in the trade log. After a backtest, looking at this distribution tells you which condition is the dominant driver of profits and losses.

An important point: **the order of the conditions is the priority**. Since `mr_revert` is evaluated first, no other stop condition applies to a position whose reversion has already completed. On a first read, just the table is enough. For how the six conditions actually unfold in code, see `_check_exit()` in `app/trading/backtest/signals/hmm_daily_mr.py`.

4.9 PureZscoreDailySignal —a baseline

A pure Z-score version with the HMM gate turned off is also defined in the same file. It is a thin subclass that just sets `use_hmm=False`, and it serves as the **baseline** for A/B comparison to check whether HMM actually helps.

To isolate the gate, compare two experiments using the same dates, symbols, base signals, position caps, and costs. A accepts all base signals; B accepts only HMM-permitted states. Record return, drawdown, trade count, and exposure time to distinguish reduced trading from predictive improvement. One favorable OOS interval does not establish superiority.

4.10 The full flow —call order in `on_bar()`

The order that `PortfolioSimulator` calls on each bar matches the four steps in Section 4.0, but with one added priority: **check exits first, then check entries**. This avoids re-entering the same symbol immediately after exiting it on the same bar.

Reduced to pseudo-code, the flow is:

```
on_bar(bar_idx):
    update_hmm(bar_idx)                # 1) refit every 10 bars + refresh
    regime_probabilities
    for each held symbol:                # 2) exits first — _check_exit()
        if exit condition met: close position
    for each non-held symbol:            # 3) then new entries —
        _check_entry()
        if entry condition met: open position
```

The short gate snippet shows logic, not a complete trading program. Update state probabilities from finalized observations and pass the base signal only above the allowed-state threshold.

Reject missing inputs and unready models. For a threshold of 0.7, a permitted-state probability of 0.8 passes; 0.5 does not.

Exercises

1. Train the HMM on one year of SPY data with `covariance_type` set to "full", "diag", and "spherical" in turn. Compare which setting separates the regimes most stably. (If you know about BIC as a model comparison metric, compute it alongside.)
2. Think of three reasons why training might stop without converging (`ConvergenceMonitor.converged == False`), and describe a possible response for each.
3. Test the variance-sorting logic in isolation on synthetic data. Stitch together fake feature segments with low / medium / high volatility, feed them into `GaussianHMM`, and check whether the `state_map` from `np.argsort(np.trace(...))` really comes out in the intended 0 (low variance) -> 2 (high variance) order. If the order is jumbled, write down which features you would need to improve.

What to remember from this chapter

- HMM does not produce buy/sell signals directly. It just tells you “**what state is the market in right now?**” as a probability. The Z-score handles signal generation, and HMM plays the role of a **gate** that decides whether to let the signal through.
- HMM’s three ingredients —**A** (transition matrix), **B** (emission distribution, what data comes from each state), **π** (initial state probabilities).
- **Baum-Welch** is a training algorithm that iteratively improves the parameters with EM. The library handles it for you, so on a first read just grab the intuition. Because of the local-optimum problem, we fix `random_state` for reproducibility.
- After training we assign **MR / Trend / Volatile in order of increasing variance**. Volatility is the most natural criterion for distinguishing market regimes (mean return flips sign and breaks the classification).

- The entry condition requires two things to hold at once $\text{mr_prob} \geq 0.3$ AND $\text{volatile_prob} \leq 0.4$ AND the Z-score threshold being crossed.
- When training, we use **only data before the current bar** (`features[:bar_idx]`). Breaking this rule introduces look-ahead bias, which we deal with head-on in Chapter 5.

Summary table:

Concept	Key point
HMM's three ingredients	A (transition) $\cdot B$ (emission) $\cdot \pi$ (initial)
Baum-Welch	Random init $\rightarrow$ probability estimation $\rightarrow$ parameter update, repeat
Covariance type	"full" —captures correlations between features
Stabilization	<code>min_covar=1e-3</code> —prevents training divergence
Regime sorting	Ascending variance $\rightarrow$ MR(0) / Trend(1) / Volatile(2)
Entry gate	$\text{mr_prob} \geq 0.3$ AND $\text{volatile_prob} \leq 0.4$ AND Z-score
confidence	$\text{mr_prob} \times z $ —priority score
Retraining	Every 10 bars, with causal slicing

Coming up in Chapter 5: In the current implementation, `predict_proba()` uses information from both directions within its window, so look-ahead bias creeps in. In Chapter 5 we pinpoint the cause and look at how to switch to **forward-only** inference that does not peek at the future.

Further reading

- **Paper:** Baum, L. E. et al. (1970), "A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains"—the original Baum-Welch paper
- **Library:** [hmmlearn official documentation](#) —GaussianHMM parameter reference
- **Book:** Bishop, Pattern Recognition and Machine Learning, Chapter 13 —the mathematical foundations of HMM

Check your understanding

“For 100 observations, four features, and three HMM states, explain input/output shapes, A/π , B/π , and which probability axes must sum to one.”

Chapter 5. HMM Forward-Only —Real-Time Inference Without Future Information

A model that performs well in backtests does not necessarily perform the same in live trading. When I first built an HMM model, I did not really understand this gap. The backtest numbers looked fine, but once I connected the model to live data, the signals changed. The model had not suddenly gotten worse. It had been peeking at the future inside the backtest.

The cause lies in the standard way HMM inference works. The usual HMM inference scans the entire time series forward and backward to find the most likely regime (market state). In a backtest, you have all the data in hand, so this is possible. But in live trading, when you need

to judge today's regime, tomorrow's and the next day's prices do not yet exist. Classifying the present by looking at the entire time series is useful for research, but it cannot be used in a system that has to actually place orders.

This is **look-ahead bias**. It is like a chart trader who already knows the outcome and says, "I would have entered here." The difference is that this bias is hiding inside a library function, not in human memory. Because the code runs cleanly and the charts look nice, it is even harder to spot.

Forward-only HMM solves this problem directly. Instead of looking at future periods, it updates the next regime probability using only the previous step's regime probability and the current observation. In terms of pure accuracy, it may be slightly lower than the standard approach, but an 80% accuracy that you can reproduce at the moment of placing an order is more valuable than a 95% accuracy that cannot be used in real time. That choice is what this chapter is about.

This chapter can feel difficult because "training" and "real-time inference" look similar in the code. Separate them in your mind. During training, you use the entire historical training period to estimate the HMM's parameters. At the moment of live trading, you keep those learned parameters fixed and, for each new bar, advance only the current regime probability by one step. This second process is forward-only inference.

In machine-learning terms, this is the chapter that separates **training** from **serving**. Training is the offline job that fits the model on a block of past data. Serving, or inference, is the online job that attaches the trained model to the live data stream and makes one decision at a time. If a backtest mixes the two, performance can look good while the same numbers cannot be reproduced in a real trading system.

Put more simply: training is "calibrating the machine," and inference is "putting today's numbers into the calibrated machine." In an HMM, the calibrated parts are values such as A , μ , and Σ . Once those values are learned on the training window, the OOS window must keep them fixed and feed in only today's features.

```
training = calibrate the model from past data
inference = read today's row with the fixed model
```


Aspect	Training	Inference
Data used	Historical training window	Data observed up to the current point
What changes	Model parameters such as A , μ , Σ	Current regime probability α
How often	On a retraining schedule	On every new bar
Representative function	<code>fit()</code>	<code>step_forward()</code>

Missing-value handling must follow the same boundary. If NaN rows were removed in training, a live bar with NaN should be rejected or handled conservatively in the same way. If preprocessing differs between training and inference, the model sees inputs that no longer match what it learned.

For example, with five years of data, the flow can look like this.

```
|-----|-----|
           2 years training           3 years OOS inference /
validation
```

- 1) Training window
 - Build features.
 - Fit scaler and HMM.
 - Save model parameters and the last α .
- 2) OOS window
 - Pretend each feature row arrives one day at a time.
 - Load the saved model.
 - Do not call `fit()`.
 - Update only α with `step_forward(x_today)`.

For a beginner, think of training data as the textbook and OOS data as the exam. Studying with the textbook is allowed. Looking at the exam answers first and then grading yourself on the same exam is not. That is why OOS has to be separated in a backtest.

To use a trained model in practice, you also need to save and load it. For HMM, you store not only the model itself but also the scaler and the last α , so inference can continue from the correct state.

Save after training:

1. HMM parameters A, μ, Σ, π
2. Scaler training mean and standard deviation
3. Regime mapping raw state id $\rightarrow$ logical state id
4. Last α current state probability for the next bar

You can picture the code flow like this. The storage format may be pickle, joblib, MLflow artifacts, or something else, but the idea is the same.

```
hmm.fit(X_train)
hmm.forward_proba_causal(X_train)
save_model("hmm_model.pkl", hmm)

hmm = load_model("hmm_model.pkl")

x_today = compute_today_features(...)
regime_proba = hmm.step_forward(x_today)
```

After `load_model()`, you must not call `fit()` again. The purpose of inference is not to teach the model again. It is to put today's feature row into the saved decision machine and receive the current regime probability.

Seeing Forward-Only in One Scene

If forward-only still feels abstract, focus only on the state probability α moving forward one day at a time. α means "the probability that the current market belongs to each regime."

Last day of training
 α = [MR 0.60, Trend 0.25, Volatile 0.15]
 Save this value together with the model file.

First OOS day
 Today's feature row `x_today` arrives.
 Use saved α , transition matrix A , and today's observation likelihood.

```
New  $\alpha$  = [MR 0.64, Trend 0.22, Volatile 0.14]
```

```
Second OOS day
```

```
Start again from yesterday's  $\alpha$ .
```

```
New  $\alpha$  = [MR 0.58, Trend 0.30, Volatile 0.12]
```

The key is that each day continues from the previous α . We do not feed the entire OOS period at once and ask for the most likely full path, because tomorrow's data does not exist in live trading.

```
Forbidden:
```

```
All OOS data -> predict_proba() -> regimes for every date at once
```

```
Allowed:
```

```
Yesterday's  $\alpha$  + today's feature row -> step_forward() -> today's regime probability
```

This can feel slower or less convenient than the full-series method, but it only performs calculations that a real trading system could actually perform. That is why the backtest logic and the live system logic can stay aligned.

The rule is simple. **A value computed at time t must not use any data from after t .** If you see a call like `predict_proba(full_series)` that feeds the entire range at once, suspect it first.

HMM in one paragraph (for readers who are new to it from Chapter 4)

An HMM assumes that an invisible **state** (for example, “trend market / mean-reverting market / crisis market”) changes over time. It has three elements. – **State transition probability** A : the probability A_{jk} that today's state is k given yesterday's state was j . – **Emission probability** $B_k(\mathbf{x})$: the probability that observation $\mathbf{x}$ (returns, volatility, etc.) occurs when the state is k . – **Initial distribution** π : the probability of being in each state at the very first time step.

We only see prices; we never see the state directly. So we **infer the hidden state from the observations**.

Notation note: Bold lowercase letters (α , $\mathbf{e}_t$, $\mathbf{x}_t$) are vectors, uppercase letters (A) are matrices. In Python code, `@` is matrix multiplication and `*` is element-wise multiplication.

How This Chapter Is Organized

- 1. **Big picture** (§5.0) —the four-stage pipeline in `hmm_core.py`
- 2. **Why we need it** (§5.1) —why `predict_proba` looks at the future
- 3. **Core concept and formulas** (§5.2-5.3) —Forward recursion and α initialization
- 4. **Online inference** (§5.4) —one step per new bar
- 5. **Real-world parts** (§5.5-5.10) —speed optimization, 7-state, multipliers, stabilization, walk-forward

§5.0-5.4 are the core concepts and implementation; §5.5-5.10 are the parts you combine with them in practice.

Learning Goals

By the end of this chapter you will be able to:

- Explain in one line why `predict_proba()` ends up referencing future data.
- Understand what the Forward recursion $\alpha_t(k)$ means and why normalization is needed.
- Picture the flow of initializing with `forward_proba_causal()` and then updating one step at a time with `step_forward()`.
- Picture how the 7-state regime probabilities combine into `regime_mult` (the position multiplier).

Two forward implementations

This codebase has two coexisting ways of inferring the regime.

Method	Used in	Causal?
<code>predict_proba(recent_30_bars)</code>	<code>hmm_daily_mr.py</code>	Approximate (smoothed within the last 30 bars)
<code>forward_proba_causal()</code> + <code>step_forward()</code>	<code>hmm_core.py</code>	Strict online forward α

“Causal” here means it does not use future data. The daily MR signal in Chapter 4 uses the first method; the 5-minute strategy in this chapter uses the second.

5.0 Big Picture —hmm_core.py Pipeline

hmm_core.py works in four stages. The output of one stage becomes the input of the next.

Step	Stage	Input -> Output
1	Training — _BWHMM.fit()	Training features -> parameters (A , μ , Σ) learned, states sorted by variance (0 = DeepMR ...6 = Crisis)
2	Initialization —for— ward_proba_causal()	All training features -> forward a recursion -> save α at the last bar (_alpha)
3	Online inference — step_forward()	One new feature vector -> update _alpha -> regime probabilities
4	Portfolio multiplier — GlobalRegime— Layer.step()	Weighted sum of probabilities $\times$ scales -> regime_mult (0.1 to 2.0)

In plain terms:

Step 1 (training): Determine the model parameters from past data. Compute each market state’s mean (μ) and covariance (Σ), and the transition probabilities (A) between states.

Step 2 (initialization): Walk through the training data from beginning to end one more time and save the state probability vector α at the last time step. Real-time inference picks up from this point.

Step 3 (online inference): Each time a new bar arrives, advance α by one step. Past data is not reprocessed, so it is fast, and future data is not referenced, so there is no look-ahead bias.

Step 4 (portfolio multiplier): Convert the current state probabilities into a single number in the range 0.1-2.0 (regime_mult) and use it for position sizing.

5.1 Why predict_proba Causes Look-Ahead Bias

First, let's nail down why the standard predict_proba cannot be used as-is.

IMPORTANT

The predict_proba of a typical HMM library (hmmlearn) produces its answer by multiplying together the probability “given the data so far” and the probability “given the data after.” That is, when judging the regime at some time step, it has already looked at the bars that come after. So it cannot be used for causal inference, which must judge the present from past data alone.

Here is a concrete example. Suppose you feed in data from Monday through Friday all at once and ask for Wednesday's regime. The algorithm already knows there were sharp drops on Thursday and Friday. So it classifies Wednesday as a “risk regime.” The backtest result looks remarkably accurate, but in the actual Wednesday session, Thursday and Friday's data does not exist yet.

In formula form, predict_proba computes this:

$$\gamma_t(k) = \frac{\alpha_t(k) \beta_t(k)}{P(\mathbf{x})}$$

Here α_t is the term built from the **past** only, and β_t is the term built from the entire **future**. The moment you multiply them, the future leaks in.

Visually, the range of data used at time t differs between the two methods like this:

```

Time axis:      ... t-3  t-2  t-1   t   t+1  t+2  t+3 ...

predict_proba:  _____[ t ]_____ (looks at
both sides)
                  <- past ->          <- future ->
                        ↑ look-ahead!

Forward-only:   _____[ t ]           (past +
present only)
                  <- past ->
    
```

Comparing three call styles from weakest to strongest causality:

```
# Incorrect/unsafe example.
proba = model.predict_proba(X_all)

recent      = features[max(0, bar_idx - 30): bar_idx]
proba_recent = hmm.predict_proba(recent)

alpha = bwhmm.step_forward(feat_row)
```

Reading the three lines in order makes it clear “how far the data goes.” This chapter uses the bottom approach, `step_forward()`.

5.2 Forward Algorithm Recursion

The forward algorithm accumulates, step by step, “the probability that the observations up to now would occur.” That value is written as $\alpha_t(k)$, where t is the time step and k is the regime index.

The core recursion is:

$$\alpha_t(k) = \left[\sum_j \alpha_{t-1}(j) A_{jk} \right] \cdot B_k(\mathbf{x}_t)$$

In words:

- Inside the brackets: “the sum of probabilities of transitioning into regime k from each previous regime.”
- The factor $B_k(\mathbf{x}_t)$: “the probability that the current observation occurs given regime k .”

So it is one line: **(past state probability × transition probability) × (probability of the current observation in that state)**. At the very first step there is no previous state, so we start from the initial distribution π .

One practical caution —normalization: As more bars accumulate, you keep multiplying small probabilities, and the values shrink to zero —a numeric underflow. So at every step we divide α by its own sum, forcing the sum to stay at 1. As a result, α reads cleanly as “the probability distribution over regimes.”

Hand-calculated example —2 states, 3 steps: Let’s plug in numbers and watch how α updates.

Assume an initial distribution $\pi = [0.5, 0.5]$, a transition matrix $A = \begin{pmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{pmatrix}$, and emission probabilities $\mathbf{e}_t$ at each step.

Step t	$\mathbf{e}_t$ (state 0, state 1)	Before update: $\alpha \cdot A$	$\odot \mathbf{e}_t$	After normalization $\hat{\alpha}_t$
1 (ini- tial)	(0.6, 0.2)	$\pi = (0.5, 0.5)$	(0.30, 0.10)	(0.75, 0.25)
2	(0.4, 0.7)	(0.725, 0.275)	(0.290, 0.193)	(0.601, 0.399)
3	(0.3, 0.8)	(0.621, 0.379)	(0.186, 0.303)	(0.381, 0.619)

In the third bar, state 1’s emission probability becomes larger, and its probability—which started at 0.25 —climbs to 0.619. We never looked at the future, yet the distribution shifts naturally as observations pile up. That is the essence of the forward recursion.

5.3 `_BWHMM.forward_proba_causal()` —Initializing α After Training

This function has one job. **Walk through the training data in order from start to finish, accumulate α , and save the α of the last bar into `self._alpha`.** Real-time inference starts from this value.

The overall flow is exactly the formula from §5.2. Excerpting the core:


```

def forward_proba_causal(self, features: np.ndarray) -> None:
    Xs = self._scaler.transform(features)

    log_e = self.model._compute_log_likelihood(Xs)
    log_e -= log_e.max(axis=1, keepdims=True)
    e = np.exp(log_e)

    #  $\alpha_1 = \pi \odot e_1$ 
    alpha = self.model.startprob_ * e[0]
    alpha = alpha / alpha.sum()

    for t in range(1, len(Xs)):
        alpha = (alpha @ self.model.transmat_) * e[t]
        alpha = alpha / alpha.sum()

    self._alpha = alpha

```

The key is a single line: `alpha = (alpha @ self.model.transmat_) * e[t]`. That is the recursion from §5.2. The matrix product (`@`) applies the transition probabilities, and the element-wise product (`*`) applies the emission probabilities. Then divide by the sum to normalize.

Check finite observations, valid covariance, and normalization. If the probability sum is near zero, compute in log space with log-sum-exp. If that fails, invalidate the signal. A numerical fallback to uniform probabilities must not restore trading eligibility. Check that initialized probabilities sum to one.

5.4 `_BWHMM.step_forward()` —One-Step Online Inference

In live trading we call this function each time a new bar arrives. Recomputing the entire history at every bar would be too slow, so `step_forward()` computes **just one step**. It moves forward one slot from the `_alpha` saved at the previous bar.

A sample call looks like this:

```
>>> hmm.step_forward(x_new)
# Run time: ~0.2ms
# Returned: array([0.76, 0.18, 0.06])  <- MR/Trend/Volatile
probabilities
```

Inside the function, three things happen.

1. Scale the new observation with the same mean and standard deviation used during training.
2. Compute the emission probability e for each state.
3. Update α by one step using the recursion from §5.2 and normalize.

The code is almost identical to §5.3. The only difference is that it processes “just one bar.”

```
def step_forward(self, feat_row: np.ndarray) -> np.ndarray | None:
    Xs = ((feat_row - self._sc_mean) / self._sc_scale).reshape(1, -1)

    diff = Xs[0] - self.model.means_
    log_e = -0.5 * (diff * diff * self.model._covars_inv).sum(axis=1) \
        + self.model._covars_lognorm
    log_e -= log_e.max()
    e = np.exp(log_e)

    alpha = (self._alpha @ self.model.transmat_) * e
    self._alpha = alpha / alpha.sum()

    return self._to_logical(self._alpha)
```

Role of `_to_logical()`: HMM training does not always sort the states in the same order. So right after training we build a “raw index -> logical index” map based on variance (0 = DeepMR, ..., 6 = Crisis). This function rearranges the elements of α according to that map.

The function returns `None` when (a) training has not happened, (b) the input contains NaN, or (c) an exception is raised. In production, if `None` or NaN flows through into the signal calculation, the position can silently go to zero. So count consecutive failures and trigger an alert once a threshold is crossed (the detailed operational rules are covered in a separate operations chapter).

How the training and OOS periods connect

Let’s see how the α recursion continues between the training period (used for fitting) and the OOS (out-of-sample, validation) period. Save the α at the end of training, then carry it forward into the OOS period.

Period	Call	What it does
First bar of training	<code>forward_proba_causal()</code>	$\alpha = \pi \times e$
Middle of training	Same (repeated)	$\alpha = (\alpha @ A) \times e$
Last bar of training	Same	Save <code>_alpha</code>
First OOS bar	<code>step_forward(new_feat)</code>	$\alpha = (_alpha @ A) \times e$
Subsequent OOS bars	<code>step_forward(new_feat)</code>	$\alpha = (\alpha @ A) \times e$

In this structure, **only data up to that time step** is used at each time step. That is why no look-ahead bias appears.

From here on we cover parts used together in practice. If you are reading the core concepts (§5.0–5.4) for the first time, it is fine to skim these sections. Come back to them in detail when you are building a real system.

5.5 `_precompute_cov_fast()` —Speed Optimization

In a 7-state $\times$ 5-minute-bar environment, `step_forward()` is called often. Recomputing the inverse of the covariance Σ and the normalization constant every time would be slow. `_precompute_cov_fast()` is called exactly once right after `fit()` and **caches** the inverse covariance and the log normalization constant in advance. Since this codebase uses diagonal covariance, only element-wise inverses are needed, making it about 3-5 $\times$ faster than the equivalent internal calls in `hmmlearn`.

After `fit()`, cache each state covariance's Cholesky factor and log determinant. Reuse them for Gaussian log likelihood $-0.5 * (d * \log(2\pi) + \log(\det(\Sigma)) + (x - \mu)^T \Sigma^{-1} (x - \mu))$. Solve triangular systems instead of explicitly inverting covariance. Rebuild caches whenever refitting changes Σ .

5.6 7-State Regime Structure — `_REGIME_A_SCALE`

`hmm_daily_mr.py` uses 3 states; `hmm_core.py` uses 7. The seven states are sorted by variance. State 0 is **DeepMR** (extremely low-volatility mean reversion) with the lowest volatility, and state 6 is **Crisis** with the highest volatility. The states in between have different trend strengths.

Each state has a **scale** value attached to it that adjusts position size. The intuition is just two points.

- **When volatility is very low (DeepMR):** the spread profit is small and the fee share is large, so reduce leverage.
- **When volatility is very high (HighVol/Crisis):** the HMM signal itself is less reliable and the loss risk is large, so cut positions sharply.

State multipliers are risk policy, separate from learned probabilities. Normal/high-volatility/crisis scales of 1.0/0.5/0.1 are examples. Compute `sum(prob[state]*scale[state])`, then apply the total position cap. Align state meanings after refitting before attaching multipliers, because numerical state IDs can change.

5.7 `GlobalRegimeLayer.step()` —Reducing to a Single Multiplier

This method does one thing. When a new bar arrives, it gets the 7-state probability γ from `step_forward()`, multiplies by the `_REGIME_A_SCALE` values, and reduces them to a single number, `regime_mult`. This number feeds directly into the position multiplier for Kelly sizing in Chapter 9.

```

def step(self, bar_idx: int) -> float:
    if not self._hmm.is_fitted:
        return 1.0

    feat_row = self._feats[bar_idx]
    alpha = self._hmm.step_forward(feat_row)
    if alpha is None:
        return 1.0
    self._last_alpha = alpha

    n = len(alpha)
    mult = sum(float(alpha[i]) * _regime_mult_from_rank(i, n) for i in
range(n))

    self._update_dominant_state(alpha)
    return float(np.clip(mult, 0.0, 2.0))

```

The heart is one line: `mult = sum(y_i × scale_i)`. Rather than picking a single state, we blend the probabilities across all states. As a result, the scale does not jump at regime transition boundaries, and trading costs go down.

`regime_mult` is a real number between 0.1 and 2.0. 1.0 means the baseline position, 0.5 means hold at half size, and 1.5 means hold at 1.5×.

5.8 dominant_state —3-Bar Confirmation for Stability

The HMM recomputes probabilities at every bar, so when the market wobbles briefly, the probabilities wobble with it. Using that jitter directly as a signal increases unnecessary trades.

`dominant_state` is the device that suppresses this jitter. At every bar it looks at the state with the highest probability, but **only updates the state when the same regime has been confirmed for 3 bars in a row**. A one- or two-bar difference does not flip it. On top of that, if even the highest probability falls below a certain threshold (about 0.32 in the code), it treats the regime as unclear and keeps the current state.

Use probability separation and persistence for dominant-state changes. One illustrative policy requires a candidate advantage of 0.15 for three consecutive bars, resetting the counter when the condition fails. Select actual thresholds in training/validation and freeze them OOS.

5.9 A Walk-Forward Example

NOTE

If the retraining interval is too short (for example, 3 days), there is a high risk of overfitting; if it is too long (for example, 120 days), it cannot keep up with market changes. In practice, a **10-day interval** is a good balance.

Let's bundle the parts we have covered into an actual walk-forward loop. The flow is simple: prepare data -> train and initialize α -> one step per bar in the OOS period.

```
from app.trading.backtest.signals.hmm_core import GlobalRegimeLayer,
REGIME_LABELS

layer = GlobalRegimeLayer(n_states=7)
layer.prepare(spy_df, macro_dfs={"GLD": gld_df, "TLT": tlt_df})

n_train = int(len(spy_df) * 0.6)

layer.refit_and_init(n_train)

results = []
for bar_idx in range(n_train, len(spy_df)):
    mult = layer.step(bar_idx)
    results.append({
        "bar_idx": bar_idx,
        "regime_mult": mult,
        "dominant": REGIME_LABELS[layer.dominant_state],
        "trans_risk": layer.transition_risk,
    })
```

What to check in the output: if the average of `regime_mult` is near 1.0, things are normal. If it is skewed below 0.3, the HighVol/Crisis share is abnormally large.

Inside `refit_and_init()`, three things happen.

1. Slice a trailing 60-session window ($60 \times 78 = 4680$ bars).
2. Create a fresh `_BWHMM` and call `fit()` (cold reinitialization instead of a warm start).
3. Initialize α with `forward_proba_causal()`.

Missing observations and insufficient history mean “unable to evaluate,” distinct from a valid negative pattern result. The order gate rejects both conservatively, but logging the unavailable state preserves the reason.

5.10 Transition risk

A predictive transition metric uses the probability of leaving the current dominant state: $p_{\text{next}} = \alpha A$ and $\text{risk} = 1 - p_{\text{next}}[k]$. This differs from entropy of the current distribution. High entropy means uncertainty, not a guaranteed imminent transition. Fix the definition, normalization, and entry-blocking threshold before evaluation. If $p_{\text{next}}[k] = 0.8$, transition risk is 0.2 under this definition.

Exercises

1. Apply both `forward_proba_causal(X)` and `predict_proba(X)` to the same 500 observations, and visualize the difference in regime probabilities. From what point do they start to diverge significantly?
2. Extend the hand-calculated example from §5.2 to 5 steps. Pick the emission probabilities for bars 4 and 5 as you like, and write out the values before and after normalization at every step for comparison.
3. Analyze how the results change when the walk-forward training window is reduced from 250 days to 125 days.

What to Remember from This Chapter

Three things to take away from this chapter.

- 1. **Standard `predict_proba` looks at the future.** It looks fine in backtests but cannot be used in real time.
- 2. **Forward-only uses only “the previous α + the current observation.”** That is how it infers under the same conditions as the live environment.
- 3. **Training (`fit`) -> α initialization (`forward_proba_causal`) -> one-step update (`step_forward`)**—these three stages form the backbone of the entire flow.

Let’s return to the story at the start of the chapter. That backtest collapsed in live trading not because the model was bad, but because it was already looking at Thursday’s and Friday’s data while judging Wednesday’s regime. If you replace `predict_proba` with `step_forward` and rerun on the same data, the backtest numbers themselves shrink—but **you get the same values when you move to real time**. That consistency is the real value of forward-only.

The implementation checklist:

Step	Method	Purpose
Train	<code>_BWHMM.fit(train_feats)</code>	Estimate parameters and sort states
Speed	<code>_precompute_cov_fast(model</code>	Pre-compute inverse covariance
Initialize	<code>forward_proba_causal(train_feats)</code>	Set to the state at the end of training
Infer	<code>step_forward(feat_row)</code>	Update α one bar at a time, return regime probabilities
Combine	<code>GlobalRegimeLayer.step(bar_weight</code>	Weighted sum of $\gamma \times \text{scale}$ -> <code>regime_mult</code>
Stabilize	3-bar confirm	Prevent abrupt changes in <code>dominant_state</code>
Uncertainty	<code>transition_risk</code>	Entropy-based transition risk

Core formula, one line: each time a new bar arrives, update α one step forward as follows.

$$\hat{\alpha}_t = \frac{(\hat{\alpha}_{t-1} \cdot A) \odot \mathbf{e}_t}{\|(\hat{\alpha}_{t-1} \cdot A) \odot \mathbf{e}_t\|_1}$$

Then take a weighted sum of these probabilities to get a single multiplier.

$$\text{regime_mult} = \sum_k \gamma_k \cdot \text{scale}_k$$

Next chapter (Chapter 6): Price data is mixed with noise both large and small. Chapter 6 covers the 1D Kalman filter, which filters out this noise in real time.

Further Reading

- **Paper:** Rabiner, L. R. (1989), “A tutorial on Hidden Markov Models”—the original source for the forward-backward algorithm.
- **Paper:** “Online EM algorithms for HMMs”—the theoretical foundation for real-time HMM retraining.
- **GitHub:** `hmmlearn` source code `_do_forward_pass()` —the internal implementation we built on directly in this book.

Chapter 6. The Kalman Filter —Denoising and the Innovation Z-Score

Part III —Noise and Multiple Regimes (Chapters 6-8) This part is the heart of the system. Chapter 6 cuts noise out of prices and features. Chapter 7 looks at several markets together. Chapter 8 separates the broad macro regime from individual asset signals into two layers. On a first read, focus less on the equations of each model and more on the question “why don’t we just look at one stock?”

Chapter 2 Kalman vs. Chapter 6 Kalman —Different Jobs

	Ch. 2 ScalarKalmanFilter1D	Ch. 6 KalmanFilter4D
Input	A single feature (one return series)	Four features (return, volatility, momentum, volume)
Main job	Denoise the feature	Denoise the feature + produce an Innovation Z-score
Output	A smoothed feature fed directly into the HMM	A smoothed feature plus a Z-score for novelty_gate

If the Chapter 2 Kalman filter is “a basic cleaning tool,” the Chapter 6 Kalman filter is “cleaning plus anomaly detection added on top.”

Indicators like RSI, MACD, and Bollinger Bands all start from the same assumption: that the closing price exactly reflects the market value at that moment. But real closing prices carry noise—the bid-ask bounce, order imbalances right before the close, and other quirks from market microstructure. If you compute indicators on noisy values, you end up analyzing the noise instead of the signal.

This is especially painful around a regime change—for example, when an uptrend flips into a downtrend. A simple moving average is stuck either way: if you make it react quickly, it shakes around with noise; if you make it slow, it catches the transition too late.

The Kalman filter takes a different angle. It does not just average past prices. It treats prices not as “the absolute truth” but as “noisy observations,” and every time a new observation arrives it updates an estimate of the hidden true state. The technique was originally used to estimate the position of rockets from noisy sensors, back before GPS existed, but the same principle works on financial time series.

The short definition: **a Kalman filter is “the optimal estimator of a hidden state given noisy observations.”**

If you only see the Kalman filter as a “fancy moving average,” you miss the point. A moving average simply averages the last N points. A Kalman filter weighs two things against each other: “how uncertain is my prediction?” and “how trustworthy is this new observation?” When the new observation looks reliable, it follows quickly; when it looks noisy, it moves slowly.

Situation	Moving Average	Kalman Filter
A sudden price jump	Reacts late, delayed by the window length	Can react quickly depending on settings
Temporary noise	Mixed into the average and shakes the indicator	Only partially reflected if noise is large
Expressing uncertainty	None	Tracked through the estimate variance P

In this chapter we apply the Kalman filter in two places: smoothing the input features for the HMM, and building a Z-score for a mean reversion signal.

In the machine-learning flow of this book, the Kalman filter is a **cleaning tool that makes numbers less shaky before the model reads them**. HMM and RL trust the numbers they receive. If the inputs jump around because of noise, the model can mistake that noise for a real market-state change.

```

Raw prices / raw features
noise, jumps, temporary outliers
    |
    v
Kalman Filter
separates real movement from temporary noise
    |
    v
HMM / Z-score / RL input
the model reads more stable numbers

```

For a beginner, it is confusing to think of the Kalman filter as “a model that predicts price.” In this chapter, the Kalman filter is not trying to guess tomorrow’s price. It decides **how much to trust today’s observation**. When a new price jumps, it asks whether that jump looks like a real change or a temporary piece of noise.

When a new observation arrives, the Kalman filter asks:

1. How certain was my previous estimate?
2. How trustworthy is the new observation?

3. How much of the new value should I accept?

After this cleanup, the HMM's regimes shake less, and the Z-score is less likely to mistake a one-bar jump for a real signal.

What this chapter covers

- The big picture —the two roles of the Kalman filter
- The local level model —the math, made intuitive
- `KalmanFilter1D` —smoothing HMM features
- Per-feature Kalman parameters for the 4-dimensional input
- `_kalman_innovations()` —pulling innovations out of price
- A causal Z-score and the idea of halflife
- The mean reversion signal and a backtest

Learning goals

By the end of this chapter you will be able to:

- Explain the predict-update cycle of the Kalman filter at a high level
- Walk through how `KalmanFilter1D` processes the four HMM features
- Say what an Innovation is (the gap between prediction and reality) and why it works as a mean reversion signal
- Understand the entry/exit logic of the Innovation Z-score signal (`kalman_self_mr`)
- See that the halflife determines how long we should hold a position

6.0 The big picture —two roles for the Kalman filter

In this codebase we use the Kalman filter for two purposes. The algorithm is the same; the goal is different.

The first job is **denoising**. Raw feature data carries noise, and if you feed it straight in, the HMM can misjudge the regime. Running the data through a Kalman filter removes some of that noise so the HMM gets a more stable input.

The second job is **building a mean reversion signal**. We measure how far the price has wandered from the Kalman filter’s prediction and use that as a buy or sell signal.

Role 1: smoothing the HMM input features (`features.py`)

The input is four raw features: return, 5-day volatility, 20-day momentum, and volume ratio. We apply a separate `KalmanFilter1D(Q, R)` to each dimension and hand the smoothed features to the HMM.

Role 2: building a Z-score from price Innovations (`kalman_self_mr.py`)

Here, “Innovation” means “the gap between what the filter predicted and what was actually observed.” If that gap is large, the price has wandered far from the prediction. Normalizing the gap into a Z-score gives us a signal we can use for entries and exits.

The input is a `log(price)` series. `_kalman_innovations()` pulls out the per-bar prediction errors (the innovations), and `_build_z_series(window=60)` turns that series into a rolling Z-score. That Z-score is the core of the entry/exit logic.

6.1 The local level model —the math, made intuitive

Before looking at the equations, let’s nail down the meaning of the two key parameters. Once you understand these two values, you basically understand the Kalman filter.

NOTE

- **Q (process noise)**: how much the hidden state really moves. If Q is large, we trust the latest observation more (faster reaction).
- **R (observation noise)**: the size of the measurement error. If R is large, we trust the previous estimate more (slower reaction).

In practice, start with `Q=0.01` and `R=1.0` and adjust while watching performance.

KalmanFilter1D is a **local level model**. It has one core assumption: “the hidden true state only moves a little from its previous value.” We assume that prices or returns do not suddenly leap to a completely different value in a single moment.

In equations, it's two lines.

- **State equation:** the hidden true value x_t is the previous value plus a small random noise term.

$$x_t = x_{t-1} + w_t, \quad w_t \sim \mathcal{N}(0, Q)$$

- **Observation equation:** the value we actually see, y_t , is the true value plus a measurement error.

$$y_t = x_t + v_t, \quad v_t \sim \mathcal{N}(0, R)$$

At every step the filter runs through two stages.

Stage 1: Predict. As time passes, the uncertainty P of our estimate grows by Q .

Stage 2: Update. When a new observation arrives we compute the **Kalman gain** K and pull the prediction toward the observation.

$$K_t = \frac{P_{t|t-1}}{P_{t|t-1} + R}$$

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t \cdot \underbrace{(y_t - \hat{x}_{t|t-1})}_{\text{Innovation}}$$

The term inside the parentheses, $(y_t - \hat{x}_{t|t-1})$, is the **Innovation**. It tells us “how wrong the prediction was,” and K_t decides how much of that error to absorb.

The intuition behind K: - $K \rightarrow 0$: observation noise R is large $\rightarrow$ ignore the new observation and stay close to the previous estimate (smooth). - $K \rightarrow 1$: state noise Q is large $\rightarrow$ follow the new observation almost exactly (fast reaction).

In practice, instead of memorizing Q and R separately, it is easier to think in terms of the ratio **Q/R**. A large Q/R says “the market really is changing—let’s accept the new value quickly.” A small Q/R says “this might just be noise—let’s react slowly.” Too fast, and you mistake noise for signal. Too slow, and you miss the regime change.

A thermometer is a good analogy. You believe the room is 22°C, then a new thermometer reading says 30°C. If the thermometer is known to glitch often, you do not believe 30°C right away. If instead someone opened the window and the room really is heating up fast, you trust the new reading more. The Kalman filter's Q/R turns this judgment into a number. Large Q/R says “the room temperature really may have shifted.” Small Q/R says “the thermometer might just be bouncing.”

A small numerical example makes it concrete. Suppose the prediction is 100 and the new observation is 110. If K is 0.2, the new estimate is $100 + 0.2 \times 10 = 102$. If K is 0.8, it is $100 + 0.8 \times 10 = 108$. The same observation of 110 only nudges the estimate when K is small, but pulls it strongly when K is large.

At steady state, P and K change little under recurring observations of similar quality. Repeat the equations with fixed Q and R and check that $\text{abs}(P_{\text{new}} - P_{\text{old}})$ becomes small. This stabilizes the fraction assigned to new information; it does not mean the data stop changing.

6.2 KalmanFilter1D —smoothing HMM features

KalmanFilter1D is a Kalman filter for a single-value time series. Every time a bar closes you call `update(z)` once, and it gives back the smoothed estimate.

The code below is the predict-update cycle from Section 6.1, written out almost word for word.

```
# app/trading/backtest/features.py

class KalmanFilter1D:
    """Scalar Kalman filter for feature smoothing."""

    def __init__(self, Q: float = 1e-4, R: float = 1e-2):
        self.Q, self.R = Q, R
        self.x = 0.0
        self.P = 1.0

    def update(self, z: float) -> float:
        self.P += self.Q
        K = self.P / (self.P + self.R)
        self.x += K * (z - self.x)
        self.P *= 1 - K
```



```
return self.x
```

What to check in the output: if you call `update(z)` repeatedly, the first few bars produce wildly swinging estimates, but the output gets smoother as you go. If you also print `P`, you will see it shrink toward a stable value.

Sanity check: if `P` settles to some value within the first 100 bars, things are healthy. If `P` only keeps growing, raise `R` or lower `Q`.

6.3 Per-feature Kalman parameters for the 4-dimensional input

Different features change at different speeds. Returns whip around; momentum drifts slowly. So we **set a separate (Q, R) pair for each feature dimension**.

Dim	Feature	Q	R	Reaction speed	Why
0	log_return	1e-4	5e-3	Fast	Returns swing quickly — reflect with no lag
1	5d_vol	5e-5	1e-3	Medium	Volatility clusters
2	20d_momentum	5e-5	1e-3	Medium	Momentum turns slowly
3	log_vol_ratio	1e-4	5e-3	Fast	Reflect volume spikes immediately

In the code these values are kept as constants (`KALMAN_PARAMS_4D`). The feature function (`compute_daily_features`) builds the four dimensions in the same order: log return, 5-day realized volatility, 20-day momentum, and volume ratio.

We then attach **its own independent KalmanFilter1D** to each dimension and run `update()` in time order. The important part is that **each dimension gets a fresh filter instance**. If you reuse one filter across all dimensions, the state (x , P) gets mixed and the result is meaningless.

Skip measurement updates for NaN and emit no result before the first valid observation. Innovation Z-score is $\text{innovation}/\sqrt{P_{\text{pred}}+R}$; reject nonpositive or nonfinite denominators. Exclude warmup from decisions because the initial P affects residuals, and record its duration in experiment settings.

6.4 `_kalman_innovations()` —pulling innovations out of price

Now we move on to the second role. We apply the same Kalman filter to a **log price series** and pull out the prediction error (the Innovation) at every step as a time series.

What this function does: for each step, build a one-step-ahead prediction of the log price and record the gap “prediction minus actual.” If the gap is positive, the price is above the Kalman-estimated level (overbought). If it is negative, the price is below (oversold).

```
# app/trading/backtest/signals/kalman_self_mr.py

def _kalman_innovations(log_prices, delta=1e-4, obs_noise=1.0):
    T = len(log_prices)
    innovations = np.full(T, np.nan)

    first = np.argmax(~np.isnan(log_prices))
    x, P = log_prices[first], 1.0

    for t in range(first + 1, T):
        y = log_prices[t]
        if np.isnan(y):
```

```

        continue

    innov = y - x
    innovations[t] = innov

    P_pred = P + delta
    K = P_pred / (P_pred + obs_noise)
    x = x + K * innov
    P = (1.0 - K) * P_pred

return innovations

```

The key line: `innov = y - x`. We use `x` before the update. In other words, the Innovation at time `t` is the difference between “a prediction built only from information up to `t-1`” and the actual observation at `t`. Doing it this way keeps future information out of the calculation (no look-ahead leakage).

Why the Innovation mean-reverts: when a stock price wanders away from its baseline level (the local level), the Innovation swings large, but it eventually returns to zero. That pattern is what we use as a mean reversion signal.

6.5 _build_z_series() —a causal Z-score

Compared to the simple rolling Z-score from Chapter 2: a rolling mean and standard deviation use a fixed window, so they adapt slowly to changes in volatility. The Kalman version adjusts its reaction speed automatically based on the noise model.

When we build a Z-score we also have to be careful not to leak future data. So we shift the rolling mean and standard deviation back by one bar (`shift(1)`) to **exclude the current bar itself from the statistics**.

```

def _build_z_series(spreads, window=60):
    s = pd.Series(spreads)
    roll_mu = s.rolling(window).mean().shift(1)
    roll_sd = s.rolling(window).std(ddof=0).shift(1)

```

```

z = ((s - roll_mu) / roll_sd).values
z[~np.isfinite(z)] = np.nan
return z

```

What `shift(1)` means: the mean and standard deviation used to compute the Z-score at time t are built from data up to $t-1$. The current bar's Innovation is left out of the statistics, so the Z-score is causal (look-ahead-free).

6.6 `_calc_halflife()` —estimating an AR(1) halflife

There is one more concept to know in mean reversion trading: the **halflife**. It captures “on average, how many bars does it take for the spread to revert to the mean?” A short halflife means fast reversion; a long halflife means weak mean reversion.

The estimation method is simple. Regress the change in the spread on “how far the spread is from its mean,” get the regression slope ϕ , and compute the halflife with this formula.

$$T_{1/2} \approx -\frac{\ln 2}{\phi}, \quad (\phi < 0 \text{ means mean-reverting})$$

Estimate half-life from trailing regression $\Delta s_t = a + b \cdot s_{t-1} + \varepsilon_t$. If $\phi = 1 + b$ is between zero and one, discrete AR(1) half-life is $\log(0.5)/\log(\phi)$ bars. For a small negative b , $-\log(2)/b$ is an approximation. $\phi > 1$ has no mean-reverting half-life under this model; negative ϕ is oscillatory and needs separate treatment.

What you really need to know: - If $\phi \geq 0$ the series is not mean-reverting. The code clamps these cases to a conservative ceiling (80 bars) to block bad entries. The same ceiling is used when there are fewer than 20 bars of data. - $hl > hl_max$ ($=60$) $\rightarrow$ reversion is too slow $\rightarrow$ **entry is blocked** - $eff_hold = \min(max_hold, \text{int}(hl_hold_mult \times hl)) \rightarrow$ **dynamic holding period** based on the halflife

6.7 KalmanSelfMRSIGNAL —entry and exit logic

This is the signal class that ties together the Innovation, the Z-score, and the halflife. During initialization it pre-computes the Innovation and Z-score series for each ticker.

```
@register_signal("kalman_self_mr")
class KalmanSelfMRSIGNAL(SIGNALGenerator):
    def initialize(self, data):
        for sym in self._symbols:
            log_p = np.log(np.maximum(self._close[sym], 1e-10))
            innov = _kalman_innovations(log_p, self._kalman_delta,
self._kalman_obs)
            self._spreads[sym] = innov
            self._z_series[sym] = _build_z_series(innov, self._z_window)
```

Default values for the key parameters:

Parameter	Default	Role
z_entry	3.0	Z-score threshold for entry
z_exit	0.3	Z-score threshold for exit
hard_stop_z	4.5	Hard stop on extreme Z-score
hl_max	60.0 bars	Maximum allowed halflife
price_stop	-8.0%	Price-based stop loss
max_hold	30 bars	Maximum holding period
kalman_delta	1e-4	Q (process noise)
kalman_obs	1.0	R (observation noise)

A four-filter entry

To open a position, all four conditions must hold **at the same time**. If even one is missing, we do not enter. More filters mean fewer bad entries.

Order	Condition	Meaning
1	<code> z >= z_entry (3.0)</code>	Has it wandered far enough?
2	<code> z_now < z_prev </code>	Is it starting to converge, not still widening?
3	<code>halflife <= hl_max (60)</code>	Is reversion fast enough?
4	Direction	<code>z < 0 -> LONG, z > 0 -> SHORT</code>

The score is set to `|z| × (1/halflife)`, which prioritizes tickers that “have wandered far and will come back quickly.”

A six-check exit

An open position is closed as soon as any one of these six conditions triggers.

Condition	Threshold	Meaning
<code>price_stop</code>	<code>PnL < -8%</code>	Absolute loss limit
<code>z_hard_stop</code>		<code>z</code>
<code>hl_degrade</code>	<code>halflife > 100 bars</code>	Mean reversion property has died
<code>z_converge</code>		<code>z</code>
<code>z_cross</code>	<code>z crosses to the opposite sign</code>	Overshooting and diverging
<code>max_hold</code>	<code>days >= hl_hold_mult × hl</code>	Dynamic holding period exceeded

Tight entry conditions and a variety of exit conditions are the typical shape of a mean reversion strategy. The idea is to cut losses fast and lock in gains once the target is reached.

6.8 kalman_self_mr vs. hmm_daily_mr

TIP

If you run a simple EWMA Z-score and a Kalman Z-score over the same period, the difference is clear. In sudden market shifts like the March 2020 COVID shock or the 2022 rate spike, the Kalman version reacts faster. EWMA uses a fixed decay coefficient, while the Kalman filter recomputes the signal-to-noise ratio at every step and adjusts to current conditions.

Both strategies use a daily Z-score to capture mean reversion, but they go about it differently.

Item	hmm_daily_mr	kalman_self_mr
Z-score base	Price moving average (20 bars)	Kalman Innovation (log price)
Regime filter	HMM 3-state	None (halflife filter instead)
Z entry threshold	1.5 / 2.0	3.0 (more conservative)
Z exit threshold	0.3	0.3
Holding period	Fixed 15 bars	Dynamic (based on halflife)
Maximum positions	5	60 (many tickers at once)
Allocation per trade	15%	3%

kalman_self_mr puts only 3% into each ticker, but holds up to 60 tickers at the same time. It spreads the bets wide rather than concentrating them in a few names.

6.9 A backtest run

Here is what it looks like when we put all of the components together and run an actual backtest.

```
from app.trading.backtest.signals.kalman_self_mr import
KalmanSelfMRSIGNAL
from app.trading.backtest.portfolio import PortfolioSimulator
from app.trading.backtest.data import download_bars

symbols = ["AAPL", "MSFT", "NVDA", "GOOGL", "META", ...]
data = download_bars(symbols, days=730, interval="1d")

sig = KalmanSelfMRSIGNAL(z_entry=3.0, hl_max=60.0,
                          kalman_delta=1e-4, kalman_obs=1.0)
sig.initialize(data)

result = PortfolioSimulator(max_positions=60,
                             per_trade_alloc=0.03).run(sig)
result.print_summary()
```

What to check in the output: `print_summary()` prints cumulative return, performance, maximum drawdown, the number of entries, and the average holding period. If you lower the Z entry threshold from 3.0 to 1.5 and run it again, you will see entries become more frequent but the average win rate drop.

Exercises

1. Compute Z-scores of the SPY-QQQ spread under two settings — $Q=0.01/R=10$ and $Q=1.0/R=0.01$ —and compare the difference around March 2020 in a plot.
2. Add NaN-observation handling to `KalmanFilter1D.update(z)`. If five or more NaNs arrive in a row, hold the predicted state without updating it.
3. Apply both the Kalman Z-score and a simple 20-day rolling Z-score to the same pair, and compare which one has fewer false positives.

Things to remember from this chapter

- **The Kalman filter is not a simple moving average.** It weighs the uncertainty of its prediction against the trustworthiness of the new observation, and updates the estimate accordingly.
- **The Q/R ratio decides reaction speed.** Large Q/R reacts fast; small Q/R reacts slowly.
- **Innovation = prediction - actual.** When this value swings away from zero and comes back, we use that as a mean reversion signal.
- **A causal Z-score** uses `shift(1)` to exclude the current bar from its statistics, blocking future-information leakage.
- **A short halflife means faster mean reversion.** If the halflife is too long (over 60 bars), we block entries.

Concept	Key point
Local level model	$F=1$, state = price level, Innovation = prediction error
Kalman gain K	Decided by the Q/R ratio —higher K reacts faster, lower K smooths more
4-D feature smoothing	An independent <code>KalmanFilter1D</code> per dimension, see <code>KALMAN_PARAMS_4D</code>
Innovation Z-score	A <code>shift(1)</code> causal Z-score that detects overbought/oversold
Halflife	Estimated via AR(1) OLS; entry blocked when over 60 bars
Dynamic holding period	<code>eff_hold = hl_hold_mult × halflife</code>

Next chapter (Chapter 7): the Cross-asset Joint HMM. Instead of looking at one asset, we model how several assets move together —their covariance structure —at the same time. This lets us catch market regimes more stably.

Further reading

- **Book:** Grewal & Andrews, *Kalman Filtering: Theory and Practice* —goes deep into parameter tuning and numerical stability.
- **Paper:** “Pairs Trading with Kalman Filtering” (Pole, 2007) —the first paper to apply the Kalman filter to spread tracking.
- **Library:** `pykalman` —a Python Kalman filter implementation; it can estimate parameters automatically via the EM algorithm.

Chapter 7. Cross-Asset Joint HMM —Regime De- tection Across Multiple Markets

Most individual investors stare at a single chart. They check that stock's trend lines, moving averages, and volume, then try to predict where it's going next. But this approach has a blind spot. If you don't look at the state of the entire market that stock belongs to —whether investors are willing to take on risk or are fleeing to safe assets —you can miss important warning signs.

Institutional traders work differently. They don't analyze a single stock in isolation. They watch what several markets are saying at the same time. For example, when the market shifts to a

Risk-Off state, that change tends to show up in bonds and credit spreads (the gap between corporate bond yields and Treasury yields—a wider gap means investors are more worried about corporate defaults) before it shows up in stock prices. If you only look at stocks, you only recognize the danger after the decline has already started. If you watch bonds and volatility measures as well, you can pick up on weakening risk appetite earlier.

The market drop in March 2020 is a good example. Before stocks fell sharply, there were already leading signals: falling Treasury yields, a rising VIX, and widening credit spreads. A model that only sees stock prices never receives those signals.

The starting point for this chapter is simple. To read the market regime properly, don't just watch one market—watch what several markets are saying at the same time.

A Joint HMM is one way to put that idea into code. The word “Joint” here means “across several assets together.” Instead of building one HMM for SPY and a separate one for TLT, you feed the observations for SPY, TLT, HYG, and VIX into a single model at the same time. Now, when the model judges the regime, it uses all of the cross-market information at once. If a single-asset HMM is a model that watches “SPY has already moved,” a Joint HMM is a model that watches “what are bonds, credit, and volatility saying right now, together?”

There's one wrinkle. Each asset uses a different scale. The VIX moves between 10 and 80, while SPY daily returns move between -5% and +5%. If you feed the raw values into a single model, the large-valued features will dominate everything else. So we standardize each asset against its own past distribution (a rolling z-score). Each value is restated as “how high or low is this compared to my recent history?”

The system in this chapter has three stages. The **Macro HMM** judges “is the overall market in risk-on or risk-off mode right now?” The **Micro HMM** judges “is this individual asset worth buying right now?” **Kelly** turns those judgments into an actual position size. The reading order is overall market direction → individual asset signal → position size.

The asset count grows, and a tool like PCA (a dimensionality reduction technique) appears, so things can look complicated. But if you hold those three stages in your head, the details will be much easier to follow.

For a beginner, do not read this chapter as “a harder HMM.” Read it as **an HMM with a wider screen**. The HMM in Chapter 4 was like measuring one stock's temperature. The Joint HMM in Chapter 7 is closer to a health check that looks at temperature, blood pressure, pulse, and oxygen level together.

Single-asset HMM

Watch only SPY and infer the market state

|

v

May notice risk only after stocks have already moved

Joint HMM

Watch SPY + TLT + HYG + VIX + GLD together

|

v

Includes early warnings from bonds, credit, and volatility

The key point is not simply that there are more features. The key point is that warnings from different markets are bundled into one observation vector for the HMM to read.

data -> feature vector -> HMM training -> regime probability inference
-> position-size adjustment

The flow is still the same as Chapter 4. Only the width of the feature vector changes. Chapter 4 looked at a few numbers such as return, volatility, and momentum. Chapter 7 looks at about twenty numbers that summarize several markets at once.

What this chapter covers

- The full Pipeline —six stages
- Why go cross-asset
- Noise reduction at the bar level (Kalman filter)
- Building the 20-dimensional session observation vector
- Layer 1 —Macro HMM (daily)
- Layer 2 —Micro HMM (session)
- Applying `macro_bias` to Kelly position sizing

Learning goals

By the end of the chapter, you will be able to:

- Explain why we feed multiple assets into a single HMM instead of just one stock.
- Describe why we use a Kalman filter to denoise returns.
- Understand how 18 assets + 2 microstructure indicators combine into a 20-dimensional observation vector.
- Explain how Layer 1 (Macro) and Layer 2 (Micro) split responsibilities.
- Understand how `macro_bias` is multiplied into Kelly sizing to combine market direction with the individual asset signal.

Chapter code: `run_rentec_joint_hmm_v7.py`

This chapter focuses on why cross-asset observations and two HMM layers produce more stable regime judgments than a single-asset HMM can.

7.0 The full Pipeline —six stages

The single-asset HMM from Chapter 4 only took SPY prices as input. So it missed signals like a sharp bond rally or dollar strength. In this chapter we expand from “just SPY” to a Joint HMM that learns from SPY + TLT + USD + VIX and others at the same time.

NOTE

Compared to watching only SPY, watching SPY + TLT + USD + VIX together detects regime turns an average of **2.3 days earlier** (based on a 2018–2024 backtest). The reason is that bonds, gold, and the VIX often react before stocks do.

Let’s look at the full flow as a table first.

Stage	What happens
1. Asset download	5-minute bars for 17 observation assets + 24 tradeable assets
2. Kalman filter	Denoise each asset's bar returns
3. Session aggregation	Bundle 78 five-minute bars into one session, build the 20-dim observation vector
4. Macro HMM	At the daily timescale, judge overall market direction (<code>macro_bias</code>)
5. Micro HMM	At the session timescale, compute each asset's up-probability (<code>p_up</code>)
6. Kelly sizing	Multiply <code>p_up</code> and <code>macro_bias</code> together to decide position size

Why two HMMs?

- The **Macro HMM** judges the direction of the whole market. “Is this a time to take risk, or to avoid it?”
- The **Micro HMM** judges entry and exit timing for individual assets. “Within the conditions the Macro HMM has set, is this asset worth buying right now?”

Splitting these two layers makes it possible to trace, after the fact, why the system made a given decision. For instance, even if the Micro HMM rates AAPL's up-probability as high, if the Macro HMM is pointing at Risk-Off, the position size shrinks. In the opposite case — Macro is Risk-On and Micro also signals an upside —the position grows. In short, “even a good individual signal goes in small when the overall market is dangerous”—that operational principle is encoded directly in the structure.

7.1 Why Cross-Asset?

WARNING

A single-asset HMM judges the regime from stock prices alone. But in reality, when the VIX spikes, the next bar’s stock return distribution itself shifts. In other words, other assets’ moves affect stock prices. To capture that reality, the model needs cross-asset features.

The problem with single-stock HMMs is timing. They only flip to the “risk” regime after stocks have already fallen. By the time the warning light turns on, the loss has already happened.

The cross-asset approach is different. By using signals from assets that tend to move before stocks —bonds, gold, and the VIX—we cut that lag down.

Aspect	Single-stock HMM	Cross-asset Joint HMM
Observations	SPY returns only	SPY + TLT + GLD + HYG + UUP + ...
Regime signal	Detected after stocks drop	Bonds/gold/VIX rise first -> earlier detection

Which asset moves before which can be measured from the data (a lead-lag relationship). For example, the dollar index (UUP) tends to lead gold (GLD) by about three days, and the Nasdaq (QQQ) tends to lead high-yield bonds (HYG) by about three days. The full table from the automated measurement is available in the repository.

7.2 Bar-level noise reduction —Kalman filter

Raw 5-minute returns are noisy. A handful of outliers can be enough to push the model into the wrong regime. So we first pass each asset’s bar returns through a Kalman filter to smooth them, then feed those smoothed values to the model.

A Kalman filter is a tool that estimates the “true” signal by taking a weighted average of “everything observed so far.” It’s similar to a moving average, except it automatically adjusts the weights every time a new observation arrives.

The filter in this chapter has one quirk. It gently pulls the previous estimate back toward 0. When you track a price level (say, SPY at \$450), the assumption “the next value will be similar” makes sense. But returns oscillate around 0. So we softly bake in the assumption “in the absence of any special signal, returns revert to 0.”

The filter is also **causal**. It updates from past and present values only—it never peeks at the future. That means backtests won’t suffer from look-ahead bias (future information leaking in).

A Kalman step predicts the prior estimate and covariance through F , then corrects the estimate by gain K times the observation error. F is state transition, Q state uncertainty, and R observation noise. `fit_series` repeats this in time order. Start with Chapter 6’s scalar $F=1$ case; for multiple assets also verify dimensions and timestamp alignment.

7.3 The 20-dimensional session observation vector

The trading day is split into 5-minute intervals. When 78 of those 5-minute bars accumulate, you get one **session**. For each session, we compute per-asset returns and volatilities and compress them into 20 numbers. That bundle is the **observation vector**. The HMM takes one such vector at a time and judges the regime.

The 20 dimensions are organized into three groups.

Group	What’s in it	Meaning
A —Core assets (10)	SPY, QQQ-SPY, TLT, SPY volatility, GLD, USO, UUP, EEM, HYG, XLE	Market direction, safe haven, macro
B —Sectors & international (8)	IWM, XLF, VNQ, EFA, XLV, XLI, XLP, UNG	Relative strength/weakness across sectors
C —Micro (2)	SPY open-to-close return, SPY-QQQ intra-session correlation	Intra-session microstructure

Why these 18 assets? Three criteria.

1. **Regime leading-ness:** bonds (TLT, HYG), gold (GLD), and the VIX move before stocks.
2. **Correlation diversity:** throwing in more assets that move alike adds no new information. SPY, QQQ, and IWM represent the market-cap regime; TLT represents the interest-rate regime; GLD and the VIX represent the fear regime.
3. **Data availability:** a long enough history is required.

Before you change the asset set, check two things first: that the new asset isn't too highly correlated with the existing ones, and that it has enough historical data.

Macro sensitivity per asset class

Asset groups can include equity, em, commodity, credit, fx, safe_haven, and volatility. Reducing equity and retaining defensive assets in Risk-Off is a hypothesis; gold and bonds are not always defensive. Compute base weight times regime scale and apply aggregate risk limits. Record asset group, regime, multiplier, and validation interval for each table row.

7.4 Layer 1 —Macro HMM (daily)

Layer 1 reads the market's big-picture flow from **daily** data. The input is a 10-dimensional vector: 5-day rolling returns for the major assets (SPY, TLT, GLD, HYG, etc.) plus SPY volatility and the SPY-TLT correlation. The HMM splits the market into three regimes (Risk-Off / Neutral / Risk-On).

Every day, when a new observation comes in, the Forward algorithm updates “the probability that we are in each regime right now.” The result is a distribution like [Risk-Off 0.7, Neutral 0.2, Risk-On 0.1].

macro_bias —squeezing the distribution into one number

Using the full three-way distribution directly is awkward to plug into Kelly. So we reduce it to a single number. We take the expected SPY 5-day return in each regime, weight it by the current regime distribution, then divide by the standard deviation to fit the result into the range -1 to +1.

- `macro_bias` near -1 means Risk-Off (defensive)
- `macro_bias` near +1 means Risk-On (offensive)

Regime	SPY 5-day return	TLT 5-day return	SPY volatility
Risk-Off	Negative (↓)	Positive (↑)	High
Neutral	Neutral	Neutral	Low
Risk-On	Positive (↑)	Negative (↓)	Low

In Risk-On, stocks rise, bonds fall, and volatility is low. Risk-Off is the opposite. Neutral is the quiet state where neither side moves much.

Why enforce regime persistence?

Encourage persistence with larger diagonal prior pseudocounts. In a three-state example, diagonal 8 and off-diagonal 1 prefer staying in-state before fitting. Excessive persistence delays real changes. Compare convergence, occupancy, and OOS detection delay across seeds; reject failed fits.

7.5 Layer 2 —Micro HMM (session)

Layer 2 detects short-term regimes at the **session** level. It uses four regimes, and its input is the 20-dimensional observation vector built in Section 7.3.

Reduce dimensions with PCA first.

Using all 20 dimensions directly causes two problems.

1. **Not enough data.** Compared to the 360-session training window, the model has too many parameters.
2. **Information overlaps.** The SPY-QQQ correlation runs around 0.9. Two dimensions are passing along almost the same information twice.

PCA (Principal Component Analysis) solves both problems at once. It compresses the 20 dimensions into 15 dimensions that are mutually independent. The result is what we feed into the HMM.

PCA_DIM=15, N_ENSEMBLE=3, TRAIN_SESSIONS=360, and REFIT_SESSIONS=40 are illustrative dimension, fit count, training history, and refit interval. PCA dimension cannot exceed available

observations or features. Fit scaler and PCA only on training data and freeze both until refit. These are not proven universal settings.

7.6 The walk-forward setup

If we train the model once and stop, it can't keep up when the market's structure changes. So we retrain on a fixed schedule. This approach is called **walk-forward** validation.

0 ————— 360 | 361 — 400 | 401 — 440 | ...
 └─ train 360 ─┘ OOS 40 retrain + OOS 40

At every retraining step, the Micro HMM is re-fit on the most recent 360 sessions. Because the volatility distribution for each asset shifts over time, the standardization tool (Standard-Scaler) and PCA are re-fit alongside it. This keeps the model adapting as the market's structure changes.

7.7 macro_bias -> Kelly modulation

Now we combine the outputs of both layers into one decision. We start from Layer 2's `p_up` (the up-probability) and scale it using Layer 1's `macro_bias`.

```
def kelly_fraction(p_up, macro_bias, asset_class):
    edge = 2 * p_up - 1.0
    class_amp = MACRO_CLASS_AMP.get(asset_class, 0.5)
    adjustment = macro_bias * class_amp
    kelly_adj = edge * (1.0 - adjustment)
    return float(np.clip(kelly_adj * 0.5, KELLY_MIN, KELLY_MAX))
```

Let's walk it through with numbers. Say Micro reports `p_up=0.58` (a weak upside signal), Macro reports `macro_bias=-0.7` (Risk-Off), and the asset is equity.

- Edge: $2 \times 0.58 - 1 = 0.16$
- Macro adjustment: Risk-Off + equity -> shrink the position
- Final position: roughly 0.108 (about two-thirds of the original 0.16 edge)

In one sentence: **“Even when Micro gives an upside signal, if Macro is Risk-Off, equity positions shrink.”** The same signal goes in at different sizes depending on the market environment.

7.8 Computing P(up) —how the up-probability is built

How is the p_{up} that Layer 2 outputs actually built? For each regime, we compute “the probability that this asset’s next return is greater than 0,” then weight-average those by the current regime distribution. For example, if the up-probability for an asset is 70% in Risk-On, 50% in Neutral, and 30% in Risk-Off, and the current regime distribution is $[0.1, 0.2, 0.7]$, then $p_{up} = 0.1 \times 0.7 + 0.2 \times 0.5 + 0.7 \times 0.3 = 0.38$.

The trading thresholds are simple. $p_{up} > 0.53$ means long, $p_{up} < 0.47$ means short, and anything in between is neutral.

7.9 The structural choices behind the results

The table below summarizes the structural choices that drove the performance improvements. The point is less the numbers themselves and more which design choice reduced which problem.

Element	Effect
20-dim cross-asset observation	Detects regimes earlier than a single asset can
Macro + Micro two-layer structure	Combines daily directional bias × session up-probability
Forced regime persistence	Prevents excessive flipping from noise
Walk-forward retraining	Keeps adapting to market structure changes
15-dim PCA	Removes collinearity and stabilizes training

Element	Effect
Half Kelly $\times$ macro_bias	Asymmetric position adjustment per asset class

Try it in code —the Macro HMM training flow on synthetic data

Use the synthetic exercise to trace observation vector $\rightarrow$ HMM fit $\rightarrow$ regime inference before acquiring market data. For markets, align asset timestamps, derive returns/volatility, and transform with the training-fitted scaler. The exercise needs neither downloads nor the production repository.

What this snippet shows: build 500 twenty-dimensional observation vectors $\rightarrow$ train the Macro HMM on 60% of them $\rightarrow$ run the Forward algorithm to infer regimes. That's the minimal flow.

```
import numpy as np
from hmmlearn.hmm import GaussianHMM

SEED, T = 42, 500
N_MACRO_STATES = 3
rng = np.random.default_rng(SEED)

X = rng.normal(0, 1, size=(T, 20))

train_len = int(T * 0.6)
macro_hmm = GaussianHMM(
    n_components=N_MACRO_STATES,
    covariance_type="full",
    n_iter=200,
    random_state=SEED,
)
macro_hmm.fit(X[:train_len])
```

```

alpha_oos = []
for t in range(train_len, T):
    alpha_t = macro_hmm.predict_proba(X[:t + 1])[-1]
    alpha_oos.append(alpha_t)

alpha_oos = np.asarray(alpha_oos)
regime_oos = alpha_oos.argmax(axis=1)

print(f"관측 벡터 shape: {X.shape}")
print(f"학습 완료: {N_MACRO_STATES} 상태")
print(f"OOS 레짐 분포: {np.bincount(regime_oos)} (OOS {T - train_len}봉)")

```

What to check in the output: - Observation vector shape: (500, 20) —that the time × dimension shape is right. - Regime distribution: [..., ..., ...] —that none of the three regimes is zero (if only one regime ever fires, training failed).

If it fails, look here: - If only one regime shows up, try changing random_state or check that the data has enough variability. - The internal `hmmlearn` functions `_compute_log_likelihood` and `_do_forward_pass` are private APIs. The interface may change across library versions.

X has timestamps as rows and return/volatility features as columns. Freeze column order from training to inference. Never fill a market holiday with a future price; exclude insufficient observations or mark them invalid under an explicit missing-data policy. Synthetic outputs are not investment performance.

Exercises

1. When the Macro HMM points at Risk-Off and the Micro HMM predicts an up-probability of 0.58 for SPY, how is the final position size determined? Walk through the formula in Section 7.7 yourself.
2. Suppose you add gold (GLD) to the current macro asset set. Form a hypothesis about how regime detection would change, and judge whether that addition meets the asset selection criteria from Section 7.3.
3. Think about the pros and cons of shortening the walk-forward retraining interval.

Summary

What to remember from this chapter:

- To read the market regime correctly, don’t watch just one asset —watch several together. Bonds, gold, and the VIX often move before stocks do.
- The system has two layers. The **Macro HMM** judges overall market direction (macro_bias); the **Micro HMM** judges each asset’s up-probability (p_up).
- The two outputs are multiplied together to set the Kelly position size. Even a good individual signal goes in small when the overall market is dangerous.
- The model isn’t trained once and left alone —it is continuously retrained walk-forward.

Layer	Input	Output	Role
Macro HMM (daily, 3 states)	10-dim major assets	macro_bias ∈ [-1, +1]	Macro directional bias
Micro HMM (session, 4 states)	18 assets + micro (20-dim) -> 15-dim PCA	per-asset p_up	Per-asset directional probability
Kelly sizing	p_up × macro_bias × asset class	Position size (%)	Risk adjustment

Next chapter (Chapter 8): A deeper look at the two-layer HMM. We cover the ensemble sub-model structure and how to compute the probability that a regime transition is imminent.

Further reading

- **Paper:** Ang & Bekaert (2002), “International Asset Allocation with Regime Shifts”—the original cross-asset regime model.
- **Paper:** Hamilton (1989), “A New Approach to the Economic Analysis of Nonstationary Time Series”—the foundation of Markov switching models.
- **Book:** Ilmanen, Expected Returns (Wiley, 2011) —research on cross-asset correlations and regimes.

Chapter 8. Two-Layer HMM Ensemble —Layer A × Layer B

Markets don't always move the same way. Sometimes prices keep drifting in one direction — that's a **trending market**. Other times they bounce inside a range —that's a **sideways market**. An indicator like RSI behaves differently in those two situations. Breakout signals behave differently when volatility is high than when it's low. A veteran trader looks at a chart and instinctively knows whether to “follow the trend” or “wait for a bounce.” But that instinct varies from person to person and isn't consistent, so it's hard to turn into code.

Most retail traders use the same signals regardless of the market situation (the **regime**). They keep using a momentum signal that works in strong trends even when the market is sideways. They keep entering on the usual rules even during high-volatility stretches. The signals themselves aren't bad. The problem is failing to separate the situations where they work from the situations where they don't.

If the word **regime** feels unfamiliar, think of the weather. The same clothes aren't right for both spring and midwinter, and no single trading signal can be good in every phase of the market. Markets also have seasons. We need to identify the season first.

Quick recap of Chapters 4, 5, and 7 – In Chapter 4 we built a single-asset HMM and learned the idea of “hidden states.”– In Chapter 5 we implemented `step_forward()`, which updates state probabilities bar by bar using the Forward recursion. – In Chapter 7 we extended the model to handle several assets together with a Joint HMM.

This chapter stacks **eight sub-models** on top of that foundation and bundles different sources of alpha into one Ensemble. As long as you remember that an HMM is “a model that estimates hidden states from observed data,” you can follow along.

The two-layer HMM ensemble solves the regime problem structurally. **Layer A (the macro HMM)** looks at several assets together and classifies the current market state. **Layer B** holds eight specialist sub-models, each designed to work best in a particular kind of market. When Layer A says “this is a state that favors mean reversion,” the mean-reversion specialist's signal becomes active. When it says “this is a trending regime,” the momentum specialist gets more weight.

This structure is the same idea as Mixture of Experts in machine learning —pick the right specialist for the situation instead of relying on one all-purpose model. The point is not to lean on a single magic signal. We pick the strategy that fits the current situation and reduce the weight of strategies that don't.

For a beginner, think of the two-layer structure like an organization. Layer A is the team lead watching the whole market. Layer B is a group of specialists, each good at a different kind of short-term signal. If the team lead says “today is a crisis regime, reduce aggressive signals,” even a strong signal from one specialist is scaled down before it reaches the final position.

The reason for splitting the model into two layers is to avoid asking one model to solve too many different questions at once. A daily macro regime and a 5-minute entry timing live on different time scales. If both are forced into one model, the model has to decide whether it should learn the broad market environment or the short-term entry pattern. Two layers make the questions cleaner.

Layer A asks:

What kind of market are we in overall?

Layer B asks:

Given that market, what short-term signal do we see now?

Final combination asks:

Should this signal become a position, and how large?

Chapter Roadmap

We start with why two layers are needed, then drill down into each layer and how they're combined.

- **8.1 Why Two Layers?** —Why we separate signals that live on different time scales
- **8.2 Effect First** —Does this structure actually pay off?
- **8.3 Layer A** —A 7-state HMM that tracks the macro regime
- **8.4 The 8 Sub-Models in Overview** —Which market each one targets
- **8.5 SubModelSpec** —A declarative data structure for sub-models
- **8.6 HMMForwardPredictor** —Training and inference for each sub-model
- **8.7 W2 Wasserstein Voting** —Suppress outliers and reinforce the majority
- **8.8 ~ 8.10 Combination and Validation** —Final signal assembly, operating principles, and summary

Learning Objectives

By the end of this chapter you should be able to:

- Understand that the system has two layers. Layer A classifies the overall market state (the macro regime), and Layer B takes that result and combines eight small predictive models. You can explain each layer's role and why it's designed that way.
- Identify, at a high level, which market situation each of the eight sub-models targets.

- Explain why W2 Wasserstein voting is safer than a plain average —it suppresses outlier predictions and reinforces the majority view.
- Explain what a regime gate (`active_regimes`) is and why it matters.
- Understand what `RenaissanceEnsemble5m.step()` returns through `e_sym`, and use it correctly in your backtest code.

8.1 Why Two Layers?

If you train a single HMM on 5-minute bars, you’re asking one model to learn two kinds of signal that have very different characters. The macro environment, on the scale of daily bars, changes slowly over days or weeks. Entry timing on 5-minute bars, on the other hand, can flip within a few bars.

Two layers may look complicated at first, but each layer’s job is simple.

- **Layer A** decides “what’s the overall mood of the market today?”
- **Layer B** decides “given that mood, is the current 5-minute signal worth trading?”

If you stuff both judgments into one model, the model loses its sense of which pattern to learn. Two layers aren’t decorative —they’re a structure for separating two judgments that operate on different time scales.

Time scale	Phenomenon	Required model
Day	Macro regime: trend / MR / crisis	Layer A —7-state HMM
Bar (intraday)	Short-term direction, VWAP deviation, Kalman velocity	Layer B —8 sub-model Ensemble

Layer A controls the **size** of the signal through `regime_mult`. Layer B’s sub-models vote on the **direction (sign)**. The final signal is the product of the two.

`final signal = e_sym (Layer B direction) × regime_mult (Layer A size)`

That single line is the conclusion of this chapter. Everything that follows is about how we build `e_sym` and `regime_mult`.

The training cadences are different too. Layer A trains on the daily scale so it can catch slow macro shifts, while Layer B trains on a window of roughly ten sessions of intraday bars so it can catch quick short-term changes.

8.2 Effect First —Backtest Performance

Before going into details, let's look at why this structure is needed. A single model tends to overfit to a particular regime. The reason we add two layers and W2 voting is to avoid relying on one model's lucky strength—we want to gain confidence only when several signals point the same way.

Version	Change	What to watch for
Baseline	7 states, performance-weighted only	The limits of weighting by performance alone
With W2 voting	Wasserstein + k-means majority	Signal stability during regime transitions

The detailed numbers are in the experiment notebook in the repository. In the main text, focus on “what failure mode shrank when we added what.”

In 8.7 we'll see directly where this difference comes from. For now, just remember the takeaway: “two layers plus W2 voting are more cautious around regime transitions than a plain average.”

8.3 Layer A: GlobalRegimeLayer

Layer A is implemented as the `GlobalRegimeLayer` class in `hmm_v5_core.py`. It's a kind of HMM, and it distinguishes seven market states.

The input is 5-minute bars, but the goal isn't to track short-term moves in individual stocks. The model follows macro features that describe the whole market —volatility, volume, sector rotation —and from those it figures out the current mood of the market.

Each of the seven regimes has a pre-set signal multiplier, `regime_mult`.

state	Name	Multiplier	Description
0	DeepMR	0.55	Extreme low-vol mean reversion -> signal damped
1	QuietTrend	1.00	Low-vol drift (baseline multiplier)
2	TrendMod	1.10	Mild trend
3	TrendStr	1.40	Strong trend
4	TrendBreak	1.70	Breakout / momentum —maximum amplitude
5	HighVol	0.35	High-vol noise -> signal damped
6	Crisis	0.10	Crisis / crash -> nearly flat

For example, if Layer B emits `e_sym = +1.0` (a strong long) and Layer A says we're in TrendBreak (state 4), the final signal becomes $1.0 \times 1.70 = 1.70$. The same `e_sym` in Crisis (state 6) becomes $1.0 \times 0.10 = 0.10$, almost flat.

Group the states into three buckets and the intent becomes clear.

- **MR / crisis group (0, 5, 6):** the market is shaken or in distress. Suppress directional signals to shrink position size.
- **Trend group (1, 2, 3):** the trend is alive. Apply baseline-to-strong multipliers.
- **Breakout group (4):** the market breaks out of its range and momentum kicks in. Push the multiplier up to 1.70.

These mapping values live in the `_REGIME_A_SCALE` constant dictionary in the code.¹

¹Layer A's state count doesn't have to be 7. So that you don't have to redo the multiplier table when experimenting with 5 states, there's a helper (`_regime_mult_from_rank()`) that automatically maps any number of states to the table above.

8.3.1 Regime Stabilization —Three-Bar Confirmation

Every time a new 5-minute bar arrives, the probabilities for each state are updated. If you immediately treat the highest-probability state as the regime, the label can change bar by bar. Markets rarely flip mood within a single bar, so most of these rapid changes are noise.

That's why we use **three-bar confirmation**. A new state must stay above the probability threshold for three bars (= 15 minutes) in a row before the regime label changes. If the regime label flips on every bar, the strategy changes character on every bar too, and that leads to overtrading and unstable positions.

8.4 Layer B: Overview of the Eight Sub-Models

Now we drop down to Layer B. Before looking at the class layout, let's lay out **what each of the eight sub-models is watching**. The sub-models split into two groups. Four are HMM-based, and four are non-HMM models built from Kalman filters, moving averages, volume, and similar tools. The non-HMM models independently confirm signals the HMM models might miss.

8.4.1 HMM Sub-Models (Two Representatives)

Model	Feature idea	Active regimes	Role
M3_vwap	VWAP deviation + BB position	All regimes	General-purpose mean reversion + sharp-drop short
M5_RetKalman	5/10/20-bar returns + Kalman velocity	Trend (1-4)	Catches trend direction

M6_VolKalman focuses on smoothed volatility observations and M7_MRFast on a short mean-reversion window. Specify each model's features, trailing window, active regimes, and output units. Duplicating the same information across more models does not automatically diversify risk.

Why the regime gate? We don't leave every model on at all times. Different signals work in different regimes.

- In trend regimes (1-4) we turn off the volatility-only model (M6). Signals that ignore price direction only add noise during trends.
- In the Crisis regime (6, sharp drops) we turn off the trend-following model (M5). Chasing momentum during a crash tends to add to the losses.

8.4.2 Non-HMM Sub-Models (Two Representatives)

The remaining four models are simple rule-based and don't use HMMs. Let's just look at two of them.

Model	model_type	Voting basis
M8_Kalman	kalman	Sign of Kalman-smoothed returns -> long-term trend direction
M10_VolBreak	volbreak	Votes on direction only on bars with unusually high volume

The other non-HMM models (M11_EMAMom, M12_TODRet5b) are listed in the appendix table. M9_DailyMom (daily momentum) from a previous version was removed because its signal magnitude was on a very different scale than the others.

The contribution of each sub-model —measured by ablation, where each one is removed in turn and the change in performance is recorded—is summarized in the repository's experiment notebook. In short, the mean-reversion models (M3 and friends) contribute the most.

8.5 SubModelSpec —Declaring Sub-Models as Data

Now that you've seen the kinds and roles of the sub-models, the next natural question is how these eight are handled in code. Layer B's sub-models aren't declared as classes—they're declared as **data**. The SubModelSpec dataclass holds every setting for one sub-model. To

add a new sub-model, you don't write a new class; you just add one `SubModelSpec` object to a list.

In the main text we'll look at only three key fields.

Field	Meaning
<code>name</code>	Model identifier (e.g., "M3_vwap")
<code>feat_cols</code>	List of feature columns this model uses
<code>active_regimes</code>	Layer A regime gate —None means active in all regimes

`n_states` is the number of latent states; `model_type` identifies the model; `mu_clip_bp` caps the absolute state mean return in basis points. A 20bp cap means -0.002 to 0.002 . `active_regimes` selects which upper-level regimes permit the output. If no model is active, abstain rather than treating zero as a confident forecast.

From here on, the focus shifts to implementation details. First-pass readers can skim 8.6 and 8.7 and then jump to 8.8. If you read the combination flow in 8.8 and the summary in 8.10, you'll have the full picture of this chapter. The details can wait for a second pass.

8.6 HMMForwardPredictor: Layer B HMM Predictor

Each HMM sub-model is wrapped in the `HMMForwardPredictor` class. It does two things.

1. **`fit()`**: trains the HMM on past data and computes the expected return for each state.
2. **`step()`**: every time a new bar arrives, updates and returns the current expected return.

8.6.1 `fit()` —What the Training Step Does

Compare candidate state counts with BIC $k \cdot \log(n) - 2 \cdot \log L$: k is free parameters, n training observations, and $\log L$ likelihood on the same data. Lower BIC balances fit and complexity, not trading profitability. Count parameters consistently for the covariance form. Clipping state means is a separate policy to prevent small-sample extremes from dominating size.

What to check in the output: `fit()` returns `True` if training succeeded. `False` means the input data was too short.

8.6.2 `step()` —Updating Expected Return Each Bar

After training, you call `step(x_bar)` every 5-minute bar. It updates the probability of each state with the new observation, then returns a weighted sum of the per-state mean returns to give the current expected return. It's the same pattern as the single-HMM `step_forward()` from Chapter 5, with the difference being that four HMM sub-models now independently run the same calculation instead of one model.

If it fails, look here: if any of the input features contain NaN, the probability update can break. Filtering NaNs in feature preprocessing comes first.

8.7 W2 Wasserstein Voting —Suppressing Outliers

If you just average the predictions of the eight sub-models, you get a problem: **one outlier sub-model can contaminate the entire signal**.

A small example makes this concrete. Suppose seven of the eight models produce a weak LONG (say, +0.2) and one produces a strong SHORT (-2.0). A plain average gives $(7 \times 0.2 + 1 \times (-2.0)) / 8 \approx -0.075$ —the opposite direction of the majority. One outlier flipped the whole thing. W2 voting pushes the outlier's weight to nearly zero and boosts the majority camp, pulling the result back toward the majority's weak LONG.

The Wasserstein distance (W2) is one way of measuring how different two probability distributions are. Unlike other distance functions (such as KL divergence), it always returns a finite value, so it stays stable even when predictions are extreme.

W2 voting works in two phases.

Phase 1—suppress the outlier. Each sub-model outputs a Gaussian-shaped prediction distribution (mean and variance). We compute the W2 distance between every pair of models, and use “how far it is on average from the others” as a measure of outlier-ness. The more it stands out, the closer to zero its weight becomes; models close to the majority keep normal weights.

Phase 2—boost the majority cluster. Phase 1 alone causes the signal to collapse near zero when opinions split fifty-fifty between LONG and SHORT. Phase 2 splits the sub-models into two camps (LONG and SHORT), boosts the majority camp, and shrinks the minority. The “majority” here is measured by **the total magnitude of the signals, not the number of models**. If three LONG models give weak signals while two SHORT models give strong ones, the SHORT camp wins by majority.

One distance-to-weight example is $w_i = \exp(-d_i/T) / \sum_j \exp(-d_j/T)$, $T > 0$. Nearby distributions receive more weight, but similarity does not establish return-prediction quality. Normalize distance and performance components separately and precommit a mixture ratio before validation.²

8.8 RenaissanceEnsemble5m.step() —Combining the Two Layers

`step()` gathers the outputs of the eight sub-models into the final trading signal. The most important return value is the first one, `e_sym`. It’s the **direction signal from Layer B**, formed as the W2-weighted sum of the surviving sub-models. The other return values (a dictionary of per-sub-model signals, long/short vote counts, and so on) are used for debugging and safety checks.

Inference proceeds through finalized-bar validation, upper-level update, active-model selection, causal predictions, weight updates from prior predictions with already-realized outcomes, and aggregation/risk limits. Using the current prediction’s future return to set its weight is leakage. Record whether update intervals are bars or sessions.

The conclusion of 8.1—`final signal = e_sym × regime_mult`—looks like this in code.

²The actual code has a blend ratio between performance-based weighting and W2 weighting. It’s currently set to 0, so only performance weighting is used; the W2 blending code is kept for future re-experiments.

What this code does: every 5-minute bar, it multiplies Layer B's direction signal by Layer A's regime multiplier to form the final position signal. When opinions split, it goes flat as a safety measure.

```
for bar_idx in range(train_bars, len(df_5m)):
    x_row = feat_df.iloc[bar_idx]
    prev_ret = log_returns[bar_idx - 1] if bar_idx > 0 else None

    e_sym, sub, v_long, v_short, n_voted = ensemble.step(
        x_row, actual_ret=prev_ret,
        regime_state=layer_a.dominant_state,
    )

    regime_mult = _REGIME_A_SCALE.get(layer_a.dominant_state, 1.0)
    signal = e_sym * regime_mult

    vote_ratio = v_long / n_voted if n_voted > 0 else 0.5
    if vote_ratio < VOTE_THRESH and vote_ratio > (1 - VOTE_THRESH):
        signal = 0.0

    positions[bar_idx] = signal
```

`e_sym` is the direction signal, `regime_mult` is the size adjustment, and `vote_ratio` is the safety check. The three multiply to form the final position.

8.9 Design Principles in Brief

Two configuration constants fix the system's time scale.

- **Refit cadence (REFIT_EVERY=156):** retrain the models roughly every two sessions. Too long, and we miss regime changes; too short, and the training window is too small for BIC to work properly. Two sessions is the compromise.
- **Minimum hold bars (MIN_HOLD_BARS=78):** hold a position for at least one session (= one day). The more often you trade, the more commissions and slippage eat into returns.

Trading costs pile up faster than you might think. If you assume 8 bp per one-way trade, turning over the book 1000 times a year already costs 16% per year. A single line for minimum hold time prevents that kind of cost explosion.

A Look at the Code —Minimal Two-Layer HMM

The compact two-layer listing is a structural sketch, not a complete training application. Store scaler, feature order, model, state meanings, and training cutoff together. Load them as one version; abstain from new signals if any component disagrees.

What this code does: uses Layer A's dominant regime to filter the Layer B sub-models, then weights the surviving models' $P(u)$ and converts it to a Kelly size.

```
import numpy as np

def combine_two_layers(global_alpha, submodels, t, kelly_half=0.5):
    dom_state = int(global_alpha[t].argmax())

    p_ups, weights = [], []
    for model in submodels:
        if dom_state not in model.active_regimes:
            continue
        p_ups.append(model.predict_up(t))
        weights.append(model.rolling_score)

    if not p_ups:
        return 0.0, 0.0

    w = np.array(weights) / np.sum(weights)
    p_up = float(np.dot(w, p_ups))

    edge = 2 * p_up - 1.0
    kelly = float(np.clip(max(edge, 0.0) * kelly_half, 0.10, 1.0))
    return p_up, kelly
```

What to check in the output: p_{up} should be between 0 and 1, and $kelly$ between 0.10 and 1.0. A p_{up} near 0.5 means there's almost no directional edge; 0.55 or above means there's an upward edge.

Exercises

1. Disable one of the sub-models and measure how the overall performance changes. This kind of analysis is called an **ablation study**. You remove components one at a time and measure the impact on overall performance.
2. Replace the Wasserstein W2 distance used in the voting weights with KL divergence. Compare the two and find out in what situations KL divergence becomes less stable than W2.
3. The first 100 bars after you start calling `HMMForwardPredictor.step(x)` are called the warm-up window. During this period the state probabilities haven't settled yet, so predictions can be shaky. Add logic that automatically reduces position size during the warm-up window.

8.10 Key Takeaways

In this chapter we combined eight sub-models into a single ensemble. Three points are the most important.

1. **Separate the two judgments that live on different time scales.** Layer A handles the long-term market mood (the macro regime), and Layer B handles short-term trade direction. Mixing the two roles into a single model destabilizes training.
2. **Use the regime gate to turn off signals that don't fit.** Disable mean-reversion models in trend regimes and momentum models in crisis regimes. This one filter alone improves signal quality a great deal.
3. **Use Wasserstein voting instead of a plain average.** Suppress outlier models and amplify the majority view. Even if one model emits a wrong signal, its effect doesn't spread to the whole.

Here is the summary table.

Layer	Class	Role
Layer A	<code>GlobalRegimeLayer</code>	Macro regime -> signal-size multiplier

Layer	Class	Role
Layer B	RenaissanceEnsemble5m	Eight sub-models -> direction signal <code>e_sym</code>
Combine	<code>e_sym × regime_mult</code>	Final position signal

One-line summary

```
signal = e_sym × regime_mult
```

Everything in this chapter is about how we build that one line.

Next chapter: In Chapter 9 we'll see how the `e_sym × regime_mult` signal we just built gets translated into an actual order size with **Half-Kelly position sizing**. Chapter 10 organizes the forward-style ensembling of multiple HMM signals, and from Chapter 11 we move from single-name regime strategies to another axis —statistical pair arbitrage.

Further Reading

- **Paper:** “Wasserstein Distances in the Analysis of Time Series and Dynamical Systems”—applying W2 distance to time-series analysis
- **Paper:** Breiman (2001), “Random Forests”—the foundational paper for the theory of ensemble methods
- **Code:** the `pyEMD` library —useful as a reference for hierarchical HMM implementations

Chapter 9. Half-Kelly Position Sizing

Part IV —Turning Signals into Positions (Chapters 9-10) The earlier parts focused on “how to read the state of the market.” Now we turn those judgments into actual position sizes and direction signals. Chapter 9 covers how large to size each trade, and Chapter 10 covers how to combine multiple HMM signals. On a first read, don’t try to memorize the formulas —hold on to one principle: “even a good signal breaks down if you bet on it too aggressively.”

Most retail investors barely think about position size. They tend to decide with round numbers: “I’ll put 10% into this,” “let me buy 100 shares.” Those numbers are closer to gut feel than to any calculation. Strategy, signals, entries, and exits get a lot of attention, but position sizing —deciding how much to invest in a single trade —gets pushed to the back of the line.

The quant approach is different. Position sizing is where actual returns are realized, and it is also where losses accumulate. The same signal sized at 2% is a tolerable wobble; sized at 30%,

it becomes an event that shakes the entire account. Signal quality and position size cannot be looked at separately.

The math of losses is not symmetric. Recovering from a 10% loss requires about an 11% gain. A 50% loss needs 100%, and a 75% loss needs 300% to get back to even. The deeper you go, the harder recovery becomes—not because of the market, but because of arithmetic. If you size positions larger than your edge supports, even a profitable strategy ends up losing money in the end.

The Kelly Criterion attacks this problem mathematically. It is a formula that, based on your win rate and your win/loss ratio, computes the theoretically optimal bet size. Half-Kelly is a conservative variant that bets only 50% of full Kelly. It reduces drawdowns while preserving most of the long-term compound growth. The system in this book goes one step further and uses a six-factor sizing model. It cuts full Kelly in half and then makes additional adjustments based on market state, volatility, macro conditions, and other inputs.

The most important takeaway in this chapter is not to read Kelly as “a formula for making more money.” Kelly is, if anything, a device for preventing overbetting. When the edge is small—say $p_{up}=0.53$ —the base position must be small no matter how complex the model is. A tiny error in the probability estimate can wipe the edge out entirely.

When you first apply this, it is better to treat the Kelly output as a diagnostic for “how small is my model’s edge, really?” rather than using it directly as an order size. The actual production position should be set only after passing through Half-Kelly, the maximum position cap, the per-symbol loss limit, and the portfolio-wide exposure cap. Kelly is a starting point for sizing, not the final answer.

In the full pipeline of this book, Kelly sizing is the step that **translates the model’s judgment into the size of money at risk**. HMM and the ensemble tell us direction and probability. Kelly calculates how large to bet for that probability edge.

```
Model output
p_up = 0.53
small edge toward the upside
    |
    v
Kelly / Half-Kelly
small edge -> only a small position is allowed
    |
```

v

Risk management

check maximum size, stop rules, and portfolio limits again

The important point is that $p_{up} = 0.53$ may sound like “53% confidence,” but it is actually a very small edge. It means that out of 100 similar situations, the model may be right 53 times and wrong 47 times. If you bet too large on that kind of edge, only a few consecutive losses can shake the account.

Prediction model's job:

find a slightly favorable direction.

Kelly's job:

if the edge is small, make the position small.

Risk management's job:

if it is still too large, shrink or block it again.

So Kelly is not a go-big button. It is a translator and a brake. The idea is not “the signal is good, so go big.” It is closer to “with this size of edge, allow only this much exposure.”

Flow of this chapter

We will first understand the Kelly formula itself, and then look one by one at the correction factors the system multiplies onto Kelly.

1. The intuition behind the Kelly formula and why we use Half-Kelly (9.1)
2. The structure of the full sizing function `calculate_kelly_size()` (9.2)
3. The role of each of the six correction factors (9.3 -9.7)
4. The simpler 5-minute-bar variant `fractional_kelly()` and a comparison of the two implementations (9.8 -9.9)
5. The design intent behind setting `KELLY_MIN` to 0.10 rather than 0 (9.10)

Learning goals

By the end of this chapter you should be able to:

- Explain the Kelly Criterion formula and why we use Half-Kelly
- Trace the flow by which `calculate_kelly_size()` decides a position size
- Tell apart what each of the six correction factors “shrinks or amplifies”
- Apply the core idea of the simple 5-minute-bar implementation `fractional_kelly()`

9.1 Kelly Criterion Basics

The Kelly formula came out of Bell Labs in 1956. J. L. Kelly used information theory to compute “the bet size that maximizes long-term capital growth when you repeat the same kind of bet.” Later the mathematician Edward Thorp applied the formula to gambling and to stocks, and several quant funds are known to use similar approaches.

Let’s nail down what Kelly is at its core. **It is not a formula for raising your short-term win rate.** It is a formula that decides how much to put on a single trade so that capital grows as much as possible over the long run, given that you repeat the same trade hundreds of times.

IMPORTANT

Full Kelly is usually too aggressive. For Kelly to be theoretically optimal, you must know your edge exactly—and in practice that is impossible. If you overestimate your edge, full Kelly leads to large losses. In this book we use Half-Kelly or less.

For a binary bet with only two outcomes—win or lose—let p be the probability of winning and b be the ratio of profit-when-winning to loss-when-losing. The optimal bet fraction is:

$$f^* = \frac{p \cdot b - (1 - p)}{b}$$

For stock trading, average wins and average losses are often similar, so setting $b \approx 1$ simplifies the equation:

$$f^* = 2p - 1 = 2 \times \text{edge}, \quad \text{edge} = p_{\text{up}} - 0.5$$

For example, if $p_{\text{up}} = 0.53$, then $\text{edge} = 0.03$, full Kelly is 6%, and Half-Kelly is 3%.

$p_{\text{up}} = 0.53$ does not mean “this trade is 53% certain.” It is a statistical estimate that, when similar situations repeat 100 times, prices go up on average 53 of those times. The remaining 47 times the call is wrong. That is why position size is not set by the size of your conviction. You size conservatively so that a small probabilistic edge can be repeated steadily over a long horizon.

Why we use Half-Kelly: in reality, p cannot be known exactly. Full Kelly is theoretically optimal, but when the estimate has error, results swing wildly. Half-Kelly ($f^*/2$) keeps a large fraction of the long-term return while sharply cutting the worst drawdowns. In Half-Kelly the bet fraction simply becomes edge itself. Every correction factor we look at in later sections is multiplied onto this edge.

9.2 The structure of `calculate_kelly_size()`

The sizing function in `v7.py` takes the base edge computed by Half-Kelly and multiplies six correction factors onto it, in order, to arrive at the final order size. Each factor is covered in the next sections. Here we only fix the structure of “in what order, and what is being multiplied.”

Below is the skeleton of the real function. The full implementation is in `run_rentec_joint_hmm_v7.py`.

```
KELLY_MIN, KELLY_MAX = 0.10, 1.0
THRESHOLD = 0.53          # pup > THRESHOLD -> long entry

def calculate_kelly_size(
    p_up, direction,
    vol_mult=1.0, macro_gate=1.0, novelty_gate=1.0,
    er=0.0, macro_bias=0.0, macro_class_amp=0.0,
) -> float:

    edge = (p_up - 0.5) if direction > 0 else ((1.0 - p_up) - 0.5)
```

```

if edge <= 0:
    return 0.0

f = edge

f *= vol_mult * macro_gate * novelty_gate

edge_max = max(THRESHOLD - 0.5, 1e-6)
return float(np.clip(f / edge_max, KELLY_MIN, KELLY_MAX))

```

There are two places to look at when reading this. First, `if edge <= 0: return 0.0`—if there is no edge, the function returns 0 immediately. Second, the final `np.clip`—after every correction is applied, the result is clipped to lie between `KELLY_MIN` and `KELLY_MAX`. Between those two lines, the six factors are being multiplied together.

The range and role of each factor:

Factor	Range	Role
edge	(0, 0.5]	Half-Kelly base from p_up
vol_mult	[0.5, 1.0]	Shrinks when the market is unstable
macro_gate	[0.55, 1.0]	Shrinks when the market state is uncertain
novelty_gate	[0.55, 1.0]	Shrinks when the market looks unfamiliar
er_mult	[0.5, 1.5]	Grows when expected return agrees with direction
macro_mult	[0.3, 2.0]	Macro response by asset class

Watch out for the combined effect: when all six factors hit low values at once, the position can become much smaller than you might expect. `KELLY_MIN=0.10` acts as a floor so the position does not collapse all the way to near-zero.

9.3 `vol_mult` —shrinking the position when volatility rises

When the market becomes unstable, position sizes should be reduced automatically. `hmm_vol_mult()` handles this. The key idea is that **instead of using the standard deviation of past prices, we use the per-regime variance that the HMM has already estimated**.

The point of this section is why the two approaches react at different speeds.

- **Rolling-window volatility:** averages the price movement over the last 20-60 bars. Even when the market crashes suddenly, older data dilutes the change, so the reaction does not appear until 20-60 bars later.
- **HMM regime variance:** uses the volatility that the HMM pre-estimated for each regime. The moment the regime probabilities `alpha_t` shift toward a high-volatility regime, the reaction is immediate.

When `alpha_t` shifts toward a high-volatility regime, the calculation proceeds in this order:

1. Regime-weighted volatility (`sigma_hmm`) becomes larger than average volatility.
2. The volatility ratio exceeds 1.0.
3. `vol_mult` drops below 1.0.
4. Position size shrinks automatically.

An illustrative volatility scale for ratios $1/2/3$ is `1.0/0.75/0.50` using `clip(1-0.25*(ratio-1),0.25,1)`. It is interpolation, not an optimized production rule. Numerator and denominator need identical units. Do not increase exposure with zero reference volatility or insufficient data.

9.4 macro_gate —shrinking when the market state is unclear

The HMM outputs the probability that the current market belongs to each regime as the array `alpha_t`. With four regimes, this might look like `alpha_t = [0.9, 0.05, 0.03, 0.02]`. When the mass is concentrated on one regime, it means “we are confident about which regime we are in.” When the mass is spread evenly, it means “we have no idea which regime this is.”

`macro_gate` turns that certainty into a number between 0.55 and 1.0, deciding how much to trust the signal. If the regime is clear, the signal goes through as-is; if regimes are mixed, the signal is shrunk. Certainty is measured with Shannon entropy from information theory.

With four regimes, for example:

- Probability is 100% on one regime -> gate = 1.0 (passes through)
- Probability is 25% on each of four regimes -> gate = 0.55 (clipped to the minimum)
- In the middle of a transition (50/30/10/10) -> gate $\approx$ 0.59 (only slightly open)

Setting `MACRO_GATE_MIN = 0.55` keeps 55% of the signal even when conditions are completely uncertain. The reason for not using 0 (full block) is that liquidating every position the moment a regime shifts causes slippage and costs to balloon.

9.5 novelty_gate —shrinking when the market looks unfamiliar

The HMM classifies the current market by the patterns it has been trained on. But when something it has never seen before arrives —like the days right after a tariff shock —the observation does not fit any regime well, and the overall fit score drops. `hmm_novelty_gate()` detects this and automatically reduces the position.

The core idea is to look at “how unfamiliar is the current observation compared to usual?” It is measured not in absolute terms but as a **percentile rank within the distribution of the last 60 sessions**. When something is much more unfamiliar than usual, the gate activates and shrinks the position to as low as 0.55.

Extreme-market check —March 2020 COVID: over the 8 trading days from March 16-23, 2020, the average value of `novelty_gate` was 0.61, and positions were shrunk by about 40%. While the S&P 500 fell -24%, the strategy's drawdown was held to -11%. Thanks to `NOVELTY_GATE_MIN=0.55`, the strategy did not leave the market entirely, so it could also catch the rebound that started after March 24.

This gate does something different from the Kalman filter:

- **Kalman filter:** removes bar-level price noise for a single instrument.
- **HMM novelty gate:** measures how well the 13-dimensional cross-asset observation built in Chapter 7 matches the learned regimes.

More recent data may improve a novel-pattern boundary, but recovery after 60 sessions is not guaranteed. One policy abstains beyond the training distance distribution's 99th percentile and requires separate validation after refit. Relaxing the threshold immediately in response to OOS losses trains on the evaluation interval.

9.6 ER Amplifier —growing the position when signals agree

The three factors so far (`vol_mult`, `macro_gate`, `novelty_gate`) all work in the direction of **shrinking** the position. The ER amplifier works the other way: it is the factor that **grows the position when the signal is strong and points in the right direction**.

ER (Expected Return) is the expected return for the next session as estimated by the HMM. It is computed as a weighted average of per-regime mean returns, using the current regime probabilities as weights.

The rule is simple:

- Expected return (`er`) has the same sign as position direction (`direction`) -> amplify up to 1.5×
- Expected return has the opposite sign of position direction -> cut to 0.5×

The second case is the important one. If `p_up` says go long but ER is negative, the two signals are in conflict. In that case the position is cut in half. It works as a built-in safety check, with one signal restraining the other inside the model.

9.7 Adjusting macro bias by asset class

The 2-Layer Macro HMM produces a `macro_bias` value between -1 and +1. -1 means full Risk-Off (risk avoidance), +1 means full Risk-On (risk appetite). This value is not applied identically to every asset. Different asset classes react in different ways, so it is scaled per asset by multiplying it with a coefficient called `macro_class_amp`.

Class	<code>macro_class_amp</code>	Effect in a Risk-Off phase
equity (stocks)	+0.5	Strongly shrinks equity longs
safe_haven (GLD, TLT)	-0.3	Grows safe-haven longs (sign flip)
commodity	+0.3	Gently shrinks commodity exposure
credit (HY)	+0.3	Shrinks high-yield exposure
fx	+0.2	Weak FX sensitivity
vix (VIXY)	-0.6	Strongly grows VIX longs (sign flip)
em (emerging markets)	+0.4	Shrinks EM equity exposure

Assets with positive signs move in the same direction as the macro bias, while assets with negative signs (safe_haven, vix) move the opposite way.

Let's check with examples.

Example 1 —equity long in a Risk-Off phase -> position is shrunk by about 25% **Example 2 —VIX long in a Risk-Off phase** -> position is grown by about 30%

The principle is simple. In a Risk-Off phase, equity longs shrink automatically and VIXY longs grow automatically. You do not have to write separate hedging logic —when the macro phase changes, the portfolio naturally moves into a defensive posture. The exact formula and clip ranges are in the body of `calculate_kelly_size()`.

9.8 fractional_kelly() —the simple version for 5-minute bars

The 5-minute-bar Ensemble system does not need the six gates. Because it deals with a single instrument and short-term signals, a simpler quarter-Kelly is enough. `fractional_kelly()` skips the discrete-bet estimates of p and b and uses the mean (μ) and variance (σ^2) of the continuous signal directly.

```
# hmm_v5_core.py (excerpt)
def fractional_kelly(mu, sigma2, kelly_frac=0.25, max_size=1.0) ->
float:
    """f* = kelly_frac × |μ| / σ²
    Reduce size for higher uncertainty; increase it for stronger
    signals.
    """
    if sigma2 <= 1e-12 or np.isnan(mu) or np.isnan(sigma2):
        return float(max_size)
    k = kelly_frac * abs(mu) / (sigma2 + 1e-12)
    return float(np.clip(k, 0.0, max_size))
```

The key point is that $|\mu|$ (signal magnitude) is in the numerator and σ^2 (uncertainty) is in the denominator. For the same signal, doubling the volatility shrinks the position to a quarter.

In the backtest, the sign of the signal sets the direction and the output of `fractional_kelly()` sets the size.

9.9 Comparing the two Kelly implementations

Item	<code>calculate_kelly_size()</code> (v7.py)	<code>fractional_kelly()</code> (v5_core.py)
System	Daily / session cross-asset	5-minute bars, single symbol
Base formula	$\text{edge} = p_{\text{up}} - 0.5$	$f^* = \text{kelly_frac} \times \mu / \sigma^2$

Item	<code>calculate_kelly_size()</code> (v7.py)	<code>fractional_kelly()</code> (v5_core.py)
Kelly fraction	Half (0.5×)	Quarter (0.25×)
Volatility adjustment	HMM regime variance	78-bar rolling standard deviation
Extra gates	<code>macro_gate</code> , <code>novelty_gate</code> , ER amp	None (simple)
Clipping	[0.10, 1.0]	[0.0, 1.0]

`calculate_kelly_size()` reflects flows between many assets, the overall market mood, and the character of each asset class. With more variables to consider, its structure is more complex. `fractional_kelly()` is built for single-symbol short-term trades, and its structure is far simpler. In situations where there is no need to handle many conditions at once, the simpler implementation often works more reliably.

9.10 Why KELLY_MIN = 0.10?

Even when confidence is low, we keep a minimum position. If you empty the position completely, you lose the ability to re-enter cleanly when a good opportunity shows up.

```
KELLY_MIN = 0.10
KELLY_MAX = 1.0
```

`KELLY_MIN = 0.10` means “if there is a signal, bet at least 10%.” Even when the Kelly edge is nearly 0, the function returns 0.1 instead of 0.

Why 10% in particular? This system has several correction factors that shrink the position. The floor was set at 10% so that even when all of those factors hit their minimums at the same time, the position does not become zero. If you lowered it to 0, the position would disappear completely during bad stretches, and you would miss entry opportunities in the rebound right after a regime change.

That said, when `edge <= 0` (`p_up <= 0.5`), the function returns 0 before `KELLY_MIN` is even applied. “No edge to begin with” and “there is an edge but corrections shrunk it” are handled differently.

Walking through scenarios

Let’s run the function in our head. With the four scenarios below, the trend looks like this.

Scenario	p_up	vol_mult	macro_gate	novelty_gate	Result
Strong bull regime	0.72	0.9	0.95	1.0	Large position
Neutral regime	0.56	0.7	0.70	0.9	Around <code>KELLY_MIN</code>
High-volatility regime	0.63	0.5	0.60	0.8	Small position
Outlier detected	0.68	0.8	0.85	0.2	Pulled down to <code>KELLY_MIN</code>

The last row shows what `novelty_gate` does. Even with `p_up` at 0.68—a fairly strong signal—if the observation is unfamiliar across every regime so that `novelty_gate = 0.2`, the position is pulled down to the floor.

Exercises

1. With $b = 1, p = 0.55$, compute the position size for Full Kelly, Half Kelly, and `fractional_kelly(factor=0.25)`. Think about which of the three will swing the least.

2. If you lower the threshold of `macro_gate`, predict which kinds of trades would be blocked more and which would pass through more.
3. If you change `KELLY_MIN = 0.10` to 0, sketch the scenarios in which the strategy would close its position completely.

Summary

The four things to remember from this chapter:

- **Position sizing matters as much as the signal.** The same strategy produces completely different results depending on the bet size.
- **Full Kelly is dangerous.** In a real world with estimation error, Half-Kelly survives longer.
- **In Half-Kelly, the bet fraction is just edge itself.** Every correction factor multiplies onto this edge.
- **Most of the six correction factors shrink the position.** The more unstable the market, the more confused the regime, and the more unfamiliar the observation, the smaller the position becomes automatically. `KELLY_MIN=0.10` is the floor that keeps the position from collapsing to zero even when every gate is fully closed.

In one line, the Half-Kelly position pipeline is:

```
p_up -> edge = p_up - 0.5 -> multiply by correction factors -> clip to [0.10, 1.0]
```

Next chapter (Chapter 10): We cover the HMM Forward Ensemble. We analyze the structure of `run_hmm_forward_ensemble_v2.py` and `v3.py`, and explain a pipeline that compares multiple sub-models in a walk-forward fashion.

Further reading

- **Paper:** Kelly, J. L. (1956), “A New Interpretation of Information Rate”—the original Kelly paper
- **Books:** Thorp, Beat the Dealer, The Mathematics of Gambling —practical Kelly applications
- **Book:** Lopez de Prado, Machine Learning for Asset Managers, Chapter 6 —linking position sizing to signal strength

Check your understanding

“Compare Kelly for win probabilities 0.5/0.6/0.7 with payoff ratio one. Also show half-Kelly and a 10% position cap.”

Chapter 10. HMM Forward Ensemble —V1 -> V3 Evolution

The first version of any complex system usually falls short. Quant systems are no different. What matters is figuring out where and why it failed, and deciding what to fix in the next version.

The V1 ensemble had a working basic structure but was too weak to trust with real money. V2 added Wasserstein-distance voting, and V3 added signals across multiple time horizons. The point is not the numbers themselves but what failure each version was trying to reduce.

This chapter is a record of that iteration cycle, version by version. Instead of just looking at the result numbers, pay attention to which failure hypothesis led to each structural change. AI helped speed up each cycle. When asked where the predictions go most wrong, the AI

pointed to regime (market state) transition periods, where the submodels disagreed most. That feedback became the starting point for the V3 changes.

Why an Ensemble

A single HMM only sees one side of the market. A model that watches trend gets weak in sideways markets, and a model that watches volatility gives blurry signals during strong trends. Since we cannot know in advance which kind of market is coming, it is safer to build several models that look at different information and combine their opinions.

Think of it like this: instead of letting one expert make every call, you ask several specialists from different fields and average their opinions. Most days they agree, but at decisive moments —like a regime change —one or two of them send a different signal. That is when a majority vote or weighted average earns its keep.

In the pipeline, this chapter sits in the signal-generation step. Chapter 10 builds the expected-return signal, Chapter 9 explains how that signal becomes position size, and Chapter 11 starts the separate track of finding tradable stock pairs. In other words, Chapter 10 is the step that produces a single number: “which direction will the price move tomorrow?”

Chapter 9 recap: Kelly Sizing takes the expected return signal $E[r]$ from the ensemble and decides the position size. This chapter looks at how that signal is built.

Across the whole book, the ensemble is a meeting room that turns several model opinions into one number. One HMM cannot read every market condition well. Some models are sensitive to trends, some to volatility changes, and some to sector flows. The ensemble collects those opinions and produces the final expected return $E[r]$.

```
Model A: tomorrow may be +0.15%
Model B: tomorrow may be -0.05%
Model C: tomorrow may be +0.20%
    |
    v
Ensemble
Give a little more weight to models that worked well recently
    |
    v
```



```

Final signal  $E[r]$ 
Example: +0.10%
      |
      v
Kelly / Risk
Decide position size and whether an order is allowed

```

A beginner should not read “ensemble” as “more models automatically makes things better.” More models also mean more opinions and more noise. The important question is how to combine models that fail in different ways.

```

Good ensemble:
Models that read different information
become more trustworthy when they point the same way.

Bad ensemble:
Many copies of similar models
count the same mistake several times.

```

How This Chapter Is Organized

We start with the big picture and step inward.

- First, see the differences between V1/V2/V3 at a glance (10.1)
- The shared core all three versions use —building $E[r]$ from Forward α (10.2)
- What kind of information each version added (10.3)
- Why Optuna threshold re-optimization is fast (10.4)
- Performance comparison and adaptive thresholds (10.5, 10.6)

Learning Goals

After this chapter you should be able to:

- Explain the key differences between V1/V2/V3 and point out the problem each version solved.
- Sketch how to use Forward α to build the next-day expected return $E[r_{t+1}]$.
- Explain in one sentence why Optuna threshold re-optimization is fast.
- Understand the roles of dynamic weights and multi-timeframe signals in the ensemble.

What to Focus On During Your First Read

This chapter throws around hard words: submodel, Forward α , transition matrix, Optuna, dynamic weights. On your first pass, just follow this flow:

1. Several models each say “tomorrow’s return looks like it leans this way.”
2. The opinion of models that did well recently is weighted a little more.
3. The end result is a single number called $E[r]$.
4. Later chapters convert that number into a position size and actual trading conditions.

You can leave the formulas and Softmax details for a second read. The goal of the first read is to understand that **an ensemble is a device that combines many models’ opinions into one number**.

10.1 Architecture Comparison —V1, V2, V3

Item	V1	V2	V3
Tickers	1 (SPY)	5 (UNIVERSE)	10
Submodels	5 (M1~M5)	7 (M1~M7)	15 (M1~M9, multi-TF)
Weights	fixed (equal)	fixed (equal)	dynamic (rolling-performance softmax)
Non-HMM models	none	none	Kalman (M8) + MeanRev (M9)

Item	V1	V2	V3
Optuna	none	yes (re-thresholding)	yes
Parallelism	none	none	ThreadPoolExecutor (4 threads)

V1 → V2 → V3 evolved along three axes: more types of submodels, weights that move with the situation, and more tickers.

The key change in V2 is the introduction of Optuna, a library that automatically searches for the model's best threshold. V3 then adds a multi-timeframe approach that splits the signal across several time horizons.

There is a reason for that order. If you add more tickers first, you also add more noise (signals that mean nothing). It is safer to confirm that the model structure and thresholds behave reliably before scaling up the universe.

10.2 The V1/V2 Shared Core —Computing $E[r]$ with Forward α

V1 and V2 share the same core calculation. Even as versions change, this part stays.

In one sentence: **use the data seen so far to update the current regime probabilities, then push those probabilities one step forward to get tomorrow's expected return.**

A regime is the market's current "mood." Categories like "uptrend," "sideways," and "sharp drop" are regimes. The probability vector that gets updated step by step is called **Forward α** . For example, if there are three regimes and today's Forward α is $[0.7, 0.2, 0.1]$, it reads as "today looks 70% likely to be an uptrend, 20% sideways, 10% sharp drop."

Two more ingredients are needed.

- **Transition matrix A:** a table of the probabilities that a regime moves to another regime in the next step. For example, $A[\text{Bull}, \text{Bear}] = 0.05$ means "if today is Bull, there is a

5% chance tomorrow becomes Bear.” This table is automatically estimated from historical data during HMM training using the Baum-Welch algorithm (see Chapter 8 for details).

- **Per-regime average return μ_{ret} :** the average of the returns observed in each regime during training. Once training ends, it is frozen inside `m.means_` and is never updated during the backtest (the reason is in 10.2.1).

A small example makes it easier. Say today’s regime probabilities are [Bull 70%, Neutral 20%, Bear 10%]. Apply the transition matrix and tomorrow’s probabilities shift slightly, say to [Bull 65%, Neutral 25%, Bear 10%]. If each regime’s average return is [+0.2%, 0.0%, -0.3%], tomorrow’s expected return is roughly $0.65 \times 0.2\% + 0.25 \times 0.0\% - 0.10 \times 0.3\% = +0.10\%$. That +0.10% is $E[r]$.

The core boils down to two lines.

- **Step 4 (predict):** `next_proba =  $\alpha_t \times A$` —push today’s regime probabilities one step into the future.
- **Step 5 (E[return]):** `$E[r_{t+1}] = \text{next_proba} \cdot \mu_{\text{ret}}$` —multiply that probability by each regime’s average return and sum.

These two lines are the heart of every version. The other steps (initialization, observation likelihood updates, ensemble averaging, turning thresholds into $\pm 1/0$ signals) only exist to support these two. The full pseudocode (Steps 1-7) is in Appendix A.

Sample output:

```
2024-01-03 -> e_ret = 0.0034 (Bull regime 0.87)
2024-01-04 -> e_ret = -0.0012 (transition phase)
2024-01-05 -> e_ret = 0.0021 (back to Bull)
```

Every time a new bar (one day of price data) arrives, run those two lines to get one number —tomorrow’s `e_ret`.

10.2.1 Avoiding the Causality Trap

When computing per-regime expected returns, there is a common trap: **future data leaking into past calculations**. This is called a causality violation. You cannot see the future in real trading, so if this error sneaks in, the backtest looks far better than it really is.

The rule to remember is one line.

- **Use:** the frozen `m.means_` after training (per-regime averages locked in at training time).
- **Avoid:** `m.predict()` —Viterbi smoothing peeks at future data when deciding states.

The single line in real code where this rule applies looks like this.

```
srm = sc.inverse_transform(m.means_)[:, idx] # causal
# m.predict(Xs) # look-ahead bias
```

If something breaks: if the backtest comes out unrealistically high (say, returns north of 3×), there is a good chance future data is leaking into this calculation.

10.3 What Each Version Added

Separate model roles: returns estimate direction, volatility estimates risk magnitude, and mean reversion measures deviation from past equilibrium. Convert outputs to a common quantity, such as expected return over one holding period. Averaging standardized scores directly with returns makes numeric scale determine weight.

Added in V2 —Sector & Order Flow

- **M6_Sector:** watches sector rotation, the phenomenon where money flows into different industries as the economic cycle moves. It bundles the excess return of the ticker over the broad market, the excess return over its own sector, and the general strength of sector ETFs, then learns three regimes (sector strong / weak / neutral).
- **M7_OrderFlow:** looks at order flow —whether buyers or sellers drove the trades. Even without the order book, you can estimate this from daily OHLCV (open, high, low, close, volume). The key idea is “the higher the close sits relative to the day’s low, the stronger the buying pressure was.”

Added in V3 —Multi-Timeframe & Non-HMM

V3 added two things.

Multi-timeframe. Some of the existing submodels (M1, M2, M3, M5) are duplicated across three time horizons: short term (1d), medium term (5d), and long term (21d). Short-term regimes catch signals that shift daily; long-term regimes catch trends that last a month or more. When two timeframes point in the same direction, the signal is more trustworthy.

This is different from just copying the same model many times. The 1d and 21d horizons see different kinds of information —quick wiggles versus slow-moving trends. The less these information sources overlap, the more the dynamic weighting (below) can shift weight toward whichever horizon is currently more accurate, creating a real synergy.

Two non-HMM models. V3 brings in statistical tools other than HMM.

- **M8_Kalman:** produces a smooth “trend estimate” that strips out the noise mixed into price, then uses that estimate as next-day expected return. (The tool is the Kalman filter.)
- **M9_MeanRev:** looks at the tendency of price to return toward its average after drifting in one direction. It only emits a counter-trend signal when the price is far from the mean and that pull-to-mean property is strong.

Read the multi-timeframe like this:

Situation	Interpretation
1d, 5d, 21d all up	short and long term agree —signal is trustworthy
1d up, 21d down	could be a short-term bounce —lower the weight
1d down, 21d up	could be a temporary pullback inside a long-term uptrend

The Kalman filter derivation, detailed definitions of statistical tools like variance ratio and z-score, and tuning hyperparameters like the noise variance are all in the appendix.

V3 Dynamic Weights —More Weight on Submodels That Did Well

Through V2, every submodel had the same weight. V3 tracks each submodel’s performance daily and **gives more weight to the ones that did well recently**.

The flow fits in one row: input -> calculation -> output.

Input	Calculation	Output
Each submodel's recent daily (predicted direction × actual return)	Rolling performance, normalized with Softmax	A weight vector w_k that sums to 1

A score-to-weight example is $w_i = \exp((s_i - \max(s))/T) / \sum_j \exp((s_j - \max(s))/T)$, with $T > 0$. Higher T approaches uniformity; lower T concentrates weights. Score only past forecasts whose outcomes are known. Exclude insufficient history or use a predefined prior. Three equal scores give weights of 1/3.

The next-step ensemble signal is then $E[r]_{\text{ens}} = \sum_k w_k \cdot E[r]_k$.

For instance, if yesterday's weights were $[0.30, 0.25, 0.20, 0.15, 0.10]$ and the first model kept being right, today they might become $[0.40, 0.20, 0.20, 0.10, 0.10]$. The exact formula matters less than the fact that **the louder voice goes to the model that has been working**.

10.4 V2 Key Idea —Optuna Threshold Re-Optimization

V2's most important change is one thing: **instead of rerunning Walk-Forward from scratch, only re-tune the thresholds**.

A threshold here is the cutoff that filters the signal: “buy if expected return exceeds this level, sell if it drops below this level.”

Why is this fast? Re-training and re-running HMMs every time is slow. But if you already have the $E[\text{return}]$ time series saved, sweeping over different thresholds is far quicker. The model stays as is—you just slide the bar.

The rough cost difference is:

- **Walk-Forward rerun:** repeatedly train the model across all the data → very slow
- **Re-thresholding:** just apply different thresholds to the saved $E[r]$ → much faster

One caution: if you let Optuna search the threshold using the entire OOS (Out-of-Sample) data, the optimizer itself has peeked at the future. So Optuna only uses the first 40% of OOS for threshold search and leaves the remaining 60% untouched as a real validation window. For this trick to work, you must save each day's `e_ret` during the walk-forward step.

10.5 V2/V3 Performance Comparison Summary

Here is how each version differs in role. OOS (Out-of-Sample) refers to data that was not used to train the model.

Version	Submodels	Universe	Notes
V1	5 HMMs	SPY only	basic structure validation
V2	7 HMMs	5 tickers	confirms the Optuna re-threshold effect
V3	15 (HMM + mixed)	10 tickers	confirms the dynamic performance-based weighting effect

For real money, the complex V3 is not always the answer. If you want a model whose decisions are easy to trace or whose behavior is stable, the simpler V2 may be the better pick.

10.6 Adaptive Threshold — $\sigma \times \text{scale}$

The threshold is not fixed. It moves with the recent $E[r]$ distribution. The core idea is one line: **make the threshold proportional to the standard deviation of recent $E[r]$.**

The behavior follows:

- **When the market is quiet:** standard deviation is small, so the threshold is low. Small signals are enough to trigger entries.
- **When the market is turbulent:** standard deviation is large, so the threshold is high. Only strong signals trigger entries.

The proportionality constant (`thresh_scale`) is not picked by a human —Optuna finds it automatically. Because of this self-calibrating structure, trading frequency does not swing wildly when the market regime changes. The actual threshold calculation and the one-line signal generation are covered in the “Try It Yourself” below.

Try It Yourself —Threshold Optimization with Optuna (Optional)

You can skip this section on the first read. After you have the flow in your head, come back on a second read and step through it by hand.

The synthetic threshold search starts from precomputed `e_ret`. Predictions 0.01/0/-0.01 with weights 0.5/0.3/0.2 give 0.003. A 0.002 threshold makes a candidate, still subject to costs and risk limits. This is not the full submodel-training pipeline.

The code below searches `thresh_scale` over 0.2-2.0 and finds the value that maximizes OOS performance. The two key lines: set the threshold to `e_ret`’s standard deviation $\times$ scale, then turn that threshold into a $\pm 1/0$ signal.

```
import numpy as np
import optuna

def optimize_threshold(e_ret, actual_ret, n_trials=50):
    def objective(trial):
        scale = trial.suggest_float("scale", 0.2, 2.0)
        threshold = float(e_ret.std()) * scale
        signals = np.where(e_ret > threshold, 1,
                           np.where(e_ret < -threshold, -1, 0))
        pnl = signals * actual_ret
        if (signals != 0).sum() < 5:
```

```

        return -999.0
    return evaluate_objective(pnl)

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=n_trials)
return study.best_params["scale"], study.best_value

```

What to check in the output: `thresh_scale` usually lands in the middle of the range, and on synthetic data the performance oscillates around zero. On real data, what matters is not the absolute number but whether the gap between in-sample and OOS performance is too wide.

If something breaks: if `-999.0` keeps appearing, the threshold is so large that almost no signals fire. Check the distribution of `e_ret` first (`np.std`, histogram).

Exercises

1. Suppose two submodels M1 and M2 emit opposite signals on a given day. For example, M1's $E[r] = +0.003$, M2's $E[r] = -0.002$, and both have the same weight ($w = 0.5$). What is the ensemble sum $E[r]_{\text{ens}}$, and with a threshold of ± 0.001 , what is the final signal (+1 / -1 / 0)?
2. To add a new submodel M9_Earnings (event pattern around the three days before and after an earnings release), what features would you need and what data sources would you use? Write it down.
3. (Thought experiment.) If you had to switch off two of the eight submodels, which kinds of submodels would hurt performance the most? Hint: a submodel that sees information none of the others see is costly to lose, while one that overlaps heavily with another can be removed with less impact.

Wrap-Up

This chapter looked at **the ensemble approach that combines several signals into one**. Signals that a single HMM cannot easily capture —sector rotation, order flow —are produced by separate submodels and then merged into a single prediction signal.

Version	Key contribution	Remaining issue
V1	Forward-only α pipeline producing $E[r]$	SPY only, fixed weights
V2	M6_Sector, M7_OrderFlow + Optuna re-thresholding	no dynamic weights
V3	Multi-TF + M8_Kalman + M9_MeanRev + performance-based softmax	—

Things to remember from this chapter

1. **Keep causality:** use only `m.means_` (locked in after training) for per-regime average returns. `m.predict()` peeks at the future and ruins the backtest.
2. **One-step-ahead prediction:** (current regime probabilities $\times$ transition matrix) $\cdot$ per-regime average returns —this one line is the core of every version.
3. **Do not fix the threshold:** make it proportional to the standard deviation of recent $E[r]$ and let Optuna tune that ratio (`scale`) automatically. That is how trading frequency stays stable when market conditions change.

Next chapter (Chapter 11): how to pick pairs for statistical arbitrage (Stat Arb). You will use the Engle-Granger cointegration test to find pairs of tickers that move together, and a Kalman filter to dynamically compute the spread and hedge ratio between them.

Further Reading

- **Paper:** Sharpe, W. F. (1994), “The Sharpe Ratio,” *Journal of Portfolio Management* —meaning and limits of Sharpe-style performance measurement.

- **Paper:** “Stacked Generalization” (Wolpert, 1992) —the theoretical foundation of ensemble learning.
- **Book:** Grinold & Kahn, Active Portfolio Management —how to fold ensemble alphas into a portfolio.

Chapter 11. Statistical Arbitrage —Pair Selection

Part V —Pair Trading Basics (Chapters 11-12) In this part, we stop asking “will this stock go up?” and start asking “will the gap between these two stocks narrow again?” Chapter 11 picks the pairs worth trading. Chapter 12 turns those pairs into entry and exit signals. On a first read, you don’t need every detail of the cointegration test —just remember that correlation alone isn’t enough.

A trader who looks at charts usually asks one question: “Will this stock go up or down?” If up, buy. If down, sell. That’s how most retail investors trade.

Statistical arbitrage asks a different question. “Is there a stable relationship between these two assets?” And, “Has that relationship temporarily stretched, with a good chance of snapping

back?”The focus is on the relationship, not on the direction. Instead of trying to guess whether Coca-Cola (KO) goes up or Pepsi (PEP) goes down, you bet that if the gap between the two prices is wider than usual, it will narrow again.

This is called a **market-neutral strategy**. Whether the broader market rises or falls doesn’t matter, because you are only betting on the relationship between the two assets converging. The main way you lose is when that relationship itself breaks. As a result, the edge you are chasing is also easier to measure than a directional bet.

There is a common mistake retail investors make when they try pairs trading: “Coke and Pepsi move together, so they must be a good pair.”That sounds intuitive, but it is not a real check. Two stocks that appear to rise together might simply be following the broader market. **Correlation** measures how closely two assets move in the same direction. **Cointegration** means there is a long-term stable relationship between them. They are different concepts, and what we need is the second one.

To pick pairs in a disciplined way, you need three things: run a cointegration test on many candidate pairs, filter out the ones that pass by chance, and finally apply economic reasoning —“why should these two companies move together?”A low p-value alone does not make a good pair. If you test 1,000 pairs, about 50 will pass purely by luck even when no relationship exists (the multiple testing problem).

In the machine-learning flow of this book, Chapter 11 is the chapter that **creates trading candidates**. HMM can read the market state, and RL can decide position size, but first the system needs something to trade. Pair selection builds that candidate list.

```

Full stock universe
hundreds of candidates
    |
    v
Sector filter
keep stocks with an economic reason to move together
    |
    v
Cointegration test
check whether the spread tends to return to its mean
    |
    v
Half-life / quality filters
  
```

```

check whether the relationship reverts fast enough to trade
    |
    v
candidate trading pairs
Chapter 12 turns them into entry and exit signals

```

For a beginner, pair selection is not “finding two stocks that look good together.” More precisely, it is **throwing away candidates that can become dangerous when the relationship breaks**. A good pair is not just two stocks that move together; it is two stocks whose stretched gap has a statistical and economic reason to narrow again.

```

Correlation:
Do the two assets move in the same direction?

Cointegration:
When the gap between them stretches too far, does it tend to return?

Pairs trading needs cointegration.

```

What this chapter covers

We'll build the intuition for pair trading first, then drop into the concrete rules we use to choose pairs.

- Why pick pairs within the same sector?
- The pair-selection pipeline
- Calculating the halflife
- Using the Kalman filter for a dynamic hedge ratio
- Pair-quality checks

Learning goals

By the end of this chapter you will be able to:

- Pick stock pairs within a sector and filter them with a cointegration test.
- Explain what the **half-life** of a spread is and why it works as a quality measure.
- Explain the role of the **Kalman filter** in pair selection (a time-varying hedge ratio instead of a fixed one).
- Read the meaning of the key `_RLConfig` parameters: `z_entry`, `corr_min`, `hl_max`.

11.1 Why pick pairs within the same sector?

Statistical arbitrage (Stat Arb) assumes the two stocks are driven by similar outside forces — interest rates, commodity prices, industry demand, or the same macro cycle. Two companies in the same industry tend to react in similar ways to interest-rate moves or commodity prices.

If you pair every stock with every other stock across the whole market, you end up with thousands of pairs. Repeat a statistical test thousands of times, and roughly 5% will cross $p < 0.05$ by chance, even when no real relationship exists. This is the **multiple testing problem**.

Restricting candidates to a single sector gives you two benefits.

- Fewer tests means fewer false positives.
- Companies in the same sector are economically connected, so you can actually answer “why do they move together?”

For example:

- AAPL-MSFT inside XLK (the tech sector): shared semiconductor supply chain.
- JPM-BAC inside XLF (financials): similar response to interest-rate moves.

What is cointegration? It’s a relationship in which two time series each wander on their own, but the gap between them stays inside a stable range. When the gap stretches, it tends to come back —and that’s the moment when there’s a trade to take.

In the implementation, we don’t compare every stock against every other stock. We split the universe into 11 sectors with a predefined `SECTOR_SYMBOLS` dictionary and only form pairs within each sector. The code lives in `app/trading/strategy/signals/mean_reversion_signal.py`.

11.2 The pair-selection pipeline

We filter hundreds of candidate pairs step by step. Each step removes pairs for a different reason. The correlation filter drops pairs whose moves look too different. The cointegration test checks for a long-term link. The halflife filter checks that the spread returns to its mean fast enough to be tradable.

Step	Stage	Criterion / Output
1	Full universe	S&P 500
2	Group by sector	SECTOR_SYMBOLS
3	Load price data	3 years of daily bars
4	Correlation filter	corr $\geq$ 0.4
5	Cointegration test	p < 0.05
6	OLS β estimate	Static hedge ratio
7	Halflife calculation	hl $\leq$ 80 days
8	Final pair list	(sym_a, sym_b, corr, pvalue, beta, halflife)

Historical context D.E. Shaw, PDT Partners, and Renaissance ran pair trading at scale starting in the 1990s. The 1998 LTCM blow-up happened when many spreads widened at once, which taught the industry that staying within a single sector helps diversify away from that kind of simultaneous loss.

WARNING

Don't rely on a single p-value. The cointegration test gives different p-values depending on the regression direction (A $\rightarrow$ B vs. B $\rightarrow$ A). In practice you should also look at supporting measures such as the halflife and the stationarity of the residuals. We cover stricter cross-checks (the Johansen test and friends) in Chapter 19.

You don't need a statistics background. The tests in this chapter (cointegration, ADF) all answer the same question —“Does the gap between these two stocks eventually return to its average?” If $p < 0.05$, it does. If $p > 0.05$, it doesn't. Even without the formulas, you can run the code, check the p-value, and use the result.

Cointegration test —judging a single pair

`statsmodels` ships a `coint()` function that does it in one line. Under the hood it (1) runs an OLS regression of one price on the other, then (2) checks whether the residuals (the spread) revert to a mean using an ADF test. The only output we use is the p-value, which is the second return value.

```
from statsmodels.tsa.stattools import coint

_, pvalue, _ = coint(price_a.values, price_b.values)

if pvalue >= self.coint_pvalue:    # default = 0.05
    continue
```

A p-value below 0.05 passes; anything else is dropped.

Full pipeline —testing every pair, sector by sector

Start pairs from economically related assets, align training dates, then check missingness, sample count, cointegration, and half-life. Adjust for multiple testing when screening many pairs. The sketch shows selection order; selected pairs still need OOS evaluation.

```
import numpy as np
import statsmodels.api as sm
from statsmodels.stats.multitest import multipletests
from statsmodels.tsa.stattools import coint

def find_cointegrated_pairs(self, symbols) -> list:
    tested = []
    for sector, sector_symbols in SECTOR_SYMBOLS.items():
        prices = self._load_prices(sector_symbols, period="3y")
```

```

corr_matrix = prices.pct_change().dropna().corr()

for i, sym_a in enumerate(corr_matrix.columns):
    for sym_b in corr_matrix.columns[i+1:]:

        corr = corr_matrix.loc[sym_a, sym_b]
        if corr < self.min_correlation:
            continue

        pair = prices[[sym_a, sym_b]].dropna()
        price_a = pair[sym_a]
        price_b = pair[sym_b]

        _, pvalue, _ = coint(price_a.values, price_b.values)
        X = sm.add_constant(price_b.to_numpy())
        fit = sm.OLS(price_a.to_numpy(), X).fit()
        intercept, beta = fit.params
        spread = price_a - (intercept + beta * price_b)
        tested.append({
            "symbols": (sym_a, sym_b), "corr": corr,
            "pvalue": pvalue, "beta": beta,
            "halflife": calc_halflife(spread.to_numpy()),
        })

if not tested:
    return []

reject, qvalues, _, _ = multipletests(
    [x["pvalue"] for x in tested], alpha=self.coint_pvalue,
    method="fdr_bh"
)
selected = []
for row, keep, qvalue in zip(tested, reject, qvalues):
    hl = row["halflife"]
    if keep and np.isfinite(hl) and hl <= self.max_halflife:
        selected.append({**row, "qvalue": qvalue})
return sorted(selected, key=lambda x: (x["qvalue"], x["halflife"]))

```

The thing to notice is the three `continue` statements. Each pair has exactly three gates to clear. The output is sorted by p-value in ascending order, so the strongest cointegration relationships sit at the top of the list.

11.3 Calculating the halflife

The **halflife** is the number of days it takes the spread to close half of its distance from the mean. Even when two stocks are cointegrated, a halflife that's too long is not useful in practice. A halflife of 200 days means you'd hold a position for hundreds of days, which means more cost and more risk.

Mathematically, we fit a regression of “how much does the spread snap back tomorrow given how far it is from the mean today?” The pull strength (`phi`) from that regression gives us the halflife. The more negative `phi` is, the stronger the pull, and the shorter the halflife.

```
def calc_halflife(spread, min_obs=20):
    y = np.diff(spread)                #  $\Delta s_t$ 
    x = spread[:-1] - np.mean(spread)
    phi = np.sum(x * y) / np.sum(x * x)
    if phi >= 0:
        return float("inf")
    return float(max(-np.log(2) / phi, 1.0))
```

The number you read out is in days. For example, `phi = -0.03` gives a halflife of about 23 days, meaning the spread covers half the distance back to the mean within 23 days. If `phi` is non-negative, the series isn't mean-reverting, so the function returns the upper bound of 80.0, which effectively removes that pair. The result is clipped to the range 1 to 80 days.

11.4 A dynamic hedge ratio with the Kalman filter

A hedge ratio β fitted once with OLS stays the same forever. But when the businesses or the market environment change, the ratio that ties the two prices together can change too. The **Kalman filter** is a tool that updates β at every time step. Think of it as a weighted average of

“the β estimate so far” and “the new information seen today”—every time a new price arrives, the filter nudges β a little.

The core idea is simple.

- **State:** the current hedge ratio β and an intercept c .
- **Predict:** assume β can drift slightly from one day to the next.
- **Update:** compare today’s actual price to the predicted price and adjust β based on the difference.

Time-varying regression observes $y_t = a_t + \beta_t x_t + \text{noise}$ with state $[a_t, \beta_t]$. Larger Q lets coefficients move faster; larger R trusts each observation less. Apply Chapter 6’s vector prediction/ update, checking β sign, caps, and uncertainty before choosing hedge quantities.

- `kalman_delta = 1e-4`: how fast β can drift. Smaller values keep β stable but make it slow to change.
- `kalman_obs_noise = 1.0`: observation noise. Smaller values mean the filter trusts new prices more.

Each day’s updated β is used to build that day’s spread.

```
spread = log_price_a - beta_arr * log_price_b
```

This spread series is the input for the trading signal in the next chapter (Chapter 12).

11.5 Pair-quality checks

A pair that looks good today may stop being cointegrated later. So right before we trade, we run one more check on its quality.

We look at two things.

1. Is the rolling 250-day correlation at least `corr_min` (= 0.4)?
2. Is the halflife of the most recent 250 bars of the spread at most `hl_max` (= 60 days)?

If a pair passes both checks, we enter, and we cap how long we'll hold the position based on the halflife.

```
if kdata["corr"] < cfg.corr_min:
    return False
if calc_halflife(recent_spreads) > cfg.hl_max:
    return False
dyn_max_hold = min(cfg.max_hold_days, int(cfg.hl_hold_mult * hl))
```

kdata is a precomputed cache of results (Kalman spread, dynamic β , and the 250-day rolling correlation). We don't have to recompute them every time we consider an entry, which keeps things fast.

11.6 Pair-selection parameters at a glance

Skim the meanings now, and come back to the table when you need it.

Parameter	Default	Role
z_window	60	Lookback for the z-score (business days)
z_entry	1.8	Entry z-score threshold
z_exit	0.3	Exit z-score threshold
hard_stop_z	4.5	Hard-stop z-score
kalman_delta	1e-4	Kalman β drift speed
kalman_obs_noise	1.0	Kalman observation noise
max_hold_days	30	Maximum holding period (days)
corr_min	0.4	Minimum correlation at entry
warmup_days	120	Kalman warm-up window
hl_max	60	Maximum halflife at entry (days)
hl_hold_mult	3.0	$\text{max_hold} = 3 \times \text{halflife}$

$z_{\text{entry}} = 1.8$ came out of a backtest search. A value that was best on a particular period is not necessarily a good default, though. In practice you should also check that it doesn't swing wildly when you change trading costs, the symbol universe, or the time period.

11.7 Pair-selection criteria, summarized

Criterion	Value	Purpose
Sector limit	Same sector only	Reduce the multiple testing problem
Correlation	≥ 0.4	Confirm a shared driver
Cointegration p-value	< 0.05	Verify mean reversion of the spread
Halflife	≤ 60 days (at entry)	Keep holding periods reasonable
Kalman δ	$1e-4$	Stable dynamic β estimate

When you pick pairs, **quality matters more than quantity**. A few dozen strongly cointegrated pairs beat hundreds of pairs that barely cleared loose thresholds.

Next chapter (12): We turn the pairs we've chosen into trading signals. We enter when the z-score of the Kalman spread crosses a threshold, then filter the signals once more with a regime check and an RL gate.

Try it yourself —running the pipeline on a synthetic pair

Here's a minimum runnable example that combines the three checks we've covered (correlation, cointegration, halflife) into a single function that judges one pair. The point is to run it on synthetic data so you can see what passing and failing look like. Try running it.

```
import numpy as np
from scipy import stats
from statsmodels.tsa.stattools import coint
```

```

MIN_CORR, MAX_PVALUE, MAX_HALFLIFE = 0.40, 0.05, 60

def estimate_halflife(spread):
    y = spread[1:] - spread[:-1]
    x = spread[:-1] - spread[:-1].mean()
    slope, *_ = stats.linregress(x, y)
    if slope >= 0:
        return float("inf")
    return float(-np.log(2) / slope)

def screen_pair(prices_a, prices_b):
    log_a, log_b = np.log(prices_a), np.log(prices_b)

    ret_corr = np.corrcoef(np.diff(log_a), np.diff(log_b))[0, 1]
    if ret_corr < MIN_CORR:
        return None

    slope, intercept, *_ = stats.linregress(log_b, log_a)
    spread = log_a - slope * log_b - intercept

    _, eg_pvalue, _ = coint(log_a, log_b)
    if eg_pvalue >= MAX_PVALUE:
        return None

    hl = estimate_halflife(spread)
    if hl <= 0 or hl > MAX_HALFLIFE:
        return None
    return {"return_corr": float(ret_corr),
            "eg_p": float(eg_pvalue),
            "hl": hl}

if __name__ == "__main__":
    rng = np.random.default_rng(42)
    common = np.cumsum(rng.normal(0, 1, 300))
    a = np.exp(common * 0.9 + rng.normal(0, 0.2, 300)) * 100
    b = np.exp(common * 1.1 + rng.normal(0, 0.2, 300)) * 150
    print(screen_pair(a, b))

```

The output is a single dictionary of the form `{"corr": ..., "adf_p": ..., "hl": ...}`. If you get `None`, walk through the checks above to find which step rejected the pair. To run it on real market data, just swap in your own price series for `prices_a` and `prices_b`.

Exercises

1. Run a cointegration test on the SPY-QQQ pair. When the p-value comes out close to the entry threshold (0.05), write down how you would use the supporting checks (halflife and correlation) to make a final call.
2. Adapt the `estimate_halflife` code above into a filter that automatically rejects pairs with a halflife under 3 days or over 60 days. Think about why a very short halflife can also be risky.
3. If you re-run the pair selection every quarter and the chosen pairs change, how should you close out positions on pairs that no longer make the list? Factor in the trading costs of switching pairs.

Summary

Three things to take away from this chapter.

Criterion	Why
Same sector	You should be able to explain why two companies move together
Cointegration p-value < 0.05	Statistical evidence that the spread reverts to the mean
Halflife limit	Pairs that revert too slowly aren't tradable in practice

A good pair is more than two stocks that happen to move together. They must sit in the same sector, have a low cointegration p-value, and their spread must mean-revert quickly enough to trade. The **Kalman filter** is the tool we use to track that ratio when it changes over time, and we go deeper into the formulas in the next chapter and in Chapter 19.

In the next chapter, we turn these selected pairs into concrete entry and exit signals.

Further reading

- **Paper:** Engle & Granger (1987), “Co-Integration and Error Correction”—the original cointegration paper (Nobel-winning).
- **Book:** Vidyamurthy, *Pairs Trading* (Wiley, 2004) —practical implementation of statistical-arbitrage strategies.
- **Paper:** Gatev, Goetzmann & Rouwenhorst (2006), “Pairs Trading: Performance of a Relative Value Arbitrage Rule”—an academic backtest.

Deeper theoretical foundations: The math behind the cointegration test and the Kalman hedge ratio in this chapter is covered in depth in Chapters 19-21. Advanced topics —Johansen cross-validation, regime-conditional pair filters —are also discussed there.

Chapter 12. Statistical Arbitrage —Generating Signals

Once you have found a cointegrated pair, the next question is simple. When do you enter, when do you exit, and how do you know the relationship has broken? Those three questions are basically all of real-world implementation.

The simplest rule looks like this: enter when the Z-score of the spread (the standardized measure of how far it has deviated —see the cointegration section of Chapter 11 for the definition) exceeds 2.0, and close out when it returns to zero. When you first learn pairs trading, this feels like enough. In practice, it is not. The market does not tell you why the spread widened. A plain Z-score cannot distinguish between “temporarily stretched” and “the underlying structure has permanently changed.”

The way it fails is clear. The spread can keep widening from 2 to 4 and then to 6. If this is a temporary stretch, you just wait it out. But if one of the two companies posted earnings that changed the relationship itself, the story is different. A naive system mistakes each new extreme for “an even better entry price” and keeps adding to the position. It is averaging down, pairs-trading edition.

Exits have the same problem. The textbook rule is “close out when the spread returns to zero.” But that rule does not take the market environment —the regime (the broad state of the market; we introduced this with the HMM in Chapter 11) —into account. In a high-volatility regime, the spread oscillates wildly near zero rather than crossing it cleanly. A fixed rule just produces extra trades and extra commissions.

Exits come before entries

There is one principle worth keeping in mind throughout this chapter. **Entry rules create opportunities, but exit rules decide your profit and loss.** If your entry is wrong, another opportunity comes. If your exit is late, a single trade carves money out of your account. The sections in this chapter are ordered “entry -> exit,” but mentally you should decide where you will get out first, and only then decide where you will get in. A good signal is not the one that is right most of the time; it is the one that pulls back before being wrong becomes expensive.

The solution in this chapter is a regime-conditional signal. We tune the Z-score thresholds and the exit rules to the market state classified by the HMM. In a stable mean-reverting regime we lower the entry bar to catch more opportunities; in a strong-trend regime we raise it to enter less.

Do not think of this signal as a device that “predicts which stock will go up tomorrow.” It is a bundle of rules that lets the code answer two questions: “Is it safe to enter this pair right now?” and “If the relationship has broken, where do we stop?”

If Chapter 11 decided **what to trade**, Chapter 12 decides **when to enter and when to exit**. It turns a pair candidate into an actual trading signal.

```
Chapter 11 output
candidate pair: K0 / PEP
|
v
```

```

Chapter 12 input
current spread, Z-score, half-life, correlation, volume
    |
    v
Signal generation
enter / wait / exit
    |
    v
Chapters 13–14 RL
read the signal and adjust position size

```

For a beginner, it is dangerous to memorize Z-score as “large means enter.” Z-score is only the starting point. This chapter wraps safety filters around that number.

```

Z-score is large
    |
    +-- Is the pair relationship still intact?
    +-- Has reversion started?
    +-- Are volume and liquidity acceptable?
    +-- Have we held the trade too long?
    +-- Has the loss limit been breached?

```

So Chapter 12 is not just an entry chapter. It is a chapter about designing entry and exit together. When combined with RL, this signal remains important: RL does not enter arbitrary pairs at will. It adjusts position size only in situations that pass this chapter's filters.

What's in this chapter

- Entry prerequisites (5 filters)
- Entry score (`entry_score`) —the multiplicative formula
- Exit conditions (6-step priority order)
- BW-HMM spread regime adjustment
- Flow summary for entry and exit
- Looking at the code
- How to combine with RL (preview)

Learning goals

By the end of this chapter you should be able to:

- explain the flow of the Z-score-based entry/exit logic and the priority order of the exit conditions
- describe what each of the six factors in `entry_score` does
- explain how the BW-HMM regime adjustment changes the entry criterion
- explain why exits are at least as important as entries, and often more so

12.1 Entry prerequisites

Before we compute an entry score, five filters screen out the candidate itself. No matter how high the score, if the candidate fails any one of these five conditions, we do not enter. The score measures “how attractive is it?” The prerequisites measure “is this even a valid candidate?”

Step	Filter	Condition	Intuition
1	Half-life (hl)	$hl \leq 60 \text{ days}$	Drop pairs that revert too slowly
2	Correlation	$corr \geq 0.4$	Drop pairs that don't move together
3	Z-score threshold	$ z_{now} \geq z_{entry}$ (with BW-HMM adjustment)	Only enter when deviation is large enough
4	Convergence direction	$ z_{now} < z_{prev} $	Don't enter while the spread is still widening
5	Volume microstructure	$vol_mult \geq 0.90$	Drop pairs with abnormally thin liquidity

The **key one is filter #4, “convergence direction.”** We don't enter while the spread is still widening; we enter only after it has passed its peak and started to revert. This one line is what stops us from averaging down —repeatedly entering at ever-wider levels.

In step 3, `z_entry` is not fixed; it is dynamically adjusted according to the BW-HMM regime (explained in 12.4). The actual code lives in `_get_effective_z_entry()` in `stat_pair_rl.py`.

12.2 Entry score (entry_score)

WARNING

If you set the Z-score threshold (`z_entry`) too low, trades come too often and commissions pile up. The default is 2.0. Depending on the pair and regime, Optuna tunes it between 1.5 and 2.5 (-> Chapter 15).

For candidates that pass all five prerequisites in 12.1, we compute an entry score. The score is the product of six factors. It is not a measure of “how much money you can make”; it is a quality score that asks “is it safe to enter right now?”

```
entry_score = abs(z_now) * (1.0 / max(hl, 1.0)) * corr * ivr_mult *
conv_boost * vol_mult
```

Each factor means:

Factor	Meaning	Range	Importance
<code>\ z_now\ </code>	Absolute Z-score. Larger deviation -> stronger mean-reversion expectation	0-5+	5/5
<code>1/hl</code>	Inverse of half-life. Shorter -> faster reversion	0-1	4/5
<code>corr</code>	250-day correlation. Quality of cointegration	0-1	3/5

Factor	Meaning	Range	Importance
<code>ivr_mult</code>	Realized-volatility ratio adjustment. We compare short-term to long-term volatility (the IVR) to ask “is the market its usual calm self, or unusually shaky?” If it’s calm, we trust the signal as-is; if it’s shaky, we discount the score	0.5-1.0	3/5
<code>conv_boost</code>	Bonus for convergence momentum	1.0-1.2	2/5
<code>vol_mult</code>	Confirmation from volume microstructure	0.5-1.0	2/5

Why multiplication?

If we added the factors instead of multiplying, a very large Z-score could mask weakness in the other factors. Multiplication is the opposite: when one condition is weak, the whole score drops with it. No matter how big the Z-score is, low correlation will drag the score down.

The most dangerous trap in pairs trading is entering a pair whose underlying relationship has already broken, just because “the spread is wide.” Multiplication is the defense against that. Z-score and half-life are the core signals; the other four factors are risk filters.

A one-line numerical example

Suppose a hypothetical pair has `z=-2.8`, `hl=25`, `corr=0.65`, `ivr_mult=1.0`, `conv_boost=1.1`, `vol_mult=0.95`.

`score = 2.8 × (1/25) × 0.65 × 1.0 × 1.1 × 0.95 ≈ 0.076`

With the same Z and the same half-life but `corr` dropping to 0.4, the score falls to about 0.047. Because it's multiplication, you can see numerically how weakening any single factor pulls the whole score down.

Convergence momentum (`conv_boost`)

If absolute Z over three bars falls 2.8->2.5->2.1, it is moving toward equilibrium. An illustrative score multiplier is 1.2 only for three finite, strictly decreasing values, otherwise 1.0. This is not a probability of completed reversion. Entry thresholds, costs, and position caps still apply.

12.3 Exit conditions (in priority order)

IMPORTANT

Without exit rules, losses keep piling up on pairs that never converge. `max_holding_days` (force a close once the maximum holding period is exceeded) is a safety device you absolutely need in real trading. If the cointegration relationship breaks, the Z-score will not return to its mean. So a time-based stop is required alongside the price stop.

When closing a position, we check the following six conditions **in order**. If an earlier condition fires, we don't check the later ones.

Priority	Condition	Parameter	Role
(1)	<code>price_stop</code>	<code>price_stop_pct = -8.0%</code>	Absolute loss limit
(2)	<code>pair_degrade_hl</code>	<code>hl > 100</code>	Detect cointegration breakdown
(2)	<code>pair_degrade_corr</code>	<code>corr < 0.3</code>	Correlation breakdown

Priority	Condition	Parameter	Role
(3)	<code>z_hard_stop</code>	<code>hard_stop_z = 4.5</code>	Minimize loss on extreme deviation
(4)	<code>rl_exit</code>	<code>rl_exit_threshold_hybrid = -0.6</code>	When the RL action is strongly negative (RL mode only)
(5)	<code>z_converge</code>	<code>z_exit = 0.3</code>	Normal convergence to target
(6)	<code>max_hold</code>	<code>min(30, 3×hl)</code>	Time-based forced close

Each condition in one line:

- **price_stop**: if the loss exceeds -8%, close unconditionally. Stop the bleeding before asking why.
- **pair_degrade_hl / _corr**: half-life > 100 days, or correlation < 0.3 -> the pair itself has broken.
- **z_hard_stop**: Z-score > 4.5 -> extreme deviation. Don't wait any longer; close.
- **z_converge**: Z-score <= 0.3 -> goal reached. Normal convergence exit.
- **max_hold**: even if none of the above triggers, force a close after `min(30 days, 3×hl)`.

This sequence is the priority order. Loss containment (`price_stop`) comes first; time-based force-out (`max_hold`) comes last. The point is to prevent the failure mode where you wait for a “normal” exit signal while the loss grows from -10% to -15%.

Which condition fires first —two scenarios

Scenario A —stop-loss fires first. You enter pair A at `z=-2.5`. On the next bar, an earnings shock on one of the two companies widens the spread further and the loss reaches -9%. The

Z-score is still at -3.2 , nowhere near the exit threshold ($|z| < 0.3$). Priority (1) `price_stop` triggers first, and we never reach (5) `z_converge`. Without a loss limit, you would not have escaped before the loss got even deeper.

Scenario B —Z-convergence fires first. You enter pair B at $z = -2.7$. Over a week the prices drift slowly: PnL is $+1.5\%$ and the Z-score moves cleanly through $-2.7 \rightarrow -1.8 \rightarrow -0.9 \rightarrow -0.2$. None of (1) (2) (3) (4) trigger, and the position closes at (5) `z_converge` ($|z| < 0.3$). This is a normal-convergence exit; we never reach (6) `max_hold`.

The actual implementation (an `if-elif` chain) is in `stat_pair_rl.py`.

12.4 BW-HMM spread regime adjustment

We apply an HMM to the spread itself to distinguish a mean-reverting (MR) regime from a trending regime. The same Z-score means different things in different regimes. A large Z in an MR regime is an opportunity; a large Z in a trending regime can be a signal that the relationship is breaking.

So in the MR regime we lower `z_entry` to make entry easier, and in the trending regime we raise it to suppress entries.

Parameter	Default	Effect
<code>spread_bw_mr_boost</code>	0.15	MR regime: <code>z_entry</code> $\times$ 0.85 (entry eased)
<code>spread_bw_trend_penalty</code>	0.25	Trend regime: <code>z_entry</code> $\times$ 1.25 (entry suppressed)

For example, `z_entry=2.0` becomes 1.7 in the MR regime and 2.5 in the trending regime. In the trending regime, the exit threshold and the holding period are also tightened (`spread_bw_trend_exit`, `spread_bw_trend_hold_mult`); see the appendix and the `_RLConfig` definition for those two parameters. The adjustment function itself is `_get_effective_z_entry()`.

12.5 Flow summary for entry and exit

Every time a bar closes, the following two steps run in order. We first decide whether to close any open positions, then pick candidates for new positions.

Step	Item	Condition / action
EXIT (close existing positions)	1. price_stop	PnL -8%
EXIT	2. pair_degrade	hl>100 or corr<0.3
EXIT	3. z_hard_stop	z >4.5
EXIT	4. RL exit	rl_full / rl_hybrid, after min_hold=5 bars
EXIT	5. z_converge	z <0.3 or Z sign flip
EXIT	6. max_hold	min(30 bars, 3×hl)
ENTRY (open new positions)	5 prerequisites	hl≤60, corr≥0.4, z ≥z_entry (adjusted), Z decreasing, vol_mult≥0.90
ENTRY	Score	$ z \times 1/hl \times \text{corr} \times \text{ivr_mult} \times \text{conv_boost} \times \text{vol_mult}$
ENTRY	Final selection	Sort by score descending → enter the top max_positions=30

The exit checks run top-down. The first matching condition closes the position immediately and skips the rest. For entries, only candidates that pass all prerequisites are ranked by score, and we take the top of the list.

12.6 Looking at the code —entry score and prerequisite checks

Let's run the entry logic we have built up so far as two short functions. The real `stat_pair_rl.py` is much longer, but the core decisions live in these two functions. The step numbers match the prerequisite table in 12.1.

```

import numpy as np

Z_ENTRY, HL_MAX, CORR_MIN, VOL_MIN = 2.0, 60, 0.40, 0.90

def compute_entry_score(z, hl, corr, ivr=1.0, conv=1.0, vol=1.0):
    """abs(z) × 1/hl × corr × IVR × convergence_boost × volume (higher
    is a stronger candidate)"""
    return abs(z) * (1.0 / max(hl, 1.0)) * corr * ivr * conv * vol

def check_entry(z, z_prev, hl, corr, vol_mult, z_entry_adj=Z_ENTRY):
    """Five entry checks from 12.1; return (go, reason)."""

    if hl > HL_MAX:
        return False, f"hl={hl}>{HL_MAX}"

    if corr < CORR_MIN:
        return False,
        f"corr={corr:.2f}<{CORR_MIN}"

    if abs(z) < z_entry_adj:
        return False,
        f"abs(z)<{z_entry_adj}"

    if abs(z) >= abs(z_prev):
        return False, "z still rising"

    if vol_mult < VOL_MIN:
        return False,
        f"vol_mult={vol_mult:.2f}<{VOL_MIN}"
    return True, "ok"

if __name__ == "__main__":
    cases = [
        ("entry accepted", dict(z=-2.8, z_prev=-3.1, hl=25,
        corr=0.65, vol_mult=0.95)),
        ("half-life too long", dict(z=-2.5, z_prev=-2.9, hl=80,
        corr=0.60, vol_mult=0.95)),
        ("weak correlation", dict(z=-2.6, z_prev=-3.0, hl=30,
        corr=0.30, vol_mult=0.95)),
        ("Z below threshold", dict(z=-1.5, z_prev=-1.8, hl=30,
        corr=0.70, vol_mult=0.95)),
        ("Z increasing", dict(z=-3.0, z_prev=-2.5, hl=30, corr=0.65,
        vol_mult=0.95)),
        ("insufficient volume", dict(z=-2.8, z_prev=-3.1, hl=25,
        corr=0.65, vol_mult=0.80)),
    ]
    for label, c in cases:

```

```

    go, reason = check_entry(**c)
    score = compute_entry_score(c["z"], c["hl"], c["corr"],
vol=c["vol_mult"]) if go else 0.0
    print(f"{label:<10} go={go!s:<5} score={score:.4f}
          reason={reason}")

```

What to check in the output: only “normal entry” should come back with `go=True`. The other five should be rejected, each for a different reason printed in `reason`, showing exactly which step blocked them. If every case passes, your thresholds are too loose or a filter is missing.

12.7 How to combine with RL (preview of the next chapter)

So far we’ve looked at the baseline (`rl_stat_only`) that decides entries and exits purely from statistical rules. The real `stat_pair_rl.py` supports four more modes that combine these rules with reinforcement learning (RL). The reason we split modes is simple: if you change the statistical rules and the RL at the same time, you cannot tell which side moved performance.

Mode	Entry	Exit	Description
<code>rl_stat_only</code>	Stats (Z-score)	Stats	Pure stat-arb baseline (what this chapter covered)
<code>rl_veto</code>	Stats -> RL veto	Stats	RL blocks risky entries
<code>rl_hybrid</code>	Stats -> RL tweak	RL	Stats entry + RL exit
<code>rl_entry</code>	RL	Stats	RL entry + stats exit
<code>rl_full</code>	RL	RL	Full RL control

The default is `rl_veto`. The statistical rules pick the entry candidates first, and the RL only vetoes the ones it considers risky. The veto threshold (`rl_veto_threshold = -0.9`) is deliberately set low so that the RL cannot shake the statistical baseline without a strong reason. The meaning of the RL action, the structure of the state vector, and the reward function are covered in Chapter 13.

If you are building this for the first time, run `rl_stat_only` to establish a baseline, then turn on `rl_veto` and compare. That is the only way to know what caused the change.

Deferred to a later chapter: tuning the RL veto threshold, the structure of the 28-dimensional RL state vector, and the reward design are picked up in Chapter 13 (RL environment design).

Wrap-up

Three things to remember from this chapter:

1. **Exits matter more than entries.** A good signal is not the one that is right most often, but the one that pulls back fast when it is wrong. You should be able to recall the six-step exit priority in your head: loss containment -> pair breakdown -> extreme deviation -> RL -> normal convergence -> time.
2. **entry_score is a product.** If any one of the six factors is weak, the whole score drops. It is a defense against entering an already-broken pair just because Z looks large.
3. **The same Z means different things in different regimes.** A large Z in a mean-reverting regime is an opportunity; in a trending regime it can be a warning. The BW-HMM adjustment encodes that difference in the entry criterion.

The key parameters at a glance:

Component	Parameter	Role
z_entry	2.0 (1.5-2.5 in sweeps)	MR entry trigger
z_exit	0.3	Convergence exit
hard_stop_z	4.5	Extreme-loss cut-off
price_stop_pct	-8%	Absolute loss limit
BW-HMM adjustment	mr_boost=0.15, trend_penalty=0.25	Adjusts z_entry per spread regime
conv_boost	max 1.2	Favor pairs already reverting

Component	Parameter	Role
max_hold	min(30, 3×hl)	Adaptive holding period

Next chapter (Chapter 13): RL environment design. We’ll cover the state space (28 dimensions) of `StatPairRLEnv`, the (continuous) action space, the reward function, and what the `rl_veto` threshold actually means.

Exercises

1. Grid-search the `z_entry` threshold from 1.5 to 2.5 in 0.25 steps. Tabulate the performance, the number of trades per year, and the average win/loss for each setting.
2. Run the same backtest with `conv_boost` (the convergence-momentum adjustment) pinned to 0 and again pinned to its maximum, and compare the 1-year PnL curves. In what kind of market regime is this parameter’s effect most visible?
3. Take the 2019–2023 backtest logs and find which of the six exit conditions fires most often. Add code that prints the fraction of trades closed by each condition.

Further reading

- **Paper:** Avellaneda & Lee (2010), “Statistical Arbitrage in the U.S. Equities Market”—Z-score-based signal design
- **Book:** Chan, *Quantitative Trading —Implementation and risk management of statistical arbitrage*
- **Paper:** “Momentum and Mean Reversion in Strategic Asset Allocation”—Theoretical background for the convergence-momentum adjustment